
Library for Time Series Anomaly Detection Evaluation Metrics (TSADmetrics)

User Manual
TSADmetrics version 1.0
May 24, 2026

Pedro Rafael Velasco Priego
Amelia Zafra Gómez

This user guide is for the TSADmetrics library (version 1.0). Copyright ©2025 Pedro Rafael Velasco Priego and Amelia Zafra Gómez.

Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 3.0 or any later version published by the Free Software Foundation; with no Invariant Sections, with no Front-Cover Texts, and with no Back-Cover Texts.

Contents

Acronyms	1
1 Introduction	3
2 Preliminary	5
2.1 Time Series Fundamentals	5
2.2 Time Series Anomaly Detection	6
2.3 Evaluation Metrics for Anomaly Detection Methods	6
2.3.1 Point-wise Evaluation	6
2.3.2 Event-level Evaluation	7
3 Getting and running the library	9
4 TSADmetrics library	11
4.1 TSADmetrics library architecture	12
5 Run available Metrics in the Library	15
5.1 Single-Point Based Metrics	15
5.1.1 Point-wise F-Score	16
5.1.1.1 Direct evaluation via PointwiseFScore Class	16
5.1.1.2 Automated evaluation via Runner Class	17
5.1.2 Dice Coefficient	17
5.1.2.1 Direct evaluation via DiceCoefficient Class	17
5.1.2.2 Automated evaluation via Runner Class	18
5.1.3 Point-wise AUC-ROC	18
5.1.3.1 Direct evaluation via PointwiseAucRoc Class	19
5.1.3.2 Automated evaluation via Runner Class	19
5.1.4 Point-wise AUC-PR	19

5.1.4.1	Direct evaluation via PointwiseAucPr Class	20
5.1.4.2	Automated evaluation via Runner Class	20
5.1.5	Precision@K	20
5.1.5.1	Direct evaluation via PrecisionAtK Class	21
5.1.5.2	Automated evaluation via Runner Class	21
5.2	Temporal Evaluation Metrics	22
5.2.1	Tolerant Partial Detection Metrics	22
5.2.1.1	Point-adjusted F-Score	22
5.2.1.1.1	Direct evaluation via PointadjustedFScore Class	23
5.2.1.1.2	Automated evaluation via Runner Class	23
5.2.1.2	Balanced Point-adjusted F-Score	23
5.2.1.2.1	Direct evaluation via BalancedPointadjustedFScore Class	24
5.2.1.2.2	Automated evaluation via Runner Class	25
5.2.1.3	Segment-wise F-Score	25
5.2.1.3.1	Direct evaluation via SegmentwiseFScore Class	26
5.2.1.3.2	Automated evaluation via Runner Class	26
5.2.1.4	Composite F-Score	26
5.2.1.4.1	Direct evaluation via CompositeFScore Class	27
5.2.1.4.2	Automated evaluation via Runner Class	27
5.2.1.5	Point-adjusted AUC-PR	28
5.2.1.5.1	Direct evaluation via PointadjustedAucPr Class	28
5.2.1.5.2	Automated evaluation via Runner Class	28
5.2.1.6	Point-adjusted AUC-ROC	29
5.2.1.6.1	Direct evaluation via PointadjustedAucRoc Class	29
5.2.1.6.2	Automated evaluation via Runner Class	30
5.2.1.7	Range-based F-Score	30
5.2.1.7.1	Range-based Recall	31
5.2.1.7.2	Range-based Precision	32
5.2.1.7.3	Definition and Configuration of the Functions γ , ω , and δ	32
5.2.1.7.3.1	Overlap Function $\omega()$	32
5.2.1.7.3.2	Positional Bias Function $\delta()$	32
5.2.1.7.3.3	Cardinality Function $\gamma()$	33
5.2.1.7.4	Summary of parameters and implementation.	33

5.2.1.7.5	Direct evaluation via RangebasedFScore Class	34
5.2.1.7.6	Automated evaluation via Runner Class	34
5.2.2	Delay-Penalized Metrics	34
5.2.2.1	Delay-thresholded Point-adjusted F-Score	35
5.2.2.1.1	Direct evaluation via DelayThresholdedPointadjustedFScore Class	35
5.2.2.1.2	Automated evaluation via Runner Class	36
5.2.2.2	Early Detection Score	36
5.2.2.2.1	Direct evaluation via EarlyDetectionScore Class	37
5.2.2.2.2	Automated evaluation via Runner Class	37
5.2.2.3	Latency-Sparsity-aware F-Score	37
5.2.2.3.1	Direct evaluation via LatencySparsityawareFScore Class	38
5.2.2.3.2	Automated evaluation via Runner Class	38
5.2.2.4	Mean Time to Detect	39
5.2.2.4.1	Direct evaluation via MeanTimeToDetect Class	40
5.2.2.4.2	Automated evaluation via Runner Class	40
5.2.2.5	NAB Score	40
5.2.2.5.1	Direct evaluation via NabScore Class	42
5.2.2.5.2	Automated evaluation via Runner Class	42
5.2.3	Precise Temporal Detection Metrics	42
5.2.3.1	Average Detection Count	43
5.2.3.1.1	Direct evaluation via AverageDetectionCount Class	43
5.2.3.1.2	Automated evaluation via Runner Class	44
5.2.3.2	Counting metrics	44
5.2.3.2.1	Total Detected in Range	45
5.2.3.2.1.1	Direct evaluation via TotalDetectedInRange Class	45
5.2.3.2.1.2	Automated evaluation via Runner Class	45
5.2.3.2.2	Detection Accuracy in Range	46
5.2.3.2.2.1	Direct evaluation via DetectionAccuracyInRange Class	46
5.2.3.2.2.2	Automated evaluation via Runner Class	47
5.2.3.2.3	Weighted Detection Difference	47
5.2.3.2.3.1	Direct evaluation via WeightedDetectionDifference Class	48

5.2.3.2.3.2	Automated evaluation via Runner Class	49
5.2.3.3	Point-adjusted at K% F-Score	49
5.2.3.3.1	Direct evaluation via PointadjustedAtKFScore Class	50
5.2.3.3.2	Automated evaluation via Runner Class	50
5.2.3.4	Point-adjusted at K% F-Score with tolerance l	50
5.2.3.4.1	Direct evaluation via PointadjustedAtKLFScore Class	51
5.2.3.4.2	Automated evaluation via Runner Class	51
5.2.3.5	Timeseries-Aware F-Score	52
5.2.3.5.1	Direct evaluation via TimeseriesAwareFScore Class	53
5.2.3.5.2	Automated evaluation via Runner Class	53
5.2.4	Temporal Matching Evaluation Metrics	53
5.2.4.1	Absolute Detection Distance	54
5.2.4.1.1	Direct evaluation via AbsoluteDetectionDistance Class	54
5.2.4.1.2	Automated evaluation via Runner Class	55
5.2.4.2	Enhanced Timeseries-Aware F-Score	55
5.2.4.2.1	Direct evaluation via EnhancedTimeseriesAwareFScore Class	56
5.2.4.2.2	Automated evaluation via Runner Class	56
5.2.4.3	Temporal Distance	57
5.2.4.3.1	Direct evaluation via TemporalDistance Class	57
5.2.4.3.2	Automated evaluation via Runner Class	58
5.2.5	Temporal Shift-Tolerant Metrics	58
5.2.5.1	Affiliation-based F-Score	58
5.2.5.1.1	Direct evaluation via AffiliationbasedFScore Class	59
5.2.5.1.2	Automated evaluation via Runner Class	60
5.2.5.2	Normalized Affiliation-based F-Score	60
5.2.5.2.1	Direct evaluation via NormalizedAffiliationbasedFScore Class	61
5.2.5.2.2	Automated evaluation via Runner Class	61
5.2.5.3	Pate F-Score	61
5.2.5.3.1	Event Categorization	62
5.2.5.3.2	Predicted anomalous event categories Q_i :	62
5.2.5.3.3	Real anomalous event categories R_i :	62
5.2.5.3.4	Point-wise Weight Assignment	62
5.2.5.3.5	Final Score Computation	63

5.2.5.3.6	Direct evaluation via PateFScore Class	63
5.2.5.3.7	Automated evaluation via Runner Class	64
5.2.5.4	Pate	64
5.2.5.4.1	Direct evaluation via Pate Class	65
5.2.5.4.2	Automated evaluation via Runner Class	65
5.2.5.5	Time-Tolerant F-Score	66
5.2.5.5.1	Direct evaluation via TimeTolerantFScore Class	66
5.2.5.5.2	Automated evaluation via Runner Class	67
5.2.5.6	Volume-under-Surface ROC (VUS-ROC)	67
5.2.5.6.1	Label smoothing	67
5.2.5.6.2	Redefinition of confusion matrix quantities	67
5.2.5.6.3	Precision and recall computation	67
5.2.5.6.4	Threshold exploration	68
5.2.5.6.5	Final metric computation	68
5.2.5.6.6	Summary of parameters and implementation.	68
5.2.5.6.7	Direct evaluation via VusRoc Class	68
5.2.5.6.8	Automated evaluation via Runner Class	69
5.2.5.7	Volume-under-Surface PR (VUS-PR)	69
5.2.5.7.1	Label smoothing	69
5.2.5.7.2	Confusion matrix adaptation	69
5.2.5.7.3	Precision and recall	69
5.2.5.7.4	Threshold and smoothing exploration	70
5.2.5.7.5	Final metric computation	70
5.2.5.7.6	Summary of parameters and implementation.	70
5.2.5.7.7	Direct evaluation via VusPr Class	70
5.2.5.7.8	Automated evaluation via Runner Class	71

6	Carry out an experimental evaluation of Anomaly Detection Metrics	73
6.1	Input artifacts and naming conventions	73
6.1.1	Result files	73
6.1.1.1	Binary predictions.	74
6.1.1.2	Binary and continuous predictions.	74
6.1.2	Metric configuration file	74
6.1.3	Global evaluation configuration	74
6.1.4	Dataset identifiers and paths	75
6.2	Runner Class	75
6.2.1	Parameters	75
6.2.2	Running evaluation	76
6.2.3	Reports	76
6.3	Evaluation Examples	77
6.3.0.1	Example 1: Direct data evaluation	77
6.3.1	Example 2: Direct binary data evaluation	78
6.3.2	Example 3: Using external result files	78
6.3.3	Example 4: Using a global configuration file	79
6.3.4	Example 5: Using a metric configuration file	79
6.3.5	Example 6: Computing a single metric manually	79
7	Add a New Metric	81
7.1	Metric Implementation Process	81
7.2	Example Implementation	82
8	Reporting bugs	83
9	Api Reference	85
	Bibliography	125

Acronyms

SPM: Single-Point Based Metrics

TEM: Temporal Evaluation Metrics

TPDM: Tolerant Partial Detection Metrics

PTDM: Precise Temporal Detection Metrics

DPM: Delay-Penalized Metrics

TSTM: Temporal Shift-Tolerant Metrics

TMEM: Temporal Matching Evaluation Metrics

AUC: Area Under Curve

ROC: Receiver Operating Characteristic

PR: Precision-Recall

TP: True Positive

FP: False Positive

TN: True Negative

FN: False Negative

Introduction

In recent years, the exponential growth of real-time data streams has increased the relevance of anomaly detection in time series. This task plays a critical role in multiple domains such as industrial monitoring [1], financial analysis [2], or the supervision of critical infrastructures [3]. Unlike anomalies in static data, temporal anomalies often span across a duration, meaning that abnormal behavior persists for a certain period of time. This temporal nature introduces unique challenges for evaluating anomaly detection models, as traditional metrics may fail to properly capture whether a method detects and correctly covers the entire anomalous interval.

The literature has introduced a wide range of evaluation metrics for time series anomaly detection [4, 5, 6]. However, there is still little consensus regarding which metrics are most suitable for each scenario. Metrics differ not only in their properties and what aspect of performance they highlight, but also in their applicability depending on the specific goals of the task. For instance, some metrics prioritize early detection of an anomaly onset, while others evaluate whether the entire duration of the anomaly is accurately covered. This diversity complicates fair comparisons between models and hinders the informed selection of appropriate evaluation strategies.

TSADmetrics addresses this gap by providing a unified Python library for time series anomaly detection metrics. The library integrates a comprehensive collection of 34 metrics and organizes them through a proposed taxonomy that categorizes metrics according to their unique properties and the specific performance dimensions they evaluate. By combining implementation and systematic organization, TSADmetrics enables users to navigate the complex landscape of evaluation metrics with clarity and efficiency.

The library offers a consistent framework for evaluating models, supporting both binary and continuous predictions, and includes automated tools for comparative studies. It is complemented with detailed documentation, usage examples, and report generation utilities, making it practical for both research and applied projects. With TSADmetrics, users can systematically assess anomaly detection methods, understand the trade-offs between metrics, and select those most aligned with their application needs.

The remainder of this document is structured as follows. Chapter 2 introduces key concepts in time series anomaly detection and classical classification metrics. Chapter 3 describes how to install the library and its dependencies. Chapter 4 presents the library’s design, architecture, and main classes. Chapter 5 overviews the implemented metrics and their taxonomy, while Chapter 6 explains how to use them for evaluations. Chapter 7 details the process for adding new metrics. Finally, Chapter 8 covers bug reporting, and Chapter 9 provides the API reference.

Preliminary

This chapter introduces the fundamental concepts required to understand the evaluation of anomaly detection methods in time series. First, we define what a time series is and its formal representation. Then, we describe the specific problem of time series anomaly detection, including how ground truth and model predictions are represented. Finally, we review the classical classification metrics traditionally used in this context and discuss their limitations for temporal anomaly detection.

2.1 Time Series Fundamentals

A **time series** is a sequence of observations collected over time, typically measured at regular or quasi-regular intervals. Formally, a univariate time series can be represented as:

$$X = \{x_1, x_2, \dots, x_n\}, \quad x_i \in \mathbb{R} \quad (2.1)$$

where x_i denotes the observed value at time step i , and n is the total number of observations.

In many real-world scenarios, multiple variables are observed simultaneously, leading to a **multivariate time series**:

$$X = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\}, \quad \mathbf{x}_i \in \mathbb{R}^d \quad (2.2)$$

where each $\mathbf{x}_i$ is a d -dimensional vector containing the values of d variables recorded at the same time step.

Time series data often exhibit complex temporal dependencies, such as trends, seasonality, and autocorrelation. These characteristics distinguish time series from independent and identically distributed (i.i.d.) data, requiring specialized modeling and evaluation techniques.

2.2 Time Series Anomaly Detection

Time series anomaly detection (TSAD) refers to the task of identifying abnormal patterns or behaviors within sequential data over time. Unlike static datasets, where anomalies typically appear as isolated points, anomalies in time series often emerge as **events**: contiguous intervals of observations that deviate from the expected temporal pattern. Detecting these events is crucial because they represent meaningful changes in system dynamics rather than single outliers.

In the context of TSAD, each time step is associated with a corresponding label indicating whether it belongs to normal or anomalous behavior. Formally, the ground truth can be represented as:

$$T = \{T_1, T_2, \dots, T_n\}, \quad T_i \in \{0, 1\} \quad (2.3)$$

where $T_i = 1$ denotes that the i -th time step is anomalous, and $T_i = 0$ indicates normal operation. While this point-wise representation is useful, the ultimate goal is to identify **interval-based anomalies**, i.e., sequences of consecutive time steps forming anomalous events.

Depending on the detection model, predictions may be expressed as:

- **Binary predictions** ($P_i \in \{0, 1\}$), directly classifying each time step as anomalous or normal.
- **Continuous predictions** ($P_i \in [0, 1]$), representing the probability or confidence that the i -th time step is anomalous.

From the binary ground truth T , we can derive the set $R = \{R_1, R_2, \dots, R_m\}$, where each R_j represents a contiguous interval of anomalous time steps (i.e., an **anomalous event**). Similarly, from the predicted sequence P , after applying a threshold if necessary, we obtain $Q = \{Q_1, Q_2, \dots, Q_k\}$, where each Q_i denotes a predicted anomalous event. Evaluating detection performance at the event level requires considering duration, temporal alignment, and overall deviation from the reference pattern.

2.3 Evaluation Metrics for Anomaly Detection Methods

The evaluation of anomaly detection methods in time series has evolved substantially over time. Early approaches treated the task as a binary classification problem, directly comparing predicted and true labels at each time step. While this perspective is conceptually simple and compatible with well-established metrics, it fails to capture the temporal nature of anomalies that unfold as continuous events. Consequently, numerous alternative metrics have been proposed to address different temporal characteristics of anomalies, yet these efforts often lack a unified framework. This section provides an overview of both perspectives.

2.3.1 Point-wise Evaluation

Historically, time series anomaly detection has often been approached as a **binary classification** problem. In this formulation, each time step is treated as an independent observation, analogous to a sample in a traditional classification task. The model's prediction at each time step is compared with the corresponding ground-truth label, disregarding any temporal dependencies.

This approach is suitable for certain types of anomalies—particularly **point anomalies**—where the presence or absence of an anomaly can be determined independently for each observation. Under this perspective, performance is evaluated using well-established metrics derived from the **confusion matrix** (Table 2.1), which compares predicted labels with true labels.

	Predicted: Anomalous	Predicted: Normal
True: Anomalous	TP	FN
True: Normal	FP	TN

Table 2.1: Confusion matrix for binary classification in anomaly detection.

From this matrix, the following classical metrics are commonly derived:

- **Precision:** the proportion of predicted anomalous instances that are truly anomalous.

$$\text{Precision} = \frac{TP}{TP + FP} \quad (2.4)$$

- **Recall:** the proportion of truly anomalous instances correctly detected by the model.

$$\text{Recall} = \frac{TP}{TP + FN} \quad (2.5)$$

- **F_β -Score:** the weighted harmonic mean of precision and recall, controlled by a parameter β . For $\beta = 1$, the F_1 -Score equally balances both aspects.

$$F_\beta\text{-Score} = (1 + \beta^2) \cdot \frac{\text{Precision} \cdot \text{Recall}}{\beta^2 \cdot \text{Precision} + \text{Recall}} \quad (2.6)$$

- **Accuracy:** the overall proportion of correctly classified time steps.

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + TN + FN} \quad (2.7)$$

These metrics are widely adopted due to their simplicity and interpretability. However, their direct application to time series anomaly detection presents notable limitations [7, 8, 9]. Because classical metrics treat each time step independently, they fail to capture temporal dependencies and structural aspects of anomalies. In particular, they do not consider:

- The continuity and duration of detected events.
- The coverage of the full anomaly interval.
- The temporal alignment between predicted and true events (e.g., anticipation or delay).

As a result, evaluations based solely on these metrics may lead to misleading conclusions about model performance [10]. Moreover, due to the strong class imbalance typical of anomaly detection tasks, accuracy often becomes uninformative: a model labeling all time steps as normal may achieve high accuracy while failing to detect any anomalies. Consequently, precision, recall, and F_β -score are usually preferred, though still insufficient for capturing the full temporal nature of anomalies.

2.3.2 Event-level Evaluation

To overcome the shortcomings of classical metrics, a wide range of **temporal-aware evaluation metrics** have been proposed. These metrics aim to explicitly account for the temporal structure of anomalies and to assess detection performance at the **event level** rather than at the individual point level [4, 5, 6, 11].

Different proposals emphasize complementary aspects of temporal evaluation, such as:

- **Event-level detection:** measuring whether each anomalous event is correctly identified.
- **Partial coverage:** quantifying how much of the anomaly duration is detected.
- **Early or delayed detection:** penalizing late responses or rewarding timely predictions.
- **Tolerance to misalignment:** allowing minor shifts in time without penalizing detections excessively.

While these contributions have enriched the evaluation landscape, they have also led to a fragmented ecosystem of metrics, each tailored to specific datasets or use cases. The absence of a clear taxonomy and unified implementation complicates the comparison of results across studies and hinders reproducibility.

TSADmetrics addresses this challenge by providing a unified framework that consolidates existing evaluation metrics for time series anomaly detection under a common taxonomy and standardized interface. By organizing metrics according to their core characteristics and computational principles, this library facilitates consistent experimentation and enables researchers to select the most suitable metrics for the specific aspects of anomaly detection they aim to evaluate.

Getting and running the library

This section describes the necessary steps to download, install, and configure everything you need to use the **TSADmetrics** library.

Table 3.1 specifies the versions of dependencies required by TSADmetrics.

Dependency	Version
joblib	1.4.2
numpy	1.24.4
packaging	24.0
pandas	2.0.3
PATE	0.1.1
patsy	0.5.6
python-dateutil	2.9.0.post0
pytz	2024.1
PyYAML	6.0.2
scikit-learn	1.3.2
scipy	1.10.1
six	1.16.0
statsmodels	0.14.1
threadpoolctl	3.5.0
tsadmetrics	1.0.7
tzdata	2024.1

Table 3.1: Versions of dependencies used by the TSADmetrics library

The library is distributed via GitHub, available at <https://github.com/kdis-lab/TSADmetrics>. The repository contains the following main folders:

- *docs*. Contains the documentation of the library, including the user’s manual in *pdf* format and the API reference in *html*.
- *tsadmetrics*. Contains the Python source code of the library.

- *examples*. Provides usage examples, including how to evaluate multiple metrics automatically, as well as specific examples for the use of each individual metric.
- *tests*. Contains unit tests to verify the correct functionality of the library.

To install the library and its dependencies, it is recommended to use **pip**:

```
$ pip install tsadmetrics
```

Alternatively, if working directly from the repository, the library can be installed in development mode using:

```
$ git clone https://github.com/kdis-lab/TSADmetrics.git
$ cd TSADmetrics
$ pip install -e .
```

TSADmetrics library

TSADmetrics is a Python library that provides a unified framework for evaluating anomaly detection models in time series, with a focus on event-based evaluation. The library is built upon the collection of metrics gathered and systematized in [11], and extends this work by offering a practical and consistent implementation that researchers and practitioners can directly use in their experiments. In addition, it introduces new event-oriented measures and provides a specialized taxonomy that categorizes metrics according to the criteria they consider for detecting anomalous events, such as temporal alignment, duration, and overlap.

TSADmetrics has been designed to be intuitive and flexible, allowing users to evaluate both binary and continuous predictions in a wide variety of anomaly detection scenarios. By integrating a taxonomy of metrics that categorizes them according to their properties and evaluation goals, the library simplifies the selection of appropriate metrics and ensures reproducible and comparable results across different studies.

These are the main remarkable features of the library:

- Provides a comprehensive implementation of 28 time series anomaly detection metrics.
- Organizes the metrics into a structured taxonomy that classifies them by the aspects that best capture the quality of detecting anomalous events, considering factors such as duration, temporal alignment, and interval overlap.
- Includes functions to evaluate multiple metrics simultaneously, simplifying the assessment of models from different perspectives.
- Provides tools to automatically generate comprehensive evaluation reports with aggregated results.
- Offers extensive documentation, usage tutorials, and ready-to-use examples to help users apply the metrics in real-world scenarios.
- The input data, which contains both ground truth and predicted labels, supports several formats, making the evaluation process applicable in different context once the algorithm results are available.

In this section, the library architecture, the data format, and the main functionalities are explained. Moreover, guidance is provided on how to develop a new metric using the features and structure offered by the library.

4.1 TSADmetrics library architecture

The library has been developed in open-source Python. It is organized in modules, with all modules containing the classes and functions required to extend its functionality. Therefore, it is easy to develop new evaluation metrics or utility functions. Below, the main functionality of each module is described:

- *base*. Contains the **Metric** base class from which all metrics in the library inherit. This ensures a consistent interface and simplifies the development of new metrics.
- *evaluation*. Includes the **Runner** class, which is responsible for evaluating multiple metrics on multiple datasets, and the **Report** class, which handles the generation of result reports.
- *utils*. Contains auxiliary functions and utilities that are used generally across metrics for tasks such as data processing, interval handling, and numerical operations.
- *metrics*. Implements all the metrics included in TSADmetrics and provides the **Registry** class, which facilitates the instantiation and management of metrics.
- *metrics.spm*. Contains the implementation of Single-Point Based Metrics (SPM).
- *metrics.tem*. Contains the implementation of Temporal Evaluation Metrics (TEM).
- *metrics.tem.tpdm*. Contains the implementation of Tolerant Partial Detection Metrics (TPDM).
- *metrics.tem.ptdm*. Contains the implementation of Precise Temporal Detection Metrics (PTDM).
- *metrics.tem.dpm*. Contains the implementation of Delay-Penalized Metrics (DPM).
- *metrics.tem.tstm*. Contains the implementation of Temporal Shift-Tolerant Metrics (TSTM).
- *metrics.tem.tmem*. Contains the implementation of Temporal Matching Evaluation Metrics (TMEM).

Figure 5.6 illustrates the overall architecture of the **TSADmetrics** library, showing its main modules and their organization.

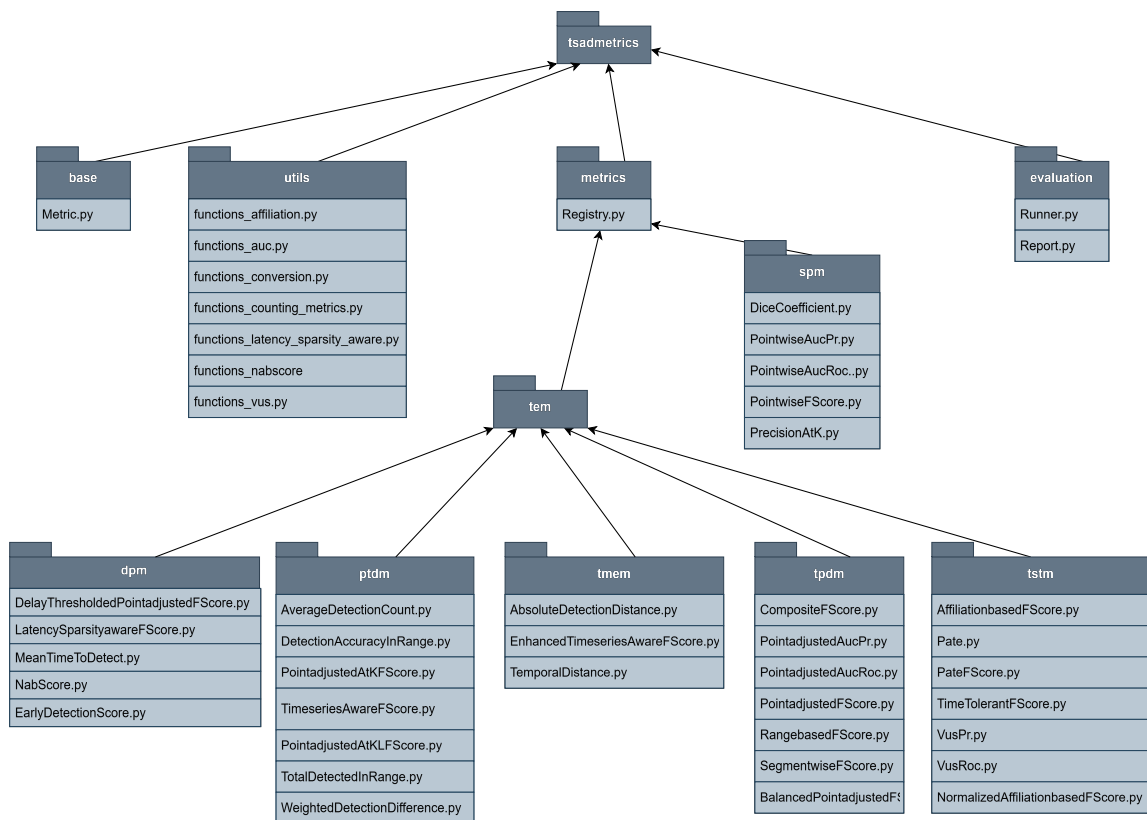


Figure 4.1: Architecture TSADmetrics library

Run available Metrics in the Library

In this chapter, we present the individual metrics implemented in the **TSADmetrics** library. For each metric, we provide a brief description of its purpose and properties, followed by two illustrative examples: one demonstrating how to evaluate the metric manually using the corresponding Python class, and another showing how to perform automated evaluation over multiple datasets using the **Runner** class.

The metrics are organized according to the taxonomy introduced in the library, which includes Single-Point Based Metrics (SPM) in Section 5.1, Temporal Evaluation Metrics in Section 5.2, Tolerant Partial Detection Metrics (TPDM) in Section 5.2.1, Delay-Penalized Metrics (DPM) in Section 5.2.2, Precise Temporal Detection Metrics (PTDM) in Section 5.2.3, Temporal Matching Evaluation Metrics (TMEM) in Section 5.2.4, and Temporal Shift-Tolerant Metrics (TSTM) in Section 5.2.5.

5.1 Single-Point Based Metrics

This category includes metrics that evaluate predictions by considering each point of the ground truth T and predicted values P independently, without taking into account the temporal context in which anomalies occur. In other words, these metrics treat each time instant in isolation, ignoring the structure or continuity of anomalies over time.

These metrics are particularly useful when anomalies can be accurately represented as isolated points. In such scenarios, they provide a straightforward and interpretable assessment of model performance, allowing precise quantification of detection accuracy at the individual time-step level.

However, they become less informative when anomalies extend over time or exhibit internal temporal structure. In these cases, single-point metrics fail to capture essential aspects of detection quality—such as the continuity, duration, or temporal alignment of the predicted anomalies—potentially leading to evaluations that do not fully reflect the practical utility of the model.

The following diagram illustrates the class structure representing the metrics in this category.

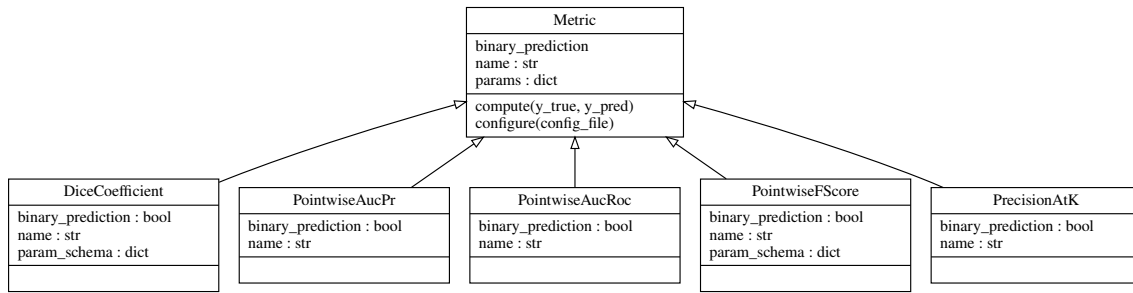


Figure 5.1: Single-Point Based Metrics class diagram

5.1.1 Point-wise F-Score

This metric computes the classical F_β -Score, using Equation 2.6, by treating each time point as an independent observation without considering the temporal context. In other words, the problem is addressed as a standard binary classification task, ignoring the temporal relationships between data instances.

The parameters used by this metric can be summarized as follows:

- **beta (float)**: weighting factor that controls the relative importance of recall versus precision. A value of $\beta = 1$ yields the F_1 -Score, which equally balances both aspects. (*Default: 1.0*)

This metric is implemented in the module `tsadmetrics.metrics.spm.PointwiseFScore`, and can be used either directly through the `PointwiseFScore` class (Section 5.1.1.1) or through the global evaluation pipeline provided by the `Runner` class (Section 5.1.1.2).

Examples of both usage modes are available in the library under: `examples/specific_examples/PointwiseFScore_example.py`.

5.1.1.1 Direct evaluation via PointwiseFScore Class

In this example, `y_true` represents the ground-truth binary labels indicating whether each time point is anomalous (1) or normal (0). Similarly, `y_pred` contains the predicted labels produced by an anomaly detection model.

```

1 from tsadmetrics.metrics.spm.PointwiseFScore import PointwiseFScore
2
3 y_true = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
4 y_pred = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
5
6 metric = PointwiseFScore(beta=1.0)
7 result = metric.compute(y_true, y_pred)
8 print("PointwiseFScore:", result)
  
```

5.1.1.2 Automated evaluation via Runner Class

In this case, the same data sequences (`y_true` and `y_pred`) are organized into a dataset structure that allows batch evaluation across multiple datasets and metrics. The `Runner` class automatically handles the execution of the metric computations and the aggregation of results. For more details on the implementation and usage of the `Runner` class, consult Chapter 6.

```

1 from tsadmetrics.evaluation.Runner import Runner
2
3 y_true = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
4 y_pred = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
5
6 dataset_evaluations = [("dataset1", y_true, y_pred)]
7 metrics = [{"pwf", {"beta": 1}}]
8
9 runner = Runner(dataset_evaluations, metrics)
10 results = runner.run()
11 print(results)

```

5.1.2 Dice Coefficient

The Dice Coefficient metric measures the similarity between predicted and ground-truth *binary* anomaly labels from a set-theoretic perspective. It is mathematically equivalent to the F_1 -Score and quantifies the overlap between predicted anomalies and true anomalies, taking values in $[0, 1]$, where 1 indicates perfect agreement [12].

Formally, the Dice Coefficient is defined as:

$$\text{DiceCoefficient} = \frac{2 \cdot TP}{2 \cdot TP + FP + FN}$$

where TP is the number of true positives, FP the number of false positives, and FN the number of false negatives. The metric is symmetric (swapping prediction and ground truth yields the same value). If both `y_true` and `y_pred` are all zeros (no anomalies), the score is defined as 1.0 to represent perfect agreement in the absence of anomalies.

This metric does not require additional parameters.

This metric is implemented in the module `tsadmetrics.metrics.spm.DiceCoefficient`, and can be used either directly through the `DiceCoefficient` class (Section 5.1.2.1) or through the global evaluation pipeline provided by the `Runner` class (Section 5.1.2.2).

Examples of both usage modes are available in the library under: `examples/specific_examples/DiceCoefficient_example.py`.

5.1.2.1 Direct evaluation via DiceCoefficient Class

In this example, `y_true` represents the ground-truth binary labels indicating whether each time point is anomalous (1) or normal (0). Similarly, `y_pred` contains the predicted labels produced by an anomaly detection model.

```

1 from tsadmetrics.evaluation.Runner import Runner
2
3 y_true = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
4 y_pred = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
5
6
7 metric = DiceCoefficient()
8 result = metric.compute(y_true, y_pred)
9 print("DiceCoefficient:", result)

```

5.1.2.2 Automated evaluation via Runner Class

In this case, the same data sequences (`y_true` and `y_pred`) are organized into a dataset structure that allows batch evaluation across multiple datasets and metrics. The `Runner` class automatically handles the execution of the metric computations and the aggregation of results. For more details on the implementation and usage of the `Runner` class, consult Chapter 6.

```

1 from tsadmetrics.evaluation.Runner import Runner
2
3 y_true = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
4 y_pred = [0,0,0,0,0,1,1,1,0,0,1,1,1,0,0,0,0,0,0,1,1,0,1,1,1,1]
5
6 dataset_evaluations = [
7     ("dataset1", y_true, y_pred)
8 ]
9
10 metrics = [
11     ("dicec", {})
12 ]
13
14 runner = Runner(dataset_evaluations, metrics)
15 results = runner.run()
16 print(results)

```

5.1.3 Point-wise AUC-ROC

The Point-wise AUC-ROC metric measures the Area Under the Receiver Operating Characteristic (ROC) Curve, which plots the True Positive Rate (TPR) against the False Positive Rate (FPR) for different threshold values θ applied to the prediction scores P [13].

It summarizes the overall discriminative ability of the model across all possible thresholds, independently of any specific choice of θ .

This metric does not require additional parameters.

This metric is implemented in the module `tsadmetrics.metrics.spm.PointwiseAucRoc`, and can be used either directly through the `PointwiseAucRoc` class (Section 5.1.3.1) or through the global evaluation pipeline provided by the `Runner` class (Section 5.1.3.2).

Examples of both usage modes are available in the library under: `examples/specific_examples/PointwiseAucRoc_example.py`.

5.1.3.1 Direct evaluation via PointwiseAucRoc Class

In this example, `y_true` represents the ground-truth binary labels indicating whether each time point is anomalous (1) or normal (0). Similarly, `y_pred` contains the predicted labels produced by an anomaly detection model.

```

1 from tsadmetrics.metrics.spm.PointwiseAucRoc import PointwiseAucRoc
2
3 y_true = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
4 y_pred = [0,0,0,0,0,0.4,0.5,0.6,0.7,0.8,0.9,0.95,0.99,0,0,0,0,0,0,0,0,0,0,0,0]
5
6 metric = PointwiseAucRoc()
7 result = metric.compute(y_true, y_pred)
8 print("PointwiseAucRoc:", result)

```

5.1.3.2 Automated evaluation via Runner Class

In this case, the same data sequences (`y_true` and `y_pred`) are organized into a dataset structure that allows batch evaluation across multiple datasets and metrics. The `Runner` class automatically handles the execution of the metric computations and the aggregation of results. For more details on the implementation and usage of the `Runner` class, consult Chapter 6.

```

1 from tsadmetrics.evaluation.Runner import Runner
2
3 dataset_evaluations = [
4     ("dataset1", y_true, y_pred)
5 ]
6
7 metrics = [
8     ("pw_auc_roc", {})
9 ]
10
11 runner = Runner(dataset_evaluations, metrics)
12 results = runner.run()
13 print(results)

```

5.1.4 Point-wise AUC-PR

The Point-wise AUC-PR metric measures the Area Under the Precision–Recall (PR) Curve, which traces Precision versus Recall for different threshold values θ applied to the prediction scores P .

It summarizes the overall discriminative ability of the model across all possible thresholds, independently of any specific choice of θ .

This metric does not require additional parameters.

This metric is implemented in the module `tsadmetrics.metrics.spm.PointwiseAucPr`, and can be used either directly through the `PointwiseAucPr` class (Section 5.1.4.1) or through the global evaluation pipeline provided by the `Runner` class (Section 5.1.4.2).

Examples of both usage modes are available in the library under: `examples/specific_examples/PointwiseAucPr_example.py`.

5.1.4.1 Direct evaluation via PointwiseAucPr Class

In this example, `y_true` represents the ground-truth binary labels indicating whether each time point is anomalous (1) or normal (0). Similarly, `y_pred` contains the predicted labels produced by an anomaly detection model.

```

1 from tsadmetrics.metrics.spm.PointwiseAucPr import PointwiseAucPr
2
3 y_true = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
4 y_pred = [0,0,0,0,0,0.4,0.5,0.6,0.7,0.8,0.9,0.95,0.99,0,0,0,0,0,0,0,0,0,0,0,0]
5
6 metric = PointwiseAucPr()
7 result = metric.compute(y_true, y_pred)
8 print("PointwiseAucPr:", result)

```

5.1.4.2 Automated evaluation via Runner Class

In this case, the same data sequences (`y_true` and `y_pred`) are organized into a dataset structure that allows batch evaluation across multiple datasets and metrics. The `Runner` class automatically handles the execution of the metric computations and the aggregation of results. For more details on the implementation and usage of the `Runner` class, consult Chapter 6.

```

1 from tsadmetrics.evaluation.Runner import Runner
2
3 dataset_evaluations = [
4     ("dataset1", y_true, y_pred)
5 ]
6
7 metrics = [
8     ("pw_auc_pr", {})
9 ]
10
11 runner = Runner(dataset_evaluations, metrics)
12 results = runner.run()
13 print(results)

```

5.1.5 Precision@K

The Precision@K metric evaluates the ability of the model to assign higher anomaly scores to true anomalies. Let $K = \sum_{i=1}^n T_i$, i.e., the total number of truly anomalous points in the ground truth T [14].

The K points with the highest anomaly scores in P are selected, and the metric is defined as:

$$\text{PrecisionAtK}(T, P) = \frac{1}{K} \sum_{i \in \text{TopK}(P)} T_i \quad (5.1)$$

where $\text{TopK}(P)$ denotes the set of indices corresponding to the K largest values in P .

This metric reflects how well the model prioritizes anomalous points in the ranking of predicted scores. Ideally, all K highest scores would correspond exactly to the true anomalies in T .

This metric does not require additional parameters.

This metric is implemented in the module `tsadmetrics.metrics.spm.PrecisionAtK`, and can be used either directly through the `PrecisionAtK` class (Section 5.1.5.1) or through the global evaluation pipeline provided by the `Runner` class (Section 5.1.5.2).

Examples of both usage modes are available in the library under: `examples/specific_examples/PrecisionAtK_example.py`.

5.1.5.1 Direct evaluation via PrecisionAtK Class

In this example, `y_true` represents the ground-truth binary labels indicating whether each time point is anomalous (1) or normal (0). Similarly, `y_pred` contains the predicted labels produced by an anomaly detection model.

```

1 from tsadmetrics.metrics.spm.PrecisionAtK import PrecisionAtK
2
3 y_true = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,0,1,1,1,1,1,1,1]
4 y_pred = [0,0,0,0,0,0.4,0.5,0.6,0.7,0.8,0.9,0.95,0.99,0,0,0,0,0,0,0,0,0,0,0,0,0]
5
6 metric = PrecisionAtK()
7 result = metric.compute(y_true, y_pred)
8 print("PrecisionAtK:", result)

```

5.1.5.2 Automated evaluation via Runner Class

In this case, the same data sequences (`y_true` and `y_pred`) are organized into a dataset structure that allows batch evaluation across multiple datasets and metrics. The `Runner` class automatically handles the execution of the metric computations and the aggregation of results. For more details on the implementation and usage of the `Runner` class, consult Chapter 6.

```

1 from tsadmetrics.evaluation.Runner import Runner
2
3 dataset_evaluations = [
4     ("dataset1", y_true, y_pred)
5 ]
6
7 metrics = [
8     ("pak", {})
9 ]
10
11 runner = Runner(dataset_evaluations, metrics)
12 results = runner.run()
13 print(results)

```

5.2 Temporal Evaluation Metrics

This category groups metrics that incorporate the temporal context in the evaluation process, unlike single-point based metrics. In other words, they consider not only whether an anomaly has been detected, but also when and how it occurs in relation to the original time series sequence.

These metrics are specifically designed for time series anomaly detection tasks and are particularly useful when analyzing one or more properties of the model related to the temporal structure of anomalies, such as their duration, anticipation, coverage, or overlap.

Since temporal evaluation metrics can focus on different aspects of time, several specialized subtypes have been defined to classify them according to the specific characteristics they assess and their intended context of application.

5.2.1 Tolerant Partial Detection Metrics

This category includes metrics that consider a predicted anomaly Q_i as valid if it occurs at any point within the interval of a real anomalous event R_j . These metrics are based on the assumption that a partial detection is sufficient to raise an alert, under the idea that an expert can then verify whether a true anomaly has occurred.

TPDM are particularly useful in contexts where the primary objective is to trigger an alert for potential intervention. In such cases, partial detection of an anomalous pattern is acceptable, and a certain temporal imprecision is tolerated to reduce the risk of false negatives caused by small misalignments between the prediction and the actual event.

The following diagram illustrates the class structure representing the metrics in this category.

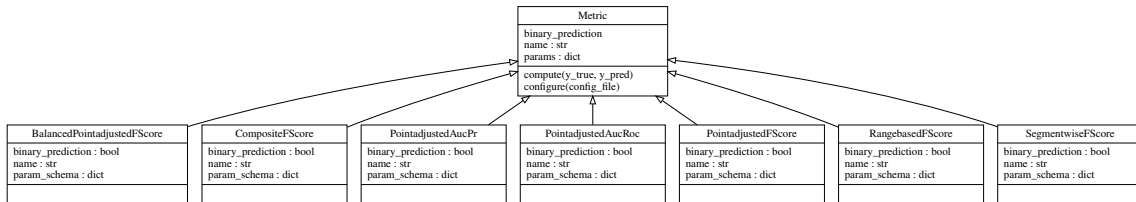


Figure 5.2: Tolerant Partial Detection Metrics class diagram

5.2.1.1 Point-adjusted F-Score

The Point-adjusted F-Score metric computes the classical F_β -Score (Equation 2.6) after performing an adjustment on the predictions P . Specifically, if at least one point within a real anomalous segment in the ground-truth labels T is correctly detected in P , then the entire segment is considered identified and is fully marked as positive in the adjusted prediction P' [15].

The parameters used by this metric can be summarized as follows:

- **beta (float)**: weighting factor that controls the relative importance of recall versus precision. A value of $\beta = 1$ yields the F_1 -Score, which equally balances both aspects. (*Default*: 1.0)

This metric is implemented in the module `tsadmetrics.metrics.tem.tpdm.PointadjustedFScore`, and can be used either directly through the `PointadjustedFScore` class (Section 5.2.1.1.1) or through the global evaluation pipeline provided by the `Runner` class (Section 5.2.1.1.2).

Examples of both usage modes are available in the library under: `examples/specific_examples/PointadjustedFScore_example.py`.

5.2.1.1.1 Direct evaluation via PointadjustedFScore Class

In this example, `y_true` represents the ground-truth binary labels indicating whether each time point is anomalous (1) or normal (0). Similarly, `y_pred` contains the predicted labels produced by an anomaly detection model.

```

1 from tsadmetrics.metrics.tem.tpdm.PointadjustedFScore import PointadjustedFScore
2
3 y_true = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
4 y_pred = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
5
6 metric = PointadjustedFScore(beta=1.0)
7 result = metric.compute(y_true, y_pred)
8 print("PointadjustedFScore:", result)

```

5.2.1.1.2 Automated evaluation via Runner Class

In this case, the same data sequences (`y_true` and `y_pred`) are organized into a dataset structure that allows batch evaluation across multiple datasets and metrics. The `Runner` class automatically handles the execution of the metric computations and the aggregation of results. For more details on the implementation and usage of the `Runner` class, consult Chapter 6.

```

1 from tsadmetrics.evaluation.Runner import Runner
2
3 dataset_evaluations = [
4     ("dataset1", y_true, y_pred)
5 ]
6
7 metrics = [
8     ("paf", {"beta":1.0})
9 ]
10
11 runner = Runner(dataset_evaluations, metrics)
12 results = runner.run()
13 print(results)

```

5.2.1.2 Balanced Point-adjusted F-Score

The Balanced Point-adjusted F-Score is an extension of the Point-adjusted F-Score, introducing an additional balancing adjustment over the already modified prediction sequence P' [16].

As in the original formulation, if at least one point within a real anomalous segment in the ground-truth labels T is correctly detected in P , the entire segment is marked as positive in the

adjusted prediction P' . However, the Balanced Point-adjusted F-Score further modifies P' by expanding the influence of predicted positives that fall *outside* the true anomaly regions.

Specifically, for every predicted positive point that does not belong to any real anomaly (i.e., a false positive), a temporal expansion is applied symmetrically around its position, within a window of radius $\lceil w/2 \rceil$:

$$[t - \lceil w/2 \rceil, t + \lceil w/2 \rceil].$$

All indices within this range are set to 1 in the adjusted prediction (clipped to the time-series bounds). This operation intentionally increases the penalty for spurious detections by generating additional false positives, thereby *balancing* the optimistic effect introduced by the full-segment confirmation step in the standard Point-adjusted approach.

After this two-step adjustment, precision and recall are computed pointwise between the adjusted predictions and the ground truth T , and the final score is obtained using the standard F-Score formulation (Equation 2.6). If either precision or recall equals zero, the resulting score is defined as 0.

The parameters used by this metric can be summarized as follows:

- **beta (float)**: weighting factor that controls the relative importance of recall versus precision. (*Default*: 1.0)
- **w (int)**: temporal window size controlling the expansion radius $\lceil w/2 \rceil$ applied around off-segment positives (*Default*: 1.0).

This metric is implemented in the module `tsadmetrics.metrics.tem.dpm.BalancedPointadjustedFScore`, and can be used either directly through the `BalancedPointadjustedFScore` class (Section 5.2.1.2.1) or through the global evaluation pipeline provided by the `Runner` class (Section 5.2.1.2.2).

Examples of both usage modes are available in the library under: `examples/specific_examples/BalancedPointadjustedFScore_example.py`.

5.2.1.2.1 Direct evaluation via `BalancedPointadjustedFScore` Class

In this example, `y_true` represents the ground-truth binary labels indicating whether each time point is anomalous (1) or normal (0). Similarly, `y_pred` contains the predicted labels produced by an anomaly detection model.

```

1 from tsadmetrics.metrics.tem.dpm.BalancedPointadjustedFScore import
   BalancedPointadjustedFScore
2
3 y_true = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
4 y_pred = [0,0,0,0,0,1,0,0,1,1,0,1,0,0,0,0,0,0,1,0,1,1,1,0,1,0]
5
6 metric = BalancedPointadjustedFScore(beta=1.0, w=3)
7 result = metric.compute(y_true, y_pred)
8 print("BalancedPointadjustedFScore:", result)

```

5.2.1.2.2 Automated evaluation via Runner Class

In this case, the same data sequences (`y_true` and `y_pred`) are organized into a dataset structure that allows batch evaluation across multiple datasets and metrics. The `Runner` class automatically handles the execution of the metric computations and the aggregation of results. For more details on the implementation and usage of the `Runner` class, consult Chapter 6.

```

1 from tsadmetrics.evaluation.Runner import Runner
2
3 y_true = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1]
4 y_pred = [0,0,0,0,0,1,0,0,1,1,0,1,0,0,0,0,0,0,1,0,1,1,1,0,1,0]
5
6 dataset_evaluations = [
7     ("dataset1", y_true, y_pred)
8 ]
9
10 metrics = [
11     ("bpaf", {"beta":1.0, "w":3})
12 ]
13
14 runner = Runner(dataset_evaluations, metrics)
15 results = runner.run()
16 print(results)

```

5.2.1.3 Segment-wise F-Score

The Segment-wise F-Score metric computes the classical F_β -Score (Equation 2.6) with a modified definition of true positives, false positives, and false negatives, where each contiguous segment of anomalous points is treated as an indivisible unit.

A real anomalous segment in T is considered correctly detected if at least one of its points is predicted as anomalous in P . In that case, it counts as a single true positive. If none of the points in the segment are detected, it counts as a false negative. Conversely, each contiguous segment of predicted anomalous points in P that does not overlap with any real anomalous segment in T is counted as a false positive [17].

The parameters used by this metric can be summarized as follows:

- **beta (float):** weighting factor that controls the relative importance of recall versus precision. (*Default: 1.0*)

This metric is implemented in the module `tsadmetrics.metrics.tem.tpdm.SegmentwiseFScore`, and can be used either directly through the `SegmentwiseFScore` class (Section 5.2.1.3.1) or through the global evaluation pipeline provided by the `Runner` class (Section 5.2.1.3.2).

Examples of both usage modes are available in the library under: `examples/specific_examples/SegmentwiseFScore_example.py`.

5.2.1.3.1 Direct evaluation via SegmentwiseFScore Class

In this example, `y_true` represents the ground-truth binary labels indicating whether each time point is anomalous (1) or normal (0). Similarly, `y_pred` contains the predicted labels produced by an anomaly detection model.

```

1 from tsadmetrics.metrics.tem.tpdm.SegmentwiseFScore import SegmentwiseFScore
2
3 y_true = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
4 y_pred = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
5
6 metric = SegmentwiseFScore(beta=1.0)
7 result = metric.compute(y_true, y_pred)
8 print("SegmentwiseFScore:", result)

```

5.2.1.3.2 Automated evaluation via Runner Class

In this case, the same data sequences (`y_true` and `y_pred`) are organized into a dataset structure that allows batch evaluation across multiple datasets and metrics. The `Runner` class automatically handles the execution of the metric computations and the aggregation of results. For more details on the implementation and usage of the `Runner` class, consult Chapter 6.

```

1 from tsadmetrics.evaluation.Runner import Runner
2
3 y_true = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
4 y_pred = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
5
6 dataset_evaluations = [
7     ("dataset1", y_true, y_pred)
8 ]
9
10 metrics = [
11     ("swf", {"beta":1.0})
12 ]
13
14 runner = Runner(dataset_evaluations, metrics)
15 results = runner.run()
16 print(results)

```

5.2.1.4 Composite F-Score

The Composite F-Score metric combines aspects of the Point-wise F-Score and the Segment-wise F-Score. It is defined as the harmonic mean between the point-wise precision P_{pw} , computed as in the Point-wise F-Score, and the segment-wise recall R_{seg} , computed using Equation 2.5 with the same adjustment applied in the Segment-wise F-Score.

In this way, the Composite F-Score leverages the strengths of both perspectives: on the one hand, point-wise precision explicitly penalizes false positives; on the other hand, segment-wise recall ensures that each anomalous segment in T must be covered by at least one predicted anomaly in P to be considered detected [18].

For example, if $P_{pw} = 0.5$ and $R_{seg} = 1.0$, then for $\beta = 1$ the score is computed as:

$$\text{CompositeFScore}_{\beta=1} = 2 \cdot \frac{0.5 \cdot 1.0}{0.5 + 1.0} = 0.67$$

The parameters used by this metric can be summarized as follows:

- **beta (float):** weighting factor that controls the relative importance of recall versus precision. (*Default: 1.0*)

This metric is implemented in the module `tsadmetrics.metrics.tem.tpdm.CompositeFScore`, and can be used either directly through the `CompositeFScore` interface (Section 5.2.1.4.1) or through the global evaluation pipeline provided by the `Runner` class (Section 5.2.1.4.2).

Examples of both usage modes are available in the library under: `examples/specific_examples/CompositeFScore_example.py`.

5.2.1.4.1 Direct evaluation via CompositeFScore Class

In this example, `y_true` represents the ground-truth binary labels indicating whether each time point is anomalous (1) or normal (0). Similarly, `y_pred` contains the predicted labels produced by an anomaly detection model.

```
1 from tsadmetrics.metrics.tem.tpdm.CompositeFScore import CompositeFScore
2
3 y_true = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
4 y_pred = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
5
6 metric = CompositeFScore(beta=1.0)
7 result = metric.compute(y_true, y_pred)
8 print("CompositeFScore:", result)
```

5.2.1.4.2 Automated evaluation via Runner Class

In this case, the same data sequences (`y_true` and `y_pred`) are organized into a dataset structure that allows batch evaluation across multiple datasets and metrics. The `Runner` class automatically handles the execution of the metric computations and the aggregation of results. For more details on the implementation and usage of the `Runner` class, consult Chapter 6.

```
1 from tsadmetrics.evaluation.Runner import Runner
2
3 y_true = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
4 y_pred = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
5
6 dataset_evaluations = [
7     ("dataset1", y_true, y_pred)
8 ]
9
10 metrics = [
11     ("cf", {"beta":1.0})
12 ]
13
```

```

14 runner = Runner(dataset_evaluations, metrics)
15 results = runner.run()
16 print(results)

```

5.2.1.5 Point-adjusted AUC-PR

The Point-adjusted AUC-PR metric is a variant of the Point-wise AUC-PR. It is computed by first binarizing the predicted scores P using all possible thresholds θ , and then applying the same point-adjustment procedure used in the Point-adjusted F-Score: if at least one point of a true anomalous segment is detected, the whole segment is considered correctly predicted in the adjusted sequence P' [19].

By integrating this adjustment, the Point-adjusted AUC-PR provides an evaluation that accounts for the temporal structure of anomalies, rewarding detectors that capture anomalous segments rather than isolated points.

This metric does not require additional parameters.

This metric is implemented in the module `tsadmetrics.metrics.tem.tpdm.PointadjustedAucPr`, and can be used either directly through the `PointadjustedAucPr` interface (Section 5.2.1.5.1) or through the global evaluation pipeline provided by the `Runner` class (Section 5.2.1.5.2).

Examples of both usage modes are available in the library under: `examples/specific_examples/PointadjustedAucPr_example.py`.

5.2.1.5.1 Direct evaluation via PointadjustedAucPr Class

In this example, `y_true` represents the ground-truth binary labels indicating whether each time point is anomalous (1) or normal (0). Similarly, `y_pred` contains the predicted labels produced by an anomaly detection model.

```

1 from tsadmetrics.metrics.tem.tpdm.PointadjustedAucPr import PointadjustedAucPr
2
3 y_true = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,0,1,1,1,1,1,1,1]
4 y_pred = [0,0,0,0,0,0.4,0.5,0.6,0.7,0.8,0.9,0.95,0.99,0,0,0,0,0,0,0,0,0,0,0,0,0,0]
5
6 metric = PointadjustedAucPr()
7 result = metric.compute(y_true, y_pred)
8 print("PointadjustedAucPr:", result)

```

5.2.1.5.2 Automated evaluation via Runner Class

In this case, the same data sequences (`y_true` and `y_pred`) are organized into a dataset structure that allows batch evaluation across multiple datasets and metrics. The `Runner` class automatically handles the execution of the metric computations and the aggregation of results. For more details on the implementation and usage of the `Runner` class, consult Chapter 6.

```

1 from tsadmetrics.evaluation.Runner import Runner
2
3 y_true = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,0,1,1,1,1,1,1,1]
4 y_pred = [0,0,0,0,0,0.4,0.5,0.6,0.7,0.8,0.9,0.95,0.99,0,0,0,0,0,0,0,0,0,0,0,0,0]
5
6 dataset_evaluations = [
7     ("dataset1", y_true, y_pred)
8 ]
9
10 metrics = [
11     ("pa_auc_pr", {})
12 ]
13
14 runner = Runner(dataset_evaluations, metrics)
15 results = runner.run()
16 print(results)

```

5.2.1.6 Point-adjusted AUC-ROC

The Point-adjusted AUC-ROC metric is a variant of the Point-wise AUC-ROC. As in the standard definition, the ROC curve is computed by varying the threshold θ applied to the predicted scores P . However, once binarized, predictions are adjusted into P' using the same procedure as in the Point-adjusted F-Score: if at least one point of an anomalous segment is detected, the entire segment is marked as correctly identified [19].

The Point-adjusted AUC-ROC therefore adapts the conventional AUC-ROC to anomaly detection settings where segment-level coverage is more relevant than isolated pointwise matches.

This metric does not require additional parameters.

This metric is implemented in the module `tsadmetrics.metrics.tem.tpdm.PointadjustedAucRoc`, and can be used either directly through the `PointadjustedAucRoc` function (Section 5.2.1.6.1) or through the global evaluation pipeline provided by the `Runner` class (Section 5.2.1.6.2).

Examples of both usage modes are available in the library under: `examples/specific_examples/PointadjustedAucRoc_example.py`.

5.2.1.6.1 Direct evaluation via PointadjustedAucRoc Class

In this example, `y_true` represents the ground-truth binary labels indicating whether each time point is anomalous (1) or normal (0). Similarly, `y_pred` contains the predicted labels produced by an anomaly detection model.

```

1 from tsadmetrics.metrics.tem.tpdm.PointadjustedAucRoc import PointadjustedAucRoc
2
3 y_true = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,0,1,1,1,1,1,1,1]
4 y_pred = [0,0,0,0,0,0.4,0.5,0.6,0.7,0.8,0.9,0.95,0.99,0,0,0,0,0,0,0,0,0,0,0,0,0]
5
6 metric = PointadjustedAucRoc()
7 result = metric.compute(y_true, y_pred)
8 print("PointadjustedAucRoc:", result)

```

5.2.1.6.2 Automated evaluation via Runner Class

In this case, the same data sequences (`y_true` and `y_pred`) are organized into a dataset structure that allows batch evaluation across multiple datasets and metrics. The `Runner` class automatically handles the execution of the metric computations and the aggregation of results. For more details on the implementation and usage of the `Runner` class, consult Chapter 6.

```

1 from tsadmetrics.evaluation.Runner import Runner
2
3 y_true = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
4 y_pred = [0,0,0,0,0,0.4,0.5,0.6,0.7,0.8,0.9,0.95,0.99,0,0,0,0,0,0,0,0,0,0,0,0]
5
6 dataset_evaluations = [
7     ("dataset1", y_true, y_pred)
8 ]
9
10 metrics = [
11     ("pa_auc_roc", {})
12 ]
13
14 runner = Runner(dataset_evaluations, metrics)
15 results = runner.run()
16 print(results)

```

5.2.1.7 Range-based F-Score

The Range-based F-Score metric extends the traditional $F\beta$ -score by redefining *precision* and *recall* to operate over anomalous **ranges** rather than individual points. This approach is specifically designed to address the temporal and structural nature of anomaly detection in time series data, following the formulation proposed by Tatbul et al. [10].

In this framework, the recall is computed by evaluating how well each true anomalous range $R_i \in \mathcal{R}$ has been covered by at least one predicted anomalous range $Q_j \in \mathcal{Q}$, while the *precision* assesses the degree to which each predicted range corresponds to a real anomalous interval.

This metric is highly configurable, as it allows the adjustment of up to six independent components that modulate how precision and recall are calculated for range-based evaluations. These components are summarized as follows:

- **Existence:** Determines whether a true anomalous range has at least one overlapping prediction.
- **Size:** Measures the proportion of the true range that is correctly covered by predictions.
- **Position:** Uses a positional bias function $\delta()$ to emphasize early, late, or central coverage within a range.
- **Cardinality:** Introduces a penalty for fragmented detections, modeled by function $\gamma()$.
- **Overlap:** Quantifies the weighted contribution of the intersection between true and predicted ranges using function $\omega()$.
- **Balancing:** The parameter α controls the trade-off between existence-based and overlap-based components.

The final Range-based F-Score is computed using the general F-score formulation (Equation 2.6), where traditional precision and recall are replaced by their range-based counterparts described below.

5.2.1.7.1 Range-based Recall

The goal of the recall metric is to reward a model when it successfully identifies real anomalies (TP) and penalize it when it fails to do so (FN). Let $\mathcal{R} = \{R_1, \dots, R_{N_r}\}$ be the set of true anomalous ranges, and $\mathcal{Q} = \{Q_1, \dots, Q_{N_q}\}$ be the set of predicted anomalous ranges. The range-based recall $\text{Recall}_T(\mathcal{R}, \mathcal{Q})$ measures the average detection quality across all true ranges:

$$\text{Recall}_T(\mathcal{R}, \mathcal{Q}) = \frac{\sum_{i=1}^{N_r} \text{Recall}_T(R_i, \mathcal{Q})}{N_r} \quad (5.2)$$

For a single real anomalous range R_i , the recall considers several key aspects:

- **Existence:** Whether the anomaly is detected at all (i.e., at least one predicted point lies within R_i).
- **Size:** The proportion of R_i correctly covered by predictions.
- **Position:** The relative position of overlap within the range, emphasizing early, late, or central coverage.
- **Cardinality:** Penalizes cases where R_i is fragmented across multiple predictions.

These components are combined as follows, where $0 \leq \alpha \leq 1$ balances the importance of existence versus overlap:

$$\text{Recall}_T(R_i, \mathcal{Q}) = \alpha \cdot \text{RecExistence}(R_i, \mathcal{Q}) + (1 - \alpha) \cdot \text{RecOverlap}(R_i, \mathcal{Q}) \quad (5.3)$$

The existence term is binary:

$$\text{RecExistence}(R_i, \mathcal{Q}) = \begin{cases} 1, & \text{if } \sum_{j=1}^{N_q} |R_i \cap Q_j| \geq 1 \\ 0, & \text{otherwise} \end{cases} \quad (5.4)$$

The overlap term depends on three functions: the *cardinality factor* $\gamma()$, the *overlap weighting* $\omega()$, and the *positional bias* $\delta()$:

$$\text{RecOverlap}(R_i, \mathcal{Q}) = \text{CardinalityFactor}(R_i, \mathcal{Q}) \cdot \sum_{j=1}^{N_q} \omega(R_i, R_i \cap Q_j, \delta) \quad (5.5)$$

The cardinality factor modulates the reward based on the fragmentation of the detections:

$$\text{CardinalityFactor}(R_i, \mathcal{Q}) = \begin{cases} 1, & \text{if } R_i \text{ overlaps with at most one } Q_j \\ \gamma(R_i, \mathcal{Q}), & \text{otherwise} \end{cases} \quad (5.6)$$

The functions $\gamma()$, $\omega()$, and $\delta()$ are detailed in Section 5.2.1.7.3.

5.2.1.7.2 Range-based Precision

Analogously to recall, range-based precision measures the accuracy of predicted anomaly ranges with respect to true anomalies. Let $\mathcal{R}$ and $\mathcal{Q}$ be defined as before. The range-based precision is formulated as:

$$\text{Precision}_T(\mathcal{R}, \mathcal{Q}) = \frac{\sum_{j=1}^{N_q} \text{Precision}_T(\mathcal{R}, Q_j)}{N_q} \quad (5.7)$$

Unlike recall, precision does not include an existence component since mere detection does not guarantee prediction quality. Thus, the focus is solely on the overlap between predicted and true ranges, with $\alpha = 0$:

$$\text{Precision}_T(\mathcal{R}, Q_j) = \text{CardinalityFactor}(Q_j, \mathcal{R}) \cdot \sum_{i=1}^{N_r} \omega(Q_j, Q_j \cap R_i, \delta) \quad (5.8)$$

This formulation rewards predictions that align closely with actual anomalies while penalizing fragmented or poorly localized detections.

5.2.1.7.3 Definition and Configuration of the Functions γ , ω , and δ

The functions $\gamma()$, $\omega()$, and $\delta()$ introduce flexibility and adaptability to the range-based F-score, allowing it to capture diverse detection objectives. Their definitions are summarized below.

5.2.1.7.3.1 Overlap Function $\omega()$

The overlap function computes the weighted intersection between a true and a predicted range:

Algorithm 1 Overlap function $\omega()$

```

1: value  $\leftarrow$  0
2: maxVal  $\leftarrow$  0
3: for  $i \leftarrow 1$  to length(range) do
4:   bias  $\leftarrow \delta(i, \text{length}(\text{range}))$ 
5:   maxVal  $\leftarrow$  maxVal + bias
6:   if range[ $i$ ] in overlap then
7:     value  $\leftarrow$  value + bias
8:   end if
9: end for
10: return value/maxVal
```

5.2.1.7.3.2 Positional Bias Function $\delta()$

Defines the positional weighting applied within a range:

Algorithm 2 Positional bias function $\delta()$

```

1: Input: mode, position  $i$ , range length  $L$ 
2: Output: positional weight
3: Modes:
4:   Flat:  $\delta(i, L) = 1$ 
5:   Front:  $\delta(i, L) = \frac{L-i+1}{L}$ 
6:   Back:  $\delta(i, L) = \frac{i}{L}$ 
7:   Middle:  $\delta(i, L) = 1 - \left\lfloor \frac{2i-L}{L} \right\rfloor$ 
```

5.2.1.7.3.3 Cardinality Function $\gamma()$

Applies a penalty for fragmented detections:

Algorithm 3 Cardinality function $\gamma()$

- 1: **Input:** mode, overlap count
 - 2: **Output:** cardinality factor
 - 3: **Modes:**
 - 4: Single: $\gamma(\text{mode}, \text{overlap}) = 1$
 - 5: Reciprocal: $\gamma(\text{mode}, \text{overlap}) = \min(1, 1/\text{overlap})$
-

The $\gamma()$, $\omega()$, and $\delta()$ functions collectively enable fine control over how detection overlap, position, and fragmentation are evaluated, providing a flexible and principled way to assess temporal anomaly detection performance.

5.2.1.7.4 Summary of parameters and implementation.

The parameters used by this metric can be summarized as follows:

- **beta (float):** weighting factor that controls the relative importance of recall versus precision. (*Default: 1.0*)
- **p_alpha, r_alpha (float):** balance factors ($0 \leq \alpha \leq 1$) controlling the trade-off between existence-based and overlap-based components for precision and recall, respectively. (*Default: 0.5*)
- **p_bias, r_bias (str):** positional bias modes used by the $\delta()$ function to emphasize early, late, or central detection. Typical values: "flat", "front", "back", "middle". (*Default: "flat"*)
- **cardinality_mode (str):** defines how the $\gamma()$ function penalizes fragmented detections. Possible values: "one" (no penalty), "reciprocal" (inverse penalty), or "gamma" (custom user-defined penalty through the parameter γ). The "gamma" mode allows adjusting the strength of the penalization applied to fragmented detections via a continuous coefficient $\gamma \in [0, 1]$, where lower values increase the penalty for fragmentation. (*Default: "one"*)

This metric is implemented in the module `tsadmetrics.metrics.tem.tpdm.RangebasedFScore`, and can be used either directly through the `RangebasedFScore` function (Section 5.2.1.7.5) or through the global evaluation pipeline provided by the `Runner` class (Section 5.2.1.7.6).

Examples of both usage modes are available in the library under: `examples/specific_examples/RangebasedFScore_example.py`.

5.2.1.7.5 Direct evaluation via RangebasedFScore Class

In this example, `y_true` represents the ground-truth binary labels indicating whether each time point is anomalous (1) or normal (0). Similarly, `y_pred` contains the predicted labels produced by an anomaly detection model.

```

1 from tsadmetrics.metrics.tem.tpdm.RangebasedFScore import RangebasedFScore
2
3 y_true = [0,0,0,0,0,1,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
4 y_pred = [0,0,0,0,0,1,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
5
6 metric = RangebasedFScore(beta=1.0, p_alpha=0.5, r_alpha=0.5,
7                             p_bias="flat", r_bias="flat",
8                             cardinality_mode="one")
9 result = metric.compute(y_true, y_pred)
10 print("RangebasedFScore:", result)

```

5.2.1.7.6 Automated evaluation via Runner Class

In this case, the same data sequences (`y_true` and `y_pred`) are organized into a dataset structure that allows batch evaluation across multiple datasets and metrics. The `Runner` class automatically handles the execution of the metric computations and the aggregation of results. For more details on the implementation and usage of the `Runner` class, consult Chapter 6.

```

1 from tsadmetrics.evaluation.Runner import Runner
2
3 dataset_evaluations = [
4     ("dataset1", y_true, y_pred)
5 ]
6
7 metrics = [
8     ("rbf", {"beta":1.0, "p_alpha":0.5, "r_alpha":0.5,
9             "p_bias":"flat", "r_bias":"flat",
10             "cardinality_mode":"one"})
11 ]
12
13 runner = Runner(dataset_evaluations, metrics)
14 results = runner.run()
15 print(results)

```

5.2.2 Delay-Penalized Metrics

This category includes metrics that penalize predictions P_i that occur significantly after the actual onset of the anomalous event R_i , thereby incentivizing early detection.

DPM are particularly useful in contexts where preventing an event is not possible, but detecting it as soon as possible after its onset is critical. These metrics emphasize minimizing detection delay while still capturing the presence of the anomaly accurately.

The following diagram illustrates the class structure representing the metrics in this category.

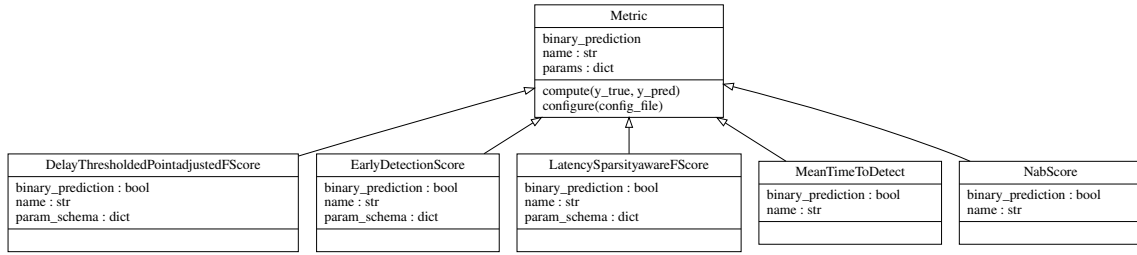


Figure 5.3: Delay-Penalized Metrics class diagram

5.2.2.1 Delay-thresholded Point-adjusted F-Score

The Delay-thresholded Point-adjusted F-Score metric is an adaptation of the Point-adjusted F-Score that introduces a delay threshold k . In this variant, an adjusted prediction sequence P' is constructed from P . An anomalous segment in T is considered correctly detected only if at least one anomalous point within the first k time steps of the segment is flagged in P . If this condition is not satisfied, all points of the segment in P' are marked as negative and therefore counted as false negatives, even if some of them were detected later in P [20].

The parameters used by this metric can be summarized as follows:

- **k (int)**: maximum allowed delay (in time steps) from the start of an anomalous segment for it to be considered correctly detected. A smaller value enforces stricter timeliness. (*Default*: 1)
- **beta (float)**: weighting factor that controls the relative importance of recall versus precision. (*Default*: 1.0)

This metric is implemented in the module `tsadmetrics.metrics.tem.dpm.DelayThresholdedPointadjustedFScore`, and can be used either directly through the `DelayThresholdedPointadjustedFScore` function (Section 5.2.2.1.1) or through the global evaluation pipeline provided by the `Runner` class (Section 5.2.2.1.2).

Examples of both usage modes are available in the library under: `examples/specific_examples/DelayThresholdedPointadjustedFScore_example.py`.

5.2.2.1.1 Direct evaluation via DelayThresholdedPointadjustedFScore Class

In this example, `y_true` represents the ground-truth binary labels indicating whether each time point is anomalous (1) or normal (0). Similarly, `y_pred` contains the predicted labels produced by an anomaly detection model.

```

1 from tsadmetrics.metrics.tem.dpm.DelayThresholdedPointadjustedFScore import
   DelayThresholdedPointadjustedFScore
2
3 y_true = [0,0,0,0,0,1,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
4 y_pred = [0,0,0,0,0,1,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
5
6 metric = DelayThresholdedPointadjustedFScore(k=1, beta=1.0)
7 result = metric.compute(y_true, y_pred)
8 print("DelayThresholdedPointadjustedFScore:", result)

```

5.2.2.1.2 Automated evaluation via Runner Class

In this case, the same data sequences (`y_true` and `y_pred`) are organized into a dataset structure that allows batch evaluation across multiple datasets and metrics. The `Runner` class automatically handles the execution of the metric computations and the aggregation of results. For more details on the implementation and usage of the `Runner` class, consult Chapter 6.

```

1 from tsadmetrics.evaluation.Runner import Runner
2
3 dataset_evaluations = [
4     ("dataset1", y_true, y_pred)
5 ]
6
7 metrics = [
8     ("dtpaf", {"k":1, "beta":1.0})
9 ]
10
11 runner = Runner(dataset_evaluations, metrics)
12 results = runner.run()
13 print(results)

```

5.2.2.2 Early Detection Score

The Early Detection Score metric quantifies how **early** an anomaly is detected relative to its actual occurrence in a time series. It evaluates the timing of the first prediction within a predefined *anomaly window* around each ground-truth anomaly, emphasizing the promptness of detections [21].

For every anomaly window $aw_d(a) = [t_b, t_e]$, the Early Detection score is defined as:

$$ed_a = \begin{cases} 1 - \frac{\text{pos}_d(T(a)) - b}{e - b}, & \text{if a detection occurs within the window,} \\ 0, & \text{otherwise.} \end{cases}$$

Here, $\text{pos}_d(T(a))$ denotes the position of the first predicted anomaly within the window, while b and e represent the start and end indices of the window, respectively. A detection closer to the start of the window yields a score near 1 (early detection), whereas detections closer to the end yield values near 0.

Following the definition proposed in the ADE framework [21], the anomaly window length is given by:

$$\|aw_d\| = 0.1 \cdot \frac{\|d\|}{\|A_d\|},$$

where $\|d\|$ is the total length of the time series, and $\|A_d\|$ is the number of anomalous segments. Each window is symmetrically centered around the anomaly and clipped within the series boundaries.

The final Early Detection Score is the mean of all individual ed_a values across ground-truth anomalies, providing a single quantitative measure of how promptly the model reacts to anomalies.

This metric does not have additional parameters.

This metric is implemented in the module `tsadmetrics.metrics.tem.dpm.EarlyDetectionScore`, and can be used either directly through the `EarlyDetectionScore` class (Section 5.2.2.2.1) or through the global evaluation pipeline provided by the `Runner` class (Section 5.2.2.2.2).

Examples of both usage modes are available in the library under: `examples/specific_examples/EarlyDetectionScore_example.py`.

5.2.2.2.1 Direct evaluation via EarlyDetectionScore Class

In this example, `y_true` represents the ground-truth binary labels indicating whether each time point is anomalous (1) or normal (0). Similarly, `y_pred` contains the predicted labels produced by an anomaly detection model.

```

1 from tsadmetrics.metrics.tem.dpm.EarlyDetectionScore import EarlyDetectionScore
2
3
4 y_true = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
5 y_pred = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
6
7 metric = EarlyDetectionScore()
8 result = metric.compute(y_true, y_pred)
9 print("EarlyDetectionScore:", result)

```

5.2.2.2.2 Automated evaluation via Runner Class

In this case, the same data sequences (`y_true` and `y_pred`) are organized into a dataset structure that allows batch evaluation across multiple datasets and metrics. The `Runner` class automatically handles the execution of the metric computations and the aggregation of results. For more details on the implementation and usage of the `Runner` class, consult Chapter 6.

```

1 from tsadmetrics.evaluation.Runner import Runner
2
3
4 y_true = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
5 y_pred = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
6
7 dataset_evaluations = [
8     ("dataset1", y_true, (y_pred, y_pred))
9 ]
10
11 metrics = [
12     ("early_detection", {})
13 ]
14
15 runner = Runner(dataset_evaluations, metrics)
16 results = runner.run()
17 print(results)

```

5.2.2.3 Latency–Sparsity-aware F-Score

The Latency–Sparsity-aware F-Score metric extends the Point-adjusted F-Score by addressing two of its main limitations: the lack of explicit consideration of early detection, and the insufficient penalization of scattered false positives [22].

Given T and P , an adjusted sequence P' is constructed through two transformations:

1. **Latency adjustment:** for each anomalous segment in T , only the points from the first correctly detected anomaly in P onwards are considered true positives in P' . This penalizes late detections, since only the trailing portion of the segment is counted.

2. **False positive subsampling:** the sequence P is evaluated in consecutive blocks of length n . If at least one point in a block is incorrectly predicted as anomalous, a single false positive is counted for that block. This reduces the impact of isolated false positives and reflects more realistically their cognitive cost for a human expert.

Finally, the adjusted sequence P' is used in Equation 2.6 to compute the final score of the metric.

The parameters used by this metric can be summarized as follows:

- **ni (int):** size of the evaluation block used for false positive subsampling. Larger values make the metric less sensitive to isolated false positives. (*Default: 1*)
- **beta (float):** weighting factor that controls the relative importance of recall versus precision. (*Default: 1.0*)

This metric is implemented in the module `tsadmetrics.metrics.tem.dpm.LatencySparsityawareFScore`, and can be used either directly through the `LatencySparsityawareFScore` function (Section 5.2.2.3.1) or through the global evaluation pipeline provided by the `Runner` class (Section 5.2.2.3.2).

Examples of both usage modes are available in the library under: `examples/specific_examples/LatencySparsityawareFScore_example.py`.

5.2.2.3.1 Direct evaluation via LatencySparsityawareFScore Class

In this example, `y_true` represents the ground-truth binary labels indicating whether each time point is anomalous (1) or normal (0). Similarly, `y_pred` contains the predicted labels produced by an anomaly detection model.

```

1 from tsadmetrics.metrics.tem.dpm.LatencySparsityawareFScore import
   LatencySparsityawareFScore
2
3 y_true = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
4 y_pred = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
5
6 metric = LatencySparsityawareFScore(ni=1, beta=1.0)
7 result = metric.compute(y_true, y_pred)
8 print("LatencySparsityawareFScore:", result)

```

5.2.2.3.2 Automated evaluation via Runner Class

In this case, the same data sequences (`y_true` and `y_pred`) are organized into a dataset structure that allows batch evaluation across multiple datasets and metrics. The `Runner` class automatically handles the execution of the metric computations and the aggregation of results. For more details on the implementation and usage of the `Runner` class, consult Chapter 6.

```

1 from tsadmetrics.evaluation.Runner import Runner
2
3 dataset_evaluations = [
4     ("dataset1", y_true, y_pred)
5 ]
6
7 metrics = [
8     ("lsaf", {"ni":1, "beta":1.0})
9 ]
10
11 runner = Runner(dataset_evaluations, metrics)
12 results = runner.run()
13 print(results)

```

5.2.2.4 Mean Time to Detect

The Mean Time to Detect metric quantifies the average detection delay of the real anomalous events in R , given the predicted binary series P [23].

Let $R_k = [a_k, b_k]$ be a real anomalous event in R . For each event R_k such that there exists at least one predicted anomalous point at or after its start, the detection delay is defined as:

$$\Delta t_k = \min\{i \mid P_i = 1, i \geq a_k\} - a_k \quad (5.9)$$

This represents the time elapsed between the true start of the event (a_k) and the first predicted anomalous point detected at or after that start. Note that the predicted point does not need to lie inside the interval R_k ; it only needs to occur at or after a_k .

The final metric is computed as the sum of the detection delays of all detected real events, divided by the total number of real anomalous events:

$$\text{MeanTimeToDetect}(R, P) = \frac{1}{|R|} \sum_{R_k \in R_{\text{detected}}} \Delta t_k \quad (5.10)$$

where $R_{\text{detected}} \subseteq R$ denotes the subset of real events for which at least one predicted anomalous point exists at or after the event start. Therefore, real events in $R \setminus R_{\text{detected}}$ do not contribute to the numerator, but they are still included in the denominator. If no predicted anomalous points are found at or after any real anomalous event start, the metric returns 0 and a warning is issued.

This metric does not require additional parameters.

This metric is implemented in the module `tsadmetrics.metrics.tem.dpm.MeanTimeToDetect`, and can be used either directly through the `MeanTimeToDetect` function (Section 5.2.2.4.1) or through the global evaluation pipeline provided by the `Runner` class (Section 5.2.2.4.2).

Examples of both usage modes are available in the library under: `examples/specific_examples/MeanTimeToDetect_example.py`.

5.2.2.4.1 Direct evaluation via MeanTimeToDetect Class

In this example, `y_true` represents the ground-truth binary labels indicating whether each time point is anomalous (1) or normal (0). Similarly, `y_pred` contains the predicted labels produced by an anomaly detection model.

```

1 from tsadmetrics.metrics.tem.dpm.MeanTimeToDetect import MeanTimeToDetect
2
3 y_true = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
4 y_pred = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
5
6 metric = MeanTimeToDetect()
7 result = metric.compute(y_true, y_pred)
8 print("MeanTimeToDetect:", result)

```

5.2.2.4.2 Automated evaluation via Runner Class

In this case, the same data sequences (`y_true` and `y_pred`) are organized into a dataset structure that allows batch evaluation across multiple datasets and metrics. The `Runner` class automatically handles the execution of the metric computations and the aggregation of results. For more details on the implementation and usage of the `Runner` class, consult Chapter 6.

```

1 from tsadmetrics.evaluation.Runner import Runner
2
3 dataset_evaluations = [
4     ("dataset1", y_true, y_pred)
5 ]
6
7 metrics = [
8     ("mttd", {})
9 ]
10
11 runner = Runner(dataset_evaluations, metrics)
12 results = runner.run()
13 print(results)

```

5.2.2.5 NAB Score

The NAB Score metric evaluates the quality of anomaly detection predictions by simultaneously rewarding the correct and timely detection of real anomalous events and penalizing false alarms. It was proposed in the context of the *Numenta Anomaly Benchmark* [24] and is based on an asymmetric, time-sensitive scoring function.

Let $\mathcal{R} = \{R_1, R_2, \dots, R_m\}$ be the set of real anomalous events induced by the ground-truth binary series T , where each event is represented as an interval $R_k = [a_k, b_k]$. The NAB Score associates a detection window with each real anomalous event and evaluates predicted anomalous points with respect to these windows. The objective is to reward models that identify anomalous behavior as early as possible within each event, while penalizing false alarms outside the detection windows.

Let $\mathcal{P} = \{i \mid P_i = 1\}$ denote the set of predicted anomalous points in the binary prediction series P . Each predicted point contributes to the overall score through a combination of **positive**

rewards for timely detections inside the detection windows and **negative penalties** for false alarms outside them, resulting in a balanced metric that reflects both responsiveness and reliability.

True positive reward: for each real anomalous event $R_k \in \mathcal{R}$, only the earliest valid predicted point inside its detection window contributes positively to the score. This contribution is defined through a sigmoidal function that decreases as the detection occurs later within the event:

$$\text{score}_{\text{TP}}(p) = 2 \cdot \sigma(-5r(p)) - 1 \quad (5.11)$$

where $r(p)$ denotes the relative temporal position of prediction p within the detection window, and $\sigma(x) = \frac{1}{1+e^{-x}}$ is the sigmoid function. Thus, earlier detections receive higher rewards, while detections closer to the end of the event receive lower positive scores.

False positive penalty: predictions outside all detection windows are considered false positives and penalized through a negative sigmoidal contribution. For false positives occurring after a detection window, the magnitude of the penalty increases with their normalized temporal distance from the end of the most recent detection window. Predictions occurring before the first detection window receive the maximum negative contribution. Formally,

$$\text{score}_{\text{FP}}(p) = \lambda_{\text{FP}} (2 \cdot \sigma(-5r_{\text{FP}}(p)) - 1) \quad (5.12)$$

where $r_{\text{FP}}(p) > 0$ denotes the normalized temporal distance of prediction p to the right boundary of the preceding detection window, $\sigma(x) = \frac{1}{1+e^{-x}}$ is the sigmoid function, and $\lambda_{\text{FP}} > 0$ is the false-positive penalty weight. In the standard NAB formulation, this parameter is fixed by the authors to $\lambda_{\text{FP}} = 0.11$. Thus, false positives occurring close to the end of a detection window receive a milder penalty, whereas sufficiently distant false positives approach the maximum negative contribution.

Final score: the raw NAB score combines the positive contributions of detected events, the negative contributions of false positives, and the penalties associated with missed real events. Let p_k^* denote the earliest valid predicted point associated with the real event R_k , let $\mathcal{P}_{\text{FP}} \subseteq \mathcal{P}$ denote the set of predicted anomalous points that fall outside all detection windows, and let $\mathcal{R}_{\text{missed}} = \mathcal{R} \setminus \mathcal{R}_{\text{detected}}$ denote the set of real anomalous events for which no valid predicted point exists inside the corresponding detection window. Then, the raw NAB score can be written as

$$S_{\text{raw}}(T, P) = \sum_{R_k \in \mathcal{R}_{\text{detected}}} \text{score}_{\text{TP}}(p_k^*) + \sum_{p \in \mathcal{P}_{\text{FP}}} \text{score}_{\text{FP}}(p) - \lambda_{\text{FN}} |\mathcal{R}_{\text{missed}}| \quad (5.13)$$

where $\mathcal{R}_{\text{detected}} \subseteq \mathcal{R}$ is the subset of real anomalous events for which at least one valid predicted point exists inside the corresponding detection window, and $\lambda_{\text{FN}} > 0$ is the false-negative penalty weight. In the standard NAB formulation, this parameter is fixed by the authors to $\lambda_{\text{FN}} = 1$. The reported score is then normalized with respect to a null prediction and a perfect prediction:

$$\text{NabScore}(T, P) = 100 \cdot \frac{S_{\text{raw}}(T, P) - S_{\text{null}}(T)}{S_{\text{perfect}}(T) - S_{\text{null}}(T)} \quad (5.14)$$

where $S_{\text{null}}(T)$ denotes the score obtained when no anomaly is predicted, and $S_{\text{perfect}}(T)$ denotes the score obtained under perfect prediction. The maximum achievable score is 100 (early detection of all events without false positives), while the lower bound is unbounded due to accumulating penalties from false positives.

This metric does not require additional parameters.

This metric is implemented in the module `tsadmetrics.metrics.tem.dpm.NabScore`, and can be used either directly through the `NabScore` function (Section 5.2.2.5.1) or through the global evaluation pipeline provided by the `Runner` class (Section 5.2.2.5.2).

Examples of both usage modes are available in the library under: `examples/specific_examples/NabScore_example.py`.

5.2.2.5.1 Direct evaluation via NabScore Class

In this example, `y_true` represents the ground-truth binary labels indicating whether each time point is anomalous (1) or normal (0). Similarly, `y_pred` contains the predicted labels produced by an anomaly detection model.

```

1 from tsadmetrics.metrics.tem.dpm.NabScore import NabScore
2
3 y_true = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
4 y_pred = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
5
6 metric = NabScore()
7 result = metric.compute(y_true, y_pred)
8 print("NabScore:", result)

```

5.2.2.5.2 Automated evaluation via Runner Class

In this case, the same data sequences (`y_true` and `y_pred`) are organized into a dataset structure that allows batch evaluation across multiple datasets and metrics. The `Runner` class automatically handles the execution of the metric computations and the aggregation of results. For more details on the implementation and usage of the `Runner` class, consult Chapter 6.

```

1 from tsadmetrics.evaluation.Runner import Runner
2
3 dataset_evaluations = [
4     ("dataset1", y_true, y_pred)
5 ]
6
7 metrics = [
8     ("nab_score", {})
9 ]
10
11 runner = Runner(dataset_evaluations, metrics)
12 results = runner.run()
13 print(results)

```

5.2.3 Precise Temporal Detection Metrics

This category groups metrics that require the detection Q_i to cover fully or partially a significant fraction of the duration of the real anomalous event R_j , with lower tolerance for errors in position or duration compared to TPDM. These metrics evaluate not only whether an anomaly is detected, but also how precise the detection is in temporal terms.

PTDM metrics are particularly useful in contexts where partial detection is not sufficiently informative, and high temporal precision is required to accurately characterize anomalous events. These metrics prioritize accurate alignment with the real event, ensuring that both the timing and duration of detections reflect the actual anomaly occurrence.

The following diagram illustrates the class structure representing the metrics in this category.

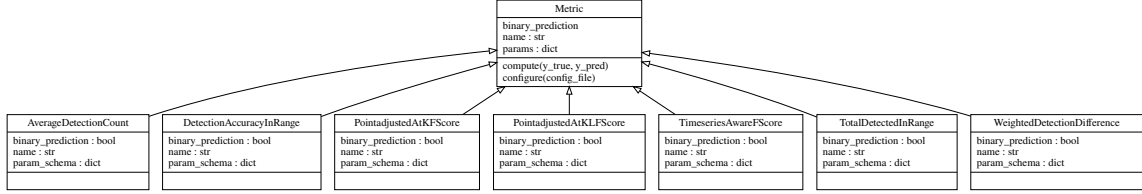


Figure 5.4: Precise Temporal Detection Metrics class diagram

5.2.3.1 Average Detection Count

The Average Detection Count metric computes, for each anomalous event $R_i \in R$, the proportion of its points that are correctly identified as anomalous in the predicted sequence P . In other words, it measures the fraction of points in R_i that also appear within any predicted anomaly range $Q_j \in Q$. The mean of these proportions is then calculated across all events $R_i \in R$, providing a global estimate of the number of detected points per anomalous event [25].

The computation is formally expressed as:

$$\text{AverageDetectionCount}(R, Q) = \frac{1}{|R|} \sum_{R_i \in R} \frac{|R_i \cap (\bigcup_{Q_j \in Q} Q_j)|}{|R_i|} \quad (5.15)$$

This metric does not require additional parameters.

This metric is implemented in the module `tsadmetrics.metrics.tem.ptdm.AverageDetectionCount`, and can be used either directly through the `AverageDetectionCount` function (Section 5.2.3.1.1) or through the global evaluation pipeline provided by the `Runner` class (Section 5.2.3.1.2).

Examples of both usage modes are available in the library under: `examples/specific_examples/AverageDetectionCount_example.py`.

5.2.3.1.1 Direct evaluation via AverageDetectionCount Class

In this example, `y_true` represents the ground-truth binary labels indicating whether each time point is anomalous (1) or normal (0). Similarly, `y_pred` contains the predicted labels produced by an anomaly detection model.

```

1 from tsadmetrics.metrics.tem.ptdm.AverageDetectionCount import AverageDetectionCount
2
3 y_true = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
4 y_pred = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
5
6 metric = AverageDetectionCount()
7 result = metric.compute(y_true, y_pred)
8 print("AverageDetectionCount:", result)
  
```

5.2.3.1.2 Automated evaluation via Runner Class

In this case, the same data sequences (`y_true` and `y_pred`) are organized into a dataset structure that allows batch evaluation across multiple datasets and metrics. The `Runner` class automatically handles the execution of the metric computations and the aggregation of results. For more details on the implementation and usage of the `Runner` class, consult Chapter 6.

```

1  from tsadmetrics.evaluation.Runner import Runner
2
3  dataset_evaluations = [
4      ("dataset1", y_true, y_pred)
5  ]
6
7  metrics = [
8      ("adc", {})
9  ]
10
11 runner = Runner(dataset_evaluations, metrics)
12 results = runner.run()
13 print(results)

```

5.2.3.2 Counting metrics

This group of metrics is based on a redefinition of the concepts of TP, FP, TN and FN in the context of anomaly detection in time series. Specifically, the following categories are introduced by comparing the ground truth sequence T with the predicted sequence P , using a parameter k that defines the size of a tolerance window for partial detection (for instance, if $k = 1$, the tolerance window for point i is $[i - k, i + k]$) [26]:

1. **Exact Match (EM)**: occurs when a real anomalous point ($T_i = 1$) exactly coincides with a predicted anomalous point ($P_i = 1$).
2. **Detected Anomaly (DA)**: occurs when a real anomalous point ($T_i = 1$) does not exactly coincide with a predicted anomalous point ($P_i = 1$), but there exists at least one predicted anomaly within the range $[i - k, i + k]$. This is considered a valid partial detection.
3. **Missed Anomaly (MA)**: occurs when a real anomalous point ($T_i = 1$) has no predicted anomalous point ($P_j = 1$) within the range $[i - k, i + k]$. This is considered a missed detection.
4. **False Anomaly (FA)**: occurs when a predicted anomalous point ($P_i = 1$) does not coincide with any real anomalous point ($T_j = 1$), nor does it fall within the range $[i - k, i + k]$ of any real anomaly. This is considered a false positive.

These metrics are referred to as *counting metrics* because they share the same paradigm: redefining TP, FP, TN and FN using the above categories, in order to adapt them to pointwise anomaly detection with temporal tolerance. Importantly, counting metrics do not constitute an additional type in the taxonomy of evaluation metrics, but rather a family of metrics built upon a common counting criterion.

5.2.3.2.1 Total Detected in Range

The Total Detected in Range metric measures the proportion of real anomalous points that are correctly detected, either through an exact match (**EM**) or through a partial detection within the tolerance window defined by k (**DA**). In other words, it quantifies the model's ability to recognize real anomalous points while allowing for a margin of temporal error.

$$TDIR = \frac{EM + DA}{EM + DA + MA} \quad (5.16)$$

The parameters used by this metric can be summarized as follows:

- **k (int)**: size of the temporal tolerance window. A point in the prediction sequence is considered a valid detection if it lies within $[i - k, i + k]$ of a true anomalous point. (*Default: 5*)

This metric is implemented in the module `tsadmetrics.metrics.tem.ptdm.TotalDetectedInRange`, and can be used either directly through the `TotalDetectedInRange` function (Section 5.2.3.2.1.1) or through the global evaluation pipeline provided by the `Runner` class (Section 5.2.3.2.1.2).

Examples of both usage modes are available in the library under: `examples/specific_examples/TotalDetectedInRange_example.py`.

5.2.3.2.1.1 Direct evaluation via TotalDetectedInRange Class

In this example, `y_true` represents the ground-truth binary labels indicating whether each time point is anomalous (1) or normal (0). Similarly, `y_pred` contains the predicted labels produced by an anomaly detection model.

```

1 from tsadmetrics.metrics.tem.ptdm.TotalDetectedInRange import TotalDetectedInRange
2
3 y_true = [0,0,0,0,0,1,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
4 y_pred = [0,0,0,0,0,1,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
5
6 metric = TotalDetectedInRange(k=5)
7 result = metric.compute(y_true, y_pred)
8 print("TotalDetectedInRange:", result)

```

5.2.3.2.1.2 Automated evaluation via Runner Class

In this case, the same data sequences (`y_true` and `y_pred`) are organized into a dataset structure that allows batch evaluation across multiple datasets and metrics. The `Runner` class automatically handles the execution of the metric computations and the aggregation of results. For more details on the implementation and usage of the `Runner` class, consult Chapter 6.

```

1 from tsadmetrics.evaluation.Runner import Runner
2
3 dataset_evaluations = [
4     ("dataset1", y_true, y_pred)
5 ]
6
7 metrics = [
8     ("tdir", {"k":5})
9 ]
10
11 runner = Runner(dataset_evaluations, metrics)
12 results = runner.run()
13 print(results)

```

5.2.3.2.2 Detection Accuracy in Range

The Detection Accuracy in Range metric measures the proportion of predicted anomalous points that are correct, either through an exact match (**EM**) or through a partial detection within the tolerance window defined by k (**DA**). In other words, it quantifies the *precision* of the model when predicting anomalous points, while considering a margin of temporal tolerance.

$$DAIR = \frac{EM + DA}{EM + DA + FA} \quad (5.17)$$

The parameters used by this metric can be summarized as follows:

- **k** (int): size of the temporal tolerance window. A point in the prediction sequence is considered a valid detection if it lies within $[i - k, i + k]$ of a true anomalous point. (*Default: 5*)

This metric is implemented in the module `tsadmetrics.metrics.tem.ptdm.DetectionAccuracyInRange`, and can be used either directly through the `DetectionAccuracyInRange` function (Section 5.2.3.2.2.1) or through the global evaluation pipeline provided by the `Runner` class (Section 5.2.3.2.2.2).

Examples of both usage modes are available in the library under: `examples/specific_examples/DetectionAccuracyInRange_example.py`.

5.2.3.2.2.1 Direct evaluation via DetectionAccuracyInRange Class

In this example, `y_true` represents the ground-truth binary labels indicating whether each time point is anomalous (1) or normal (0). Similarly, `y_pred` contains the predicted labels produced by an anomaly detection model.

```

1 from tsadmetrics.metrics.tem.ptdm.DetectionAccuracyInRange import DetectionAccuracyInRange
2
3 y_true = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
4 y_pred = [0,0,0,0,0,1,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1]
5
6 metric = DetectionAccuracyInRange(k=5)
7 result = metric.compute(y_true, y_pred)
8 print("DetectionAccuracyInRange:", result)

```

5.2.3.2.2 Automated evaluation via Runner Class

In this case, the same data sequences (`y_true` and `y_pred`) are organized into a dataset structure that allows batch evaluation across multiple datasets and metrics. The `Runner` class automatically handles the execution of the metric computations and the aggregation of results. For more details on the implementation and usage of the `Runner` class, consult Chapter 6.

```

1 from tsadmetrics.evaluation.Runner import Runner
2
3 dataset_evaluations = [
4     ("dataset1", y_true, y_pred)
5 ]
6
7 metrics = [
8     ("dair", {"k":5})
9 ]
10
11 runner = Runner(dataset_evaluations, metrics)
12 results = runner.run()
13 print(results)

```

5.2.3.2.3 Weighted Detection Difference

The Weighted Detection Difference metric introduces a distance-based weighting mechanism to assess the quality of anomaly detections, giving higher relevance to points located near the center of each real anomalous event. The underlying intuition is that central points within an anomaly range better represent the event's characteristic behavior, while peripheral points contribute less to its identification.

For each real anomalous range $R_i \in R$, a weight $w(t_j)$ is assigned to every point $t_j \in R_i$ according to its distance from the center of the event. The weighting function is defined as:

$$w(d) = 1 - \frac{d}{L_i} \quad (5.18)$$

where $d = |t_j - c_i|$ is the absolute distance between the point t_j and the event center $c_i = \frac{a_i + b_i}{2}$, and $L_i = b_i - a_i + 1$ is the length of the event R_i . This linear function assigns higher weights to points closer to the event center, gradually decreasing toward the boundaries.

Using this weighting scheme, the metric first computes a **weighted score (WS)** that measures the quality of correctly detected anomalies for the given pair (T, P) . For each predicted anomalous point $P_l = 1$, the contribution is weighted according to the function $w(d)$, taking into account its proximity to the nearest real anomalous range R_i :

$$WS(T, P) = \sum_{l=1}^{N_p} P_l \cdot w(d_l) \quad (5.19)$$

where d_l denotes the distance from the predicted point P_l to the center of the closest real anomaly in T .

In addition, a penalty term is introduced for false alarms (**FA**), corresponding to predicted anomalous points that do not belong to any real range in T . These are also weighted according to their distance to the nearest true event, using the same weighting function and a penalization factor w_f :

$$FA(T, P) = \sum_{l=1}^{N_{fa}} w(d_l) \quad (5.20)$$

Finally, **Weighted Detection Difference** for the pair (T, P) is defined as:

$$WeightedDetectionDifference(T, P) = WS(T, P) - w_f \cdot FA(T, P) \quad (5.21)$$

where $WS(T, P)$ represents the weighted quality of correct detections, and $w_f \cdot FA(T, P)$ penalizes false positive predictions according to their proximity to real anomalies.

This formulation allows the metric to balance the reward for well-localized detections and the penalty for spurious predictions, adapting naturally to the temporal structure of anomalous events.

The parameters used by this metric can be summarized as follows:

- **k (int)**: size of the temporal tolerance window. A point in the prediction sequence is considered a valid detection if it lies within $[i - k, i + k]$ of a true anomalous point. (*Default*: 5)

This metric is implemented in the module `tsadmetrics.metrics.tem.ptdm.WeightedDetectionDifference`, and can be used either directly through the `WeightedDetectionDifference` function (Section 5.2.3.2.3.1) or through the global evaluation pipeline provided by the `Runner` class (Section 5.2.3.2.3.2).

Examples of both usage modes are available in the library under: `examples/specific_examples/WeightedDetectionDifference_example.py`.

5.2.3.2.3.1 Direct evaluation via WeightedDetectionDifference Class

In this example, `y_true` represents the ground-truth binary labels indicating whether each time point is anomalous (1) or normal (0). Similarly, `y_pred` contains the predicted labels produced by an anomaly detection model.

```

1 from tsadmetrics.metrics.tem.ptdm.WeightedDetectionDifference import
   WeightedDetectionDifference
2
3 y_true = [0,0,0,0,0,1,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1,1]
4 y_pred = [0,0,0,0,0,1,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1,1]
5
6 metric = WeightedDetectionDifference(k=5)
7 result = metric.compute(y_true, y_pred)
8 print("WeightedDetectionDifference:", result)

```

5.2.3.2.3.2 Automated evaluation via Runner Class

In this case, the same data sequences (`y_true` and `y_pred`) are organized into a dataset structure that allows batch evaluation across multiple datasets and metrics. The `Runner` class automatically handles the execution of the metric computations and the aggregation of results. For more details on the implementation and usage of the `Runner` class, consult Chapter 6.

```

1 from tsadmetrics.evaluation.Runner import Runner
2
3 dataset_evaluations = [
4     ("dataset1", y_true, y_pred)
5 ]
6
7 metrics = [
8     ("wdd", {"k":5})
9 ]
10
11 runner = Runner(dataset_evaluations, metrics)
12 results = runner.run()
13 print(results)

```

5.2.3.3 Point-adjusted at K% F-Score

The Point-adjusted at K% F-Score metric is a variant of the `PointadjustedFScore` that introduces an additional requirement on the detection proportion for each anomalous segment [27].

To construct the adjusted prediction sequence P' , a real anomalous segment in T is considered correctly detected only if at least a fraction $K\%$ of its points have been identified as anomalous in the predicted sequence P . If this threshold is not reached, the segment is not marked as detected and therefore does not contribute as a true positive at the segment level [28].

However, individual points correctly identified within such under-detected segments are still preserved as true positives for point-level evaluation. This ensures that partial detections retain some contribution to the final score, even when the overall segment-level detection criterion is not satisfied.

The parameters used by this metric can be summarized as follows:

- **beta (float):** weighting factor that controls the relative importance of recall versus precision. (*Default: 1.0*)
- **k (float):** minimum proportion (expressed as a percentage or fraction) of an anomalous segment that must be detected in order to count the segment as a true positive at the segment level. (*Default: 0.5*)

This metric is implemented in the module `tsadmetrics.metrics.tem.ptdm.PointadjustedAtKFScore`, and can be used either directly through the `PointadjustedAtKFScore` function (Section 5.2.3.3.1) or through the global evaluation pipeline provided by the `Runner` class (Section 5.2.3.3.2).

Examples of both usage modes are available in the library under: `examples/specific_examples/PointadjustedAtKFScore_example.py`.

5.2.3.3.1 Direct evaluation via PointadjustedAtKFScore Class

In this example, `y_true` represents the ground-truth binary labels indicating whether each time point is anomalous (1) or normal (0). Similarly, `y_pred` contains the predicted labels produced by an anomaly detection model.

```

1 from tsadmetrics.metrics.tem.ptdm.PointadjustedAtKFScore import PointadjustedAtKFScore
2
3 y_true = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
4 y_pred = [0,0,0,0,0,1,1,1,0,0,1,1,1,0,0,0,0,0,0,1,1,0,1,1,1,1]
5
6 metric = PointadjustedAtKFScore(beta=1.0, k_ratio=0.5)
7 result = metric.compute(y_true, y_pred)
8 print("PointadjustedAtKFScore:", result)

```

5.2.3.3.2 Automated evaluation via Runner Class

In this case, the same data sequences (`y_true` and `y_pred`) are organized into a dataset structure that allows batch evaluation across multiple datasets and metrics. The `Runner` class automatically handles the execution of the metric computations and the aggregation of results. For more details on the implementation and usage of the `Runner` class, consult Chapter 6.

```

1 from tsadmetrics.evaluation.Runner import Runner
2
3 dataset_evaluations = [
4     ("dataset1", y_true, y_pred)
5 ]
6
7 metrics = [
8     ("patkf", {"beta":1.0, "k_ratio":0.5})
9 ]
10
11 runner = Runner(dataset_evaluations, metrics)
12 results = runner.run()
13 print(results)

```

5.2.3.4 Point-adjusted at K% F-Score with tolerance l

The Point-adjusted at K% F-Score with tolerance l metric builds upon the Point-adjusted at K% F-Score by introducing a refined temporal adjustment to the prediction sequence P' that controls how detected points are counted within correctly identified anomaly segments [27].

As in the base metric, each real anomalous segment in the ground-truth sequence T is first evaluated to determine whether at least a fraction $K\%$ of its points have been predicted as anomalous in P . Only if this detection ratio is reached is the segment considered successfully identified.

However, unlike the original metric, where all points in the detected segment are marked as true positives, this variant applies a temporal tolerance window of size l around each predicted positive point within that segment. Thus, only the points inside the intersection between the real anomalous segment and the $\pm l$ neighborhood of each detected prediction are marked as true positives in the adjusted prediction P' . Points outside these tolerance zones remain unchanged, and no additional false positives are generated beyond the true anomaly boundaries.

After constructing the adjusted prediction P' , precision and recall are computed pointwise against the ground-truth labels, and the final score is obtained using the standard F_β formulation (Equation 2.6). If either precision or recall equals zero, the resulting score is defined as 0.

The parameters used by this metric can be summarized as follows:

- **k (float)**: minimum proportion of an anomaly segment that must be detected to count as correctly identified. (*Default*: 0.5)
- **l (int)**: tolerance window in time steps; detections within $\pm l$ positions of a true anomaly are counted as correct. (*Default*: 1)
- **beta (float)**: weighting factor that controls the relative importance of recall versus precision. (*Default*: 1.0)

This metric is implemented in the module `tsadmetrics.metrics.tem.dpm.PointadjustedAtKLFScore`, and can be used either directly through the `PointadjustedAtKLFScore` class (Section 5.2.3.4.1) or through the global evaluation pipeline provided by the `Runner` class (Section 5.2.3.4.2).

Examples of both usage modes are available in the library under: `examples/specific_examples/PointadjustedAtKLFScore_example.py`.

5.2.3.4.1 Direct evaluation via PointadjustedAtKLFScore Class

In this example, `y_true` represents the ground-truth binary labels indicating whether each time point is anomalous (1) or normal (0). Similarly, `y_pred` contains the predicted labels produced by an anomaly detection model.

```
1 from tsadmetrics.metrics.tem.dpm.PointadjustedAtKLFScore import PointadjustedAtKLFScore
2
3 y_true = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
4 y_pred = [0,0,0,0,0,1,1,0,0,1,1,1,0,0,0,0,0,0,0,1,0,1,1,1,1,0]
5
6 metric = PointadjustedAtKLFScore(k=0.5, l=1, beta=1.0)
7 result = metric.compute(y_true, y_pred)
8 print("PointadjustedAtKLFScore:", result)
```

5.2.3.4.2 Automated evaluation via Runner Class

In this case, the same data sequences (`y_true` and `y_pred`) are organized into a dataset structure that allows batch evaluation across multiple datasets and metrics. The `Runner` class automatically handles the execution of the metric computations and the aggregation of results. For more details on the implementation and usage of the `Runner` class, consult Chapter 6.

```
1 from tsadmetrics.evaluation.Runner import Runner
2
3 y_true = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
4 y_pred = [0,0,0,0,0,1,1,0,0,1,1,1,0,0,0,0,0,0,0,1,0,1,1,1,1,0]
5
6 dataset_evaluations = [
7     ("dataset1", y_true, y_pred)
8 ]
```

```

9
10 metrics = [
11     ("paklf", {"k":0.5, "l":1, "beta":1.0})
12 ]
13
14 runner = Runner(dataset_evaluations, metrics)
15 results = runner.run()
16 print(results)

```

5.2.3.5 Timeseries-Aware F-Score

The Timeseries-Aware F-Score metric is based on the Range-based F-Score, but introduces two key modifications.

First, a predicted anomalous segment is only considered correctly detected if it covers at least a fraction θ of the ground-truth anomaly range. This rewards sufficient coverage of anomalous events while discouraging fragmented predictions [29].

Second, the metric incorporates a tolerance region of length δ extending beyond the right endpoint of each anomalous event. Within this extended region, the contribution to the score gradually decreases as the distance from the original endpoint increases, allowing some flexibility in delimiting the end of an event and reducing penalties for minor temporal deviations.

Importantly, the interpretation of δ depends on the evaluation mode: when `past_range` is enabled, $\delta \in (0, 1]$ specifies a fractional extension relative to each segment's length; otherwise, $\delta \geq 0$ denotes an absolute number of time steps.

This variant does not consider cardinality or positional bias, focusing exclusively on the minimum required overlap (θ) and the end-tolerance decay (δ).

The parameters used by this metric can be summarized as follows:

- **beta (float)**: weighting factor that controls the relative importance of recall versus precision. (*Default*: 1.0)
- **alpha (float)**: balance between existence and overlap terms in the range-based components. (*Default*: 0.5)
- **delta (float)**: length of the temporal tolerance region appended to the right endpoint of each event. Its meaning depends on `past_range`: if `True`, $\delta \in (0, 1]$ is a fraction of the event length; if `False`, $\delta \geq 0$ is an absolute number of time steps. (*Default*: 5)
- **theta (float)**: minimum overlap fraction ($0 \leq \theta \leq 1$) between predicted and true ranges required to consider a segment detected. (*Default*: 0.5)
- **past_range (bool)**: toggles the interpretation of δ . If `True`, δ is fractional w.r.t. segment length; if `False`, δ is an absolute number of time steps. (*Default*: `False`)

This metric is implemented in the module `tsadmetrics.metrics.tem.ptdm.TimeseriesAwareFScore`, and can be used either directly through the `TimeseriesAwareFScore` function (Section 5.2.3.5.1) or through the global evaluation pipeline provided by the `Runner` class (Section 5.2.3.5.2).

Examples of both usage modes are available in the library under: `examples/specific_examples/TimeseriesAwareFScore_example.py`.

5.2.3.5.1 Direct evaluation via TimeseriesAwareFScore Class

In this example, `y_true` represents the ground-truth binary labels indicating whether each time point is anomalous (1) or normal (0). Similarly, `y_pred` contains the predicted labels produced by an anomaly detection model.

```

1 from tsadmetrics.metrics.tem.ptdm.TimeseriesAwareFScore import TimeseriesAwareFScore
2
3 y_true = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
4 y_pred = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
5
6 metric = TimeseriesAwareFScore(beta=1.0, alpha=0.5, delta=5, theta=0.5, past_range=False)
7 result = metric.compute(y_true, y_pred)
8 print("TimeseriesAwareFScore:", result)

```

5.2.3.5.2 Automated evaluation via Runner Class

In this case, the same data sequences (`y_true` and `y_pred`) are organized into a dataset structure that allows batch evaluation across multiple datasets and metrics. The `Runner` class automatically handles the execution of the metric computations and the aggregation of results. For more details on the implementation and usage of the `Runner` class, consult Chapter 6.

```

1 from tsadmetrics.evaluation.Runner import Runner
2
3 dataset_evaluations = [
4     ("dataset1", y_true, y_pred)
5 ]
6
7 metrics = [
8     ("taf", {"beta":1.0, "alpha":0.5, "delta":5, "theta":0.5, "past_range":False})
9 ]
10
11 runner = Runner(dataset_evaluations, metrics)
12 results = runner.run()
13 print(results)

```

5.2.4 Temporal Matching Evaluation Metrics

This category includes metrics that measure how well real anomalous events R_i are aligned with predicted anomalous events Q_i , penalizing temporal deviations in start, duration, or end.

TMEM metrics are particularly suitable for contexts where it is crucial to accurately evaluate the temporal alignment between predictions and real events, ensuring that the model not only detects anomalies but also precisely locates them in time. This is essential in applications where temporal accuracy directly affects the interpretation or response to detected events.

The following diagram illustrates the class structure representing the metrics in this category.

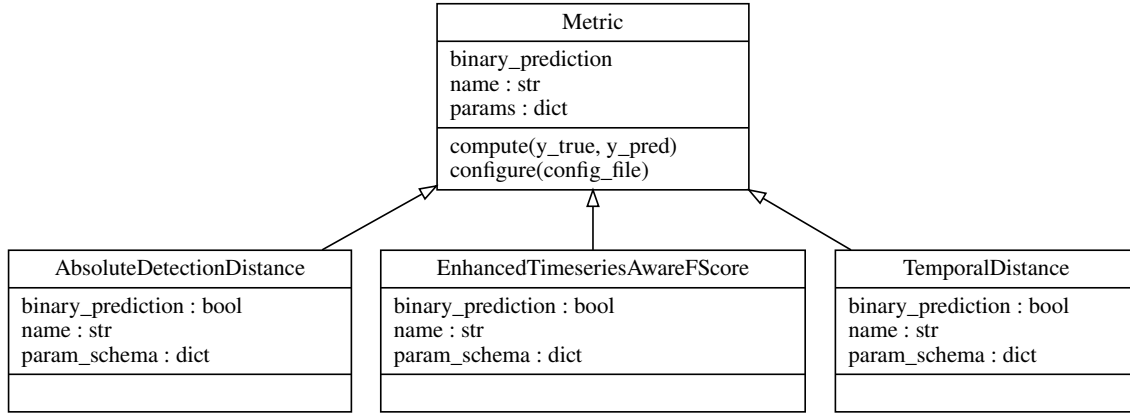


Figure 5.5: Temporal Matching Evaluation Metrics class diagram

5.2.4.1 Absolute Detection Distance

The Absolute Detection Distance metric evaluates, for each predicted point $P_i \in P$ that coincides with a point belonging to a real anomalous segment $R_j \in R$, the distance between P_i and the center of that segment R_j . In other words, if $P_i \in R_j$, its distance to the central point of R_j , defined as the midpoint between its endpoints, is computed. All these distances are then summed and divided by the total number of predicted points that match any real segment $R_j \in R$ [25]. Formally:

$$\text{AbsoluteDetectionDistance}(P, R) = \frac{1}{|\mathcal{P}_{\text{match}}|} \sum_{P_i \in \mathcal{P}_{\text{match}}} |P_i - \text{center}(R_j)| \quad \text{where } P_i \in R_j \quad (5.22)$$

Here, $\mathcal{P}_{\text{match}} = \{P_i \in P \mid \exists R_j \in R : P_i \in R_j\}$ denotes the set of predicted points that coincide with any real anomalous segment. If no predicted anomalous points match any real anomalous segment, the metric returns 0 and a warning is issued.

This metric provides an estimate of the spatial *precision* of correct detections, measuring how centrally located predictions are within real anomalous events. A lower score indicates that predictions tend to be near the center of anomalous segments, reflecting better localization of the core of the event.

This metric does not require additional parameters.

This metric is implemented in the module `tsadmetrics.metrics.tem.tmem.AbsoluteDetectionDistance`, and can be used either directly through the `AbsoluteDetectionDistance` function (Section 5.2.4.1.1) or through the global evaluation pipeline provided by the `Runner` class (Section 5.2.4.1.2).

Examples of both usage modes are available in the library under: `examples/specific_examples/AbsoluteDetectionDistance_example.py`.

5.2.4.1.1 Direct evaluation via AbsoluteDetectionDistance Class

In this example, `y_true` represents the ground-truth binary labels indicating whether each time point is anomalous (1) or normal (0). Similarly, `y_pred` contains the predicted labels produced by an anomaly detection model.

```

1 from tsadmetrics.metrics.tem.tmem.AbsoluteDetectionDistance import
   AbsoluteDetectionDistance
2
3 y_true = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
4 y_pred = [0,0,0,0,0,1,1,0,0,0,0,0,0,0,0,0,0,0,1,1,1,1,1,1,1]
5
6 metric = AbsoluteDetectionDistance()
7 result = metric.compute(y_true, y_pred)
8 print("AbsoluteDetectionDistance:", result)

```

5.2.4.1.2 Automated evaluation via Runner Class

In this case, the same data sequences (`y_true` and `y_pred`) are organized into a dataset structure that allows batch evaluation across multiple datasets and metrics. The `Runner` class automatically handles the execution of the metric computations and the aggregation of results. For more details on the implementation and usage of the `Runner` class, consult Chapter 6.

```

1 from tsadmetrics.evaluation.Runner import Runner
2
3 dataset_evaluations = [
4     ("dataset1", y_true, y_pred)
5 ]
6
7 metrics = [
8     ("add", {})
9 ]
10
11 runner = Runner(dataset_evaluations, metrics)
12 results = runner.run()
13 print(results)

```

5.2.4.2 Enhanced Timeseries-Aware F-Score

The Enhanced Timeseries-Aware F-Score metric is similar to the Range-based F-Score in that it considers both a detection score and an overlap score. Additionally, it introduces an extra quality criterion: for a real anomalous event in T to be considered correctly detected, at least a fraction θ_1 of its points must have been identified as anomalous in P . Symmetrically, a predicted event in P must overlap with at least a fraction θ_2 of some real event in T . This dual threshold ensures that partial or irrelevant detections do not contribute significantly to the metric.

Finally, the precision is computed by weighting each event according to the square root of its length, which balances sensitivity between point-wise methods and full-segment detection approaches [30].

The parameters used by this metric can be summarized as follows:

- `theta_p` (float): minimum fraction of points in a real anomalous segment that must be detected to consider it correctly identified. (*Default: 0.5*)
- `theta_r` (float): minimum fraction of overlap required for a predicted segment to be considered a valid detection of a real anomaly. (*Default: 0.5*)

- **beta** (float): weighting factor that controls the relative importance of recall versus precision. (Default: 1.0)

This metric is implemented in the module `tsadmetrics.metrics.tem.tmem.EnhancedTimeseriesAwareFScore`, and can be used either directly through the `EnhancedTimeseriesAwareFScore` function (Section 5.2.4.2.1) or through the global evaluation pipeline provided by the `Runner` class (Section 5.2.4.2.2).

Examples of both usage modes are available in the library under: `examples/specific_examples/EnhancedTimeseriesAwareFScore_example.py`.

5.2.4.2.1 Direct evaluation via EnhancedTimeseriesAwareFScore Class

In this example, `y_true` represents the ground-truth binary labels indicating whether each time point is anomalous (1) or normal (0). Similarly, `y_pred` contains the predicted labels produced by an anomaly detection model.

```

1 from tsadmetrics.metrics.tem.tmem.EnhancedTimeseriesAwareFScore import
   EnhancedTimeseriesAwareFScore
2
3 y_true = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
4 y_pred = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
5
6 metric = EnhancedTimeseriesAwareFScore(theta_p=0.5, theta_r=0.5)
7 result = metric.compute(y_true, y_pred)
8 print("EnhancedTimeseriesAwareFScore:", result)

```

5.2.4.2.2 Automated evaluation via Runner Class

In this case, the same data sequences (`y_true` and `y_pred`) are organized into a dataset structure that allows batch evaluation across multiple datasets and metrics. The `Runner` class automatically handles the execution of the metric computations and the aggregation of results. For more details on the implementation and usage of the `Runner` class, consult Chapter 6.

```

1 from tsadmetrics.evaluation.Runner import Runner
2
3 dataset_evaluations = [
4     ("dataset1", y_true, y_pred)
5 ]
6
7 metrics = [
8     ("etaf", {"theta_p":0.5, "theta_r":0.5})
9 ]
10
11 runner = Runner(dataset_evaluations, metrics)
12 results = runner.run()
13 print(results)

```

5.2.4.3 Temporal Distance

The Temporal Distance metric evaluates the quality of anomaly detection by measuring the temporal proximity between real anomalous points and predicted points. Specifically, it computes the sum of distances between each real anomalous point and its closest predicted point, as well as between each predicted point and its closest real anomaly. This approach penalizes both missed detections and false positives proportionally to their temporal displacement [26]. Formally, the metric is defined as:

$$\text{TemporalDistance}(T, P) = \sum_{t \in T} \min_{p \in P} \|t - p\|^d + \sum_{p \in P} \min_{t \in T} \|p - t\|^d \quad (5.23)$$

where $d = 1$ corresponds to the standard temporal **distance**, and $d = 2$ corresponds to the squared temporal **distance**, which penalizes distant errors more heavily. Low values of this metric indicate good temporal alignment between predicted and true anomalies, whereas higher values reflect misalignment or dispersed erroneous predictions.

The parameters used by this metric can be summarized as follows:

- **distance (int)**: exponent applied to the temporal difference between real and predicted anomaly points. A value of *distance* = 0 corresponds to the standard temporal distance, while *distance* = 1 yields the squared temporal distance, increasing the penalty for distant mismatches. (*Default*: 0)

This metric is implemented in the module `tsadmetrics.metrics.tem.tmem.TemporalDistance`, and can be used either directly through the `TemporalDistance` function (Section 5.2.4.3.1) or through the global evaluation pipeline provided by the `Runner` class (Section 5.2.4.3.2).

Examples of both usage modes are available in the library under: `examples/specific_examples/TemporalDistance_example.py`.

5.2.4.3.1 Direct evaluation via TemporalDistance Class

In this example, `y_true` represents the ground-truth binary labels indicating whether each time point is anomalous (1) or normal (0). Similarly, `y_pred` contains the predicted labels produced by an anomaly detection model.

```

1 from tsadmetrics.metrics.tem.tmem.TemporalDistance import TemporalDistance
2
3 y_true = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1]
4 y_pred = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1]
5
6 metric = TemporalDistance(distance=0) # distance=0 for standard, distance=1 for squared
7 result = metric.compute(y_true, y_pred)
8 print("TemporalDistance:", result)

```

5.2.4.3.2 Automated evaluation via Runner Class

In this case, the same data sequences (`y_true` and `y_pred`) are organized into a dataset structure that allows batch evaluation across multiple datasets and metrics. The `Runner` class automatically handles the execution of the metric computations and the aggregation of results. For more details on the implementation and usage of the `Runner` class, consult Chapter 6.

```

1 from tsadmetrics.evaluation.Runner import Runner
2
3 dataset_evaluations = [
4     ("dataset1", y_true, y_pred)
5 ]
6
7 metrics = [
8     ("td", {"distance":0}) # distance=0 for standard, distance=1 for squared
9 ]
10
11 runner = Runner(dataset_evaluations, metrics)
12 results = runner.run()
13 print(results)

```

5.2.5 Temporal Shift-Tolerant Metrics

This category includes metrics that allow a temporal tolerance in the detection of an anomalous event R_i , meaning that the associated prediction may occur slightly before or after the actual interval of the event. These metrics consider detections near the real event as correct, even if they do not exactly match its start or end. This temporal flexibility is particularly useful when strict temporal accuracy is not critical, but it is important that the model identifies the event within a reasonable time window.

TSTM metrics are especially suitable for contexts where minor temporal deviations in detection do not significantly affect the interpretation or utility of the anomalous event.

The following diagram illustrates the class structure representing the metrics in this category.

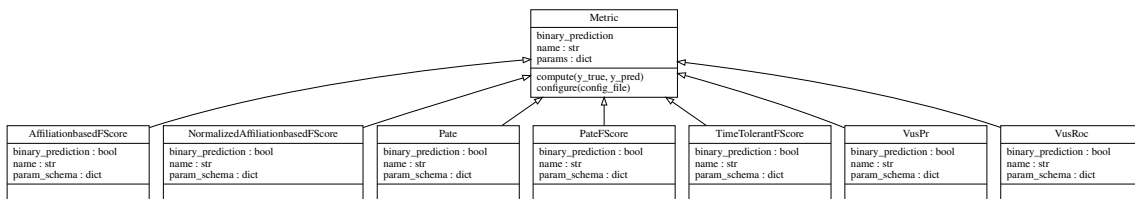


Figure 5.6: Temporal Shift-Tolerant Metrics class diagram

5.2.5.1 Affiliation-based F-Score

The Affiliation-based F-Score metric evaluates the quality of anomaly predictions by considering the temporal proximity between predicted anomalous points in sequence P and labeled anomalous points in sequence T . Unlike range- or segment-based metrics that assess overlap between aggregated events, this metric analyzes each anomalous event individually, providing a more localized evaluation of detection quality [31].

For each real anomalous event $R_i \in R$, a **local recall** is computed as the average distance between each point $t \in R_i$ and the closest predicted point in P :

$$\text{recall}_{\text{local}}(R_i) = \frac{1}{|R_i|} \sum_{t \in R_i} \min_{p \in P} |t - p| \quad (5.24)$$

Similarly, for each predicted anomalous event $Q_j \in Q$, a **local precision** is computed as the average distance between each point $p \in Q_j$ and the nearest labeled point in T :

$$\text{precision}_{\text{local}}(Q_j) = \frac{1}{|Q_j|} \sum_{p \in Q_j} \min_{t \in T} |p - t| \quad (5.25)$$

The **global precision** and **global recall** are then calculated as the mean of all local precision and recall values, respectively:

$$\text{precision}_{\text{global}} = \frac{1}{|Q|} \sum_{Q_j \in Q} \text{precision}_{\text{local}}(Q_j) \quad (5.26)$$

$$\text{recall}_{\text{global}} = \frac{1}{|R|} \sum_{R_i \in R} \text{recall}_{\text{local}}(R_i) \quad (5.27)$$

Finally, the *F-score* is computed by combining these global measures according to Equation 2.6.

This metric emphasizes **precise detection** of anomalies over full coverage, as it rewards predictions that are close to real anomalies even if they do not fully cover them. Calculating precision and recall per event and then averaging ensures a balanced evaluation and an intuitive interpretation of model performance.

The parameters used by this metric can be summarized as follows:

- **beta (float)**: weighting factor that controls the relative importance of recall versus precision. (*Default*: 1.0)

This metric is implemented in the module `tsadmetrics.metrics.tem.tstm.AffiliationbasedFScore`, and can be used either directly through the `AffiliationbasedFScore` function (Section 5.2.5.1.1) or through the global evaluation pipeline provided by the `Runner` class (Section 5.2.5.1.2).

Examples of both usage modes are available in the library under: `examples/specific_examples/AffiliationbasedFScore_example.py`.

5.2.5.1.1 Direct evaluation via AffiliationbasedFScore Class

In this example, `y_true` represents the ground-truth binary labels indicating whether each time point is anomalous (1) or normal (0). Similarly, `y_pred` contains the predicted labels produced by an anomaly detection model.

```

1 from tsadmetrics.metrics.tem.tstm.AffiliationbasedFScore import AffiliationbasedFScore
2
3 y_true = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
4 y_pred = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
5
6 metric = AffiliationbasedFScore(beta=1.0)
7 result = metric.compute(y_true, y_pred)
8 print("AffiliationbasedFScore:", result)

```

5.2.5.1.2 Automated evaluation via Runner Class

In this case, the same data sequences (`y_true` and `y_pred`) are organized into a dataset structure that allows batch evaluation across multiple datasets and metrics. The `Runner` class automatically handles the execution of the metric computations and the aggregation of results. For more details on the implementation and usage of the `Runner` class, consult Chapter 6.

```

1 from tsadmetrics.evaluation.Runner import Runner
2
3 dataset_evaluations = [
4     ("dataset1", y_true, y_pred)
5 ]
6
7 metrics = [
8     ("aff_f", {"beta":1.0})
9 ]
10
11 runner = Runner(dataset_evaluations, metrics)
12 results = runner.run()
13 print(results)

```

5.2.5.2 Normalized Affiliation-based F-Score

The Normalized Affiliation-based F-Score combines affiliation-based precision and recall into a single, β -weighted score after applying an **affine normalization** to the affiliation-based precision using a threshold α . Let precision_{aff} and recall_{aff} denote the affiliation-based precision and recall computed from predicted and real anomaly *events* [32]. First, define the normalized (possibly signed) precision:

$$P_u^{aff} = \frac{\text{precision}_{aff} - \alpha}{1 - \alpha}.$$

Then the *signed* F-score is computed as:

$$F_\beta = \frac{(1 + \beta^2) |P_u^{aff}| \text{recall}_{aff}}{\beta^2 P_u^{aff} + \text{recall}_{aff}} \times \text{sign}(P_u^{aff}).$$

Notes. (i) If there are no predicted anomalies, the score is 0. (ii) If $1 - \alpha = 0$, the score is 0 to avoid division by zero. (iii) If the denominator $\beta^2 P_u^{aff} + \text{recall}_{aff}$ is 0, the score is 0.

The parameters used by this metric can be summarized as follows:

- **alpha (float):** normalization threshold applied to affiliation-based precision (must satisfy $\alpha < 1$). (*Default: 0.0*)
- **beta (float):** weighting factor that controls the relative importance of recall versus precision. (*Default: 1.0*)

This metric is implemented in the module `tsadmetrics.metrics.tem.tstm.NormalizedAffiliationbasedFScore`, and can be used either directly through the `NormalizedAffiliationbasedFScore` class (Section 5.2.5.2.1) or through the global evaluation pipeline provided by the `Runner` class (Section 5.2.5.2.2).

Examples of both usage modes are available in the library under: `examples/specific_examples/NormalizedAffiliationbasedFScore_example.py`.

5.2.5.2.1 Direct evaluation via NormalizedAffiliationbasedFScore Class

In this example, `y_true` represents the ground-truth binary labels indicating whether each time point is anomalous (1) or normal (0). Similarly, `y_pred` contains the predicted labels produced by an anomaly detection model.

```

1 from tsadmetrics.metrics.tem.tstm.NormalizedAffiliationbasedFScore import
   NormalizedAffiliationbasedFScore
2
3 y_true = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
4 y_pred = [0,0,0,0,0,1,1,1,0,0,1,1,1,0,0,0,0,0,0,1,1,0,1,1,1,1]
5
6 metric = NormalizedAffiliationbasedFScore(alpha=0.0, beta=1.0)
7 result = metric.compute(y_true, y_pred)
8 print("NormalizedAffiliationbasedFScore:", result)

```

5.2.5.2.2 Automated evaluation via Runner Class

In this case, the same data sequences (`y_true` and `y_pred`) are organized into a dataset structure that allows batch evaluation across multiple datasets and metrics. The `Runner` class automatically handles the execution of the metric computations and the aggregation of results. For more details on the implementation and usage of the `Runner` class, consult Chapter 6.

```

1 from tsadmetrics.evaluation.Runner import Runner
2
3 y_true = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
4 y_pred = [0,0,0,0,0,1,1,1,0,0,1,1,1,0,0,0,0,0,0,1,1,0,1,1,1,1]
5
6 dataset_evaluations = [
7     ("dataset1", y_true, y_pred)
8 ]
9
10 metrics = [
11     ("naff_f", {"alpha":0.0, "beta":1.0})
12 ]
13
14 runner = Runner(dataset_evaluations, metrics)
15 results = runner.run()
16 print(results)

```

5.2.5.3 Pate F-Score

The Pate F-Score metric evaluates how well real anomalous events in R align with predicted binary events in Q , taking temporal proximity into account. For each real anomaly interval $R_i = [a, b]$, two tolerance buffers are defined: an early buffer of length e preceding the interval and a delay buffer of length d following it, together forming an extended region $[a - e, b + d]$.

Detections within the true interval receive full weight, while those in the early and delay buffers are assigned linearly decaying weights according to their temporal distance from the anomaly boundaries. Predictions occurring outside these tolerance regions are regarded as false positives, whereas undetected intervals contribute as false negatives. Weighted precision and recall are then combined

into a F-Score, capturing both the timing accuracy and the overall completeness of the binary detection process [33].

This formulation assumes a discrete prediction signal, where each time step is classified as anomalous or normal, enabling a direct computation of precision and recall from binary labels. The continuous-score extension of this framework is presented in Section 5.2.5.4.

5.2.5.3.1 Event Categorization

Before weighting, points in both R and Q are categorized based on their temporal relationships.

5.2.5.3.2 Predicted anomalous event categories Q_i :

- **True Detection:** Subsegments of Q_i overlapping with any real event R_k ; considered correct detections.
- **Post-Buffer Detection:** Subsegments of Q_i within the post-buffer $(b_k, b_k + d]$ of R_k ; considered late detections.
- **Pre-Buffer Detection:** Subsegments of Q_i within the pre-buffer $[a_k - e, a_k)$; valid only if R_k is partially detected as True Detection and does not overlap the post-buffer of R_{k-1} ; otherwise, classified as Outside.
- **Outside:** Subsegments of Q_i not within any R_k or buffer; considered false positives.

5.2.5.3.3 Real anomalous event categories R_i :

- **Total Missed Anomaly:** Events R_i completely undetected; considered complete false negatives.
- **Partial Missed Anomaly:** Events R_i partially detected; undetected points treated as partial false negatives.

5.2.5.3.4 Point-wise Weight Assignment

After categorization, each temporal point t in R or Q receives a weight $w(t)$ according to its category, used for weighted metric computation:

- **True Detection:** $w^{\text{TP}}(t) = 1$
- **Post-Buffer Detection:** $w^{\text{TP}}(t)$ decreases with distance from the real event, and $w^{\text{FP}}(t) = 1 - w^{\text{TP}}(t)$
- **Pre-Buffer Detection:** $w^{\text{TP}}(t)$ decreases with distance to the start of the event; $w^{\text{FP}}(t) = 1 - w^{\text{TP}}(t)$
- **Outside:** $w^{\text{FP}}(t) = 1$
- **Total Missed Anomaly:** $w^{\text{FN}}(t) = 1$
- **Partial Missed Anomaly:** $w^{\text{FN}}(t)$ adjusted according to coverage fraction and temporal distance within R_k

5.2.5.3.5 Final Score Computation

To comprehensively evaluate the model, **PateFScore** considers multiple values for $e \in E$ and $d \in D$, with

$$E = \{1, 2, \dots, e'\}, \quad D = \{1, 2, \dots, d'\} \quad (5.28)$$

For each pair (e, d) , points are categorized, weighted, and weighted precision and recall are calculated:

$$\text{Precision}_{e,d}(T, P) = \frac{\sum_{t=1}^K w^{\text{TP}}(t)}{\sum_{t=1}^K w^{\text{TP}}(t) + \sum_{t=1}^K w^{\text{FP}}(t)} \quad (5.29)$$

$$\text{Recall}_{e,d}(T, P) = \frac{\sum_{t=1}^K w^{\text{TP}}(t)}{\sum_{t=1}^K w^{\text{TP}}(t) + \sum_{t=1}^K w^{\text{FN}}(t)} \quad (5.30)$$

$$\text{F1-Score}_{e,d}(T, P) = 2 \times \frac{\text{Precision}_{e,d}(T, P) \times \text{Recall}_{e,d}(T, P)}{\text{Precision}_{e,d}(T, P) + \text{Recall}_{e,d}(T, P)} \quad (5.31)$$

Finally, the overall **PateFScore** is obtained by averaging over all (e, d) combinations:

$$\text{PateFScore}(T, P) = \frac{1}{|E| \cdot |D|} \sum_{e \in E, d \in D} \text{F1-Score}_{e,d}(T, P) \quad (5.32)$$

The parameters used by this metric can be summarized as follows:

- **early (int)**: length of the pre-buffer preceding each real anomaly, used to define early detection tolerance. (*Default: 5*)
- **delay (int)**: length of the post-buffer following each real anomaly, used to define delayed detection tolerance. (*Default: 5*)

This metric is implemented in the module `tsadmetrics.metrics.tem.tstm.PateFScore`, and can be used either directly through the **PateFScore** function (Section 5.2.5.3.6) or through the global evaluation pipeline provided by the **Runner** class (Section 5.2.5.3.7).

Examples of both usage modes are available in the library under: `examples/specific_examples/PateFScore_example.py`.

5.2.5.3.6 Direct evaluation via PateFScore Class

In this example, `y_true` represents the ground-truth binary labels indicating whether each time point is anomalous (1) or normal (0). Similarly, `y_pred` contains the predicted labels produced by an anomaly detection model.

```

1 from tsadmetrics.metrics.tem.tstm.PateFScore import PateFScore
2
3 y_true = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
4 y_pred = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
5
6 metric = PateFScore(early=5, delay=5)
7 result = metric.compute(y_true, y_pred)
8 print("PateFScore:", result)

```

5.2.5.3.7 Automated evaluation via Runner Class

In this case, the same data sequences (`y_true` and `y_pred`) are organized into a dataset structure that allows batch evaluation across multiple datasets and metrics. The `Runner` class automatically handles the execution of the metric computations and the aggregation of results. For more details on the implementation and usage of the `Runner` class, consult Chapter 6.

```

1 from tsadmetrics.evaluation.Runner import Runner
2
3 dataset_evaluations = [
4     ("dataset1", y_true, y_pred)
5 ]
6
7 metrics = [
8     ("pate_f1", {"early":5, "delay":5})
9 ]
10
11 runner = Runner(dataset_evaluations, metrics)
12 results = runner.run()
13 print(results)

```

5.2.5.4 Pate

The Pate metric extends the Pate F-Score framework to the evaluation of continuous anomaly scores, allowing a more nuanced assessment of prediction confidence and temporal alignment. Instead of relying on discrete anomaly labels, this version considers the real-valued outputs of an anomaly detector and integrates their magnitude and timing into a unified performance measure [33].

Similar to Pate F-Score, multiple combinations of $e \in E$ and $d \in D$ are considered:

$$E = \{1, 2, \dots, e'\}, \quad D = \{1, 2, \dots, d'\} \quad (5.33)$$

Since P represents a continuous signal, for each (e, d) multiple thresholds θ are applied to generate binary predictions P^θ . From these, predicted ranges Q^θ and real ranges R are constructed, and the same categorization and point-wise weighting process as in Pate F-Score is applied.

For each (e, d) and threshold θ , the weighted precision and recall are computed as:

$$\text{Precision}_{e,d}(\theta) = \frac{\sum_{t=1}^K w^{\text{TP}}(t)}{\sum_{t=1}^K w^{\text{TP}}(t) + \sum_{t=1}^K w^{\text{FP}}(t)} \quad (5.34)$$

$$\text{Recall}_{e,d}(\theta) = \frac{\sum_{t=1}^K w^{\text{TP}}(t)}{\sum_{t=1}^K w^{\text{TP}}(t) + \sum_{t=1}^K w^{\text{FN}}(t)} \quad (5.35)$$

where K denotes the total number of temporal points in T and P^θ . Using these values, a Precision-Recall curve is constructed for each (e, d) pair, and the corresponding area under the curve (AUC-PR) is calculated. The global **Pate** score is then obtained by averaging these areas across all (e, d) combinations, analogous to the averaging of F1-Scores in Pate F-Score:

$$\text{Pate}(T, P) = \frac{1}{|E| \cdot |D|} \sum_{e \in E, d \in D} \text{AUC-PR}_{e,d} \quad (5.36)$$

Here, $|E|$ and $|D|$ represent the number of distinct values considered for e and d , respectively.

The parameters used by this metric can be summarized as follows:

- **early** (`int`): defines the size of the pre-buffer preceding each real anomaly, allowing tolerance for early detections. (*Default: 5*)
- **delay** (`int`): defines the size of the post-buffer following each real anomaly, allowing tolerance for late detections. (*Default: 5*)

This metric is implemented in the module `tsadmetrics.metrics.tem.tstm.Pate`, and can be used either directly through the `Pate` function (Section 5.2.5.4.1) or through the global evaluation pipeline provided by the `Runner` class (Section 5.2.5.4.2).

Examples of both usage modes are available in the library under: `examples/specific_examples/Pate_example.py`.

5.2.5.4.1 Direct evaluation via Pate Class

In this example, `y_true` represents the ground-truth binary labels indicating whether each time point is anomalous (1) or normal (0). Similarly, `y_pred` contains the predicted labels produced by an anomaly detection model.

```

1 from tsadmetrics.metrics.tem.tstm.Pate import Pate
2
3 y_true = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,0,1,1,1,1,1,1,1]
4 y_pred = [0,0,0,0,0,0.4,0.5,0.6,0.7,0.8,0.9,0.95,0.99,0,0,0,0,0,0,0,0,0,0,0,0,0,0]
5
6 metric = Pate(early=5, delay=5)
7 result = metric.compute(y_true, y_pred)
8 print("Pate:", result)

```

5.2.5.4.2 Automated evaluation via Runner Class

In this case, the same data sequences (`y_true` and `y_pred`) are organized into a dataset structure that allows batch evaluation across multiple datasets and metrics. The `Runner` class automatically handles the execution of the metric computations and the aggregation of results. For more details on the implementation and usage of the `Runner` class, consult Chapter 6.

```

1 from tsadmetrics.evaluation.Runner import Runner
2
3 dataset_evaluations = [
4     ("dataset1", y_true, y_pred)
5 ]
6
7 metrics = [
8     ("pate", {"early":5, "delay":5})
9 ]

```

```

10
11 runner = Runner(dataset_evaluations, metrics)
12 results = runner.run()
13 print(results)

```

5.2.5.5 Time-Tolerant F-Score

The Time-Tolerant F-Score metric extends the classic F_β -Score by introducing a temporal tolerance parameter τ , which allows controlled flexibility when comparing predicted and true anomalies over time. Instead of applying a single adjustment to the predicted sequence, two separate adjusted sequences, P'_1 and P'_2 , are created from the original predictions P . These are used respectively to compute *recall* and *precision* (Equations 2.4 and 2.5) [34].

A temporal tolerance parameter τ defines a window of alignment between points in the predicted and true sequences. For the recall-adjusted sequence P'_1 , each point $T_i = 1$ in the true sequence T is examined. If there exists any $P_j = 1$ such that $|i - j| \leq \tau$, then $P'_{1i} = 1$; otherwise, $P'_{1i} = 0$.

Similarly, for the precision-adjusted sequence P'_2 , each predicted point $P_j = 1$ is examined. If there exists any $T_i = 1$ such that $|i - j| \leq \tau$, then $P'_{2j} = 1$; otherwise, $P'_{2j} = 0$.

This approach allows for a controlled temporal margin when comparing predictions with true anomalies, without modifying the original sequences. Importantly, the sets of true positives used to compute precision and recall may differ due to the independent adjustments applied in each case.

The parameters used by this metric can be summarized as follows:

- **t (int)**: defines the temporal tolerance window in both directions when matching predictions with true anomalies. A higher value increases flexibility but reduces temporal precision. (*Default*: 5)
- **beta (float)**: weighting factor that controls the relative importance of recall versus precision. (*Default*: 1.0)

This metric is implemented in the module `tsadmetrics.metrics.tem.tstm.TimeTolerantFScore`, and can be used either directly through the `TimeTolerantFScore` function (Section 5.2.5.5.1) or via the automated evaluation pipeline (Section 5.2.5.5.2).

Examples of both usage modes are available in the library under: `examples/specific_examples/TimeTolerantFScore_example.py`.

5.2.5.5.1 Direct evaluation via TimeTolerantFScore Class

In this example, `y_true` represents the ground-truth binary labels indicating whether each time point is anomalous (1) or normal (0). Similarly, `y_pred` contains the predicted labels produced by an anomaly detection model.

```

1 from tsadmetrics.metrics.tem.tstm.TimeTolerantFScore import TimeTolerantFScore
2
3 y_true = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
4 y_pred = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
5
6 metric = TimeTolerantFScore(t=5, beta=1.0)
7 result = metric.compute(y_true, y_pred)
8 print("TimeTolerantFScore:", result)

```

5.2.5.5.2 Automated evaluation via Runner Class

In this case, the same data sequences (`y_true` and `y_pred`) are organized into a dataset structure that allows batch evaluation across multiple datasets and metrics. The `Runner` class automatically handles the execution of the metric computations and the aggregation of results. For more details on the implementation and usage of the `Runner` class, consult Chapter 6.

```

1 from tsadmetrics.evaluation.Runner import Runner
2
3 dataset_evaluations = [
4     ("dataset1", y_true, y_pred)
5 ]
6
7 metrics = [
8     ("ttf", {"t":5, "beta":1.0})
9 ]
10
11 runner = Runner(dataset_evaluations, metrics)
12 results = runner.run()
13 print(results)

```

5.2.5.6 Volume-under-Surface ROC (VUS-ROC)

The VUS-ROC metric generalizes the classical AUC-ROC by introducing a temporal tolerance parameter l , which smooths the true labels T and allows flexibility in the localization of anomalous events [35]. This smoothing mechanism converts the binary ground-truth sequence into a continuous representation, where nearby points to anomalies are partially weighted, resulting in a more tolerant evaluation of temporal misalignment.

5.2.5.6.1 Label smoothing

Given a binary ground-truth sequence $T \in \{0,1\}^n$, a smoothed vector $T' \in [0,1]^n$ is generated. Each point i coinciding with an anomaly retains $T'_i = 1$, while neighboring points within a distance $\leq l_i$ from any anomalous point are assigned intermediate values $T'_i \in (0,1)$. Points far from anomalies remain 0. This process introduces a tolerance region around true anomalies, reducing harsh penalization for small temporal offsets.

5.2.5.6.2 Redefinition of confusion matrix quantities

Since both P and T' contain continuous values, the classical confusion matrix components are redefined in a soft, element-wise manner:

$$TP' = T' \cdot P, \quad FP' = (1_n - T') \cdot P \quad (5.37)$$

$$TN' = (1_n - T') \cdot (1_n - P), \quad FN' = T' \cdot (1_n - P) \quad (5.38)$$

5.2.5.6.3 Precision and recall computation

Precision is computed as in the standard ROC formulation, while recall is adjusted using the event-existence factor introduced in the Range-based F-Score metric (Section 5.2.1.7), to account for partial coverage of real anomalous ranges.

5.2.5.6.4 Threshold exploration

For a given temporal tolerance l , all integers $l_i \in \{0, 1, \dots, l\}$ are considered. For each l_i :

- A new smoothed vector $T'^{(l_i)}$ is generated.
- Multiple thresholds θ are applied to the predicted scores P to obtain binary predictions P^θ .
- The ROC curve is computed from $T'^{(l_i)}$ and P^θ , producing an AUC value.

5.2.5.6.5 Final metric computation

The final **VusRoc** score is the mean of all AUC-ROC values obtained across tolerance levels l_i :

$$\text{VusRoc}(T, P) = \frac{1}{l+1} \sum_{i=0}^l \text{AUC}_{\text{ROC}}(T'^{(l_i)}, P) \quad (5.39)$$

This formulation effectively integrates the ROC surface over the tolerance parameter, producing a “volume-under-surface” interpretation of detection performance.

5.2.5.6.6 Summary of parameters and implementation.

The parameters used by this metric can be summarized as follows:

- **1 (int)**: defines the maximum temporal smoothing radius used when generating tolerance-extended ground-truth labels. Higher values increase tolerance to temporal misalignment but may blur the precision of anomaly boundaries. (*Default: 4*)

This metric is implemented in the module `tsadmetrics.metrics.tem.tstm.VusRoc`, and can be used either directly through the **VusRoc** function (Section 5.2.5.6.7) or through the global evaluation pipeline provided by the **Runner** class (Section 5.2.5.6.8).

Examples of both usage modes are available in the library under: `examples/specific_examples/VusRoc_example.py`.

5.2.5.6.7 Direct evaluation via **VusRoc** Class

In this example, `y_true` represents the ground-truth binary labels indicating whether each time point is anomalous (1) or normal (0). Similarly, `y_pred` contains the predicted labels produced by an anomaly detection model.

```

1 from tsadmetrics.metrics.tem.tstm.VusRoc import VusRoc
2
3 y_true = [0,0,0,0,0,1,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
4 y_pred = [0,0,0,0,0,0.4,0.5,0.6,0.7,0.8,0.9,0.95,0.99,0,0,0,0,0,0,0,0,0,0,0,0]
5
6 metric = VusRoc(window=4)
7 result = metric.compute(y_true, y_pred)
8 print("VusRoc:", result)

```

5.2.5.6.8 Automated evaluation via Runner Class

In this case, the same data sequences (`y_true` and `y_pred`) are organized into a dataset structure that allows batch evaluation across multiple datasets and metrics. The `Runner` class automatically handles the execution of the metric computations and the aggregation of results. For more details on the implementation and usage of the `Runner` class, consult Chapter 6.

```

1 from tsadmetrics.evaluation.Runner import Runner
2
3 dataset_evaluations = [
4     ("dataset1", y_true, y_pred)
5 ]
6
7 metrics = [
8     ("vus_roc", {"window":4})
9 ]
10
11 runner = Runner(dataset_evaluations, metrics)
12 results = runner.run()
13 print(results)

```

5.2.5.7 Volume-under-Surface PR (VUS-PR)

The VUS-PR metric extends the classical AUC-PR by introducing a temporal tolerance parameter l , which smooths the true labels T and allows a flexible evaluation of anomaly alignment over time [35]. By incorporating temporal smoothing, this metric provides a more robust evaluation for detectors that produce slightly delayed or advanced predictions relative to the real anomalous intervals.

5.2.5.7.1 Label smoothing

Starting from a binary ground-truth sequence $T \in \{0,1\}^n$, a smoothed vector $T' \in [0,1]^n$ is generated. Points corresponding to true anomalies retain $T'_i = 1$, while points within a distance $\leq l_i$ from any anomaly receive intermediate values $T'_i \in (0,1)$. Points far from anomalies remain 0. This smoothing introduces tolerance regions around true anomalies, reducing excessive penalization of minor temporal offsets.

5.2.5.7.2 Confusion matrix adaptation

The continuous values of T' and P allow redefining the confusion matrix entries as:

$$TP' = T' \cdot P, \quad FP' = (1_n - T') \cdot P \quad (5.40)$$

$$TN' = (1_n - T') \cdot (1_n - P), \quad FN' = T' \cdot (1_n - P) \quad (5.41)$$

5.2.5.7.3 Precision and recall

Precision is computed in the standard way, while recall is adjusted using the same partial event detection factor as in the Range-based F-Score metric (Section 5.2.1.7), ensuring partial coverage of anomalies is properly reflected in the evaluation.

5.2.5.7.4 Threshold and smoothing exploration

For a given temporal tolerance l , all integers $l_i \in \{0, 1, \dots, l\}$ are considered. For each l_i :

- Construct the smoothed vector $T'^{(l_i)}$.
- Evaluate all thresholds θ to binarize the predicted scores P into P^θ .
- Compute the Precision–Recall curve using $T'^{(l_i)}$ and P^θ , obtaining the corresponding AUC value.

5.2.5.7.5 Final metric computation

The final **VusPr** score is defined as the mean of the AUC-PR values obtained across all smoothing levels l_i :

$$\text{VusPr}(T, P) = \frac{1}{l+1} \sum_{i=0}^l \text{AUC}_{PR}(T'^{(l_i)}, P) \quad (5.42)$$

This process effectively computes the “volume-under-surface” across multiple tolerance levels, providing a temporally aware generalization of the AUC-PR.

5.2.5.7.6 Summary of parameters and implementation.

The parameters used by this metric can be summarized as follows:

- **1 (int)**: defines the maximum temporal smoothing radius used when generating tolerance-extended ground-truth labels. Higher values increase tolerance to temporal misalignment but reduce temporal precision at anomaly boundaries. (*Default*: 4)

This metric is implemented in the module `tsadmetrics.metrics.tem.tstm.VusPr`, and can be used either directly through the **VusPr** function (Section 5.2.5.7.7) or through the global evaluation pipeline provided by the **Runner** class (Section 5.2.5.7.8).

Examples of both usage modes are available in the library under: `examples/specific_examples/VusPr_example.py`.

5.2.5.7.7 Direct evaluation via **VusPr** Class

In this example, `y_true` represents the ground-truth binary labels indicating whether each time point is anomalous (1) or normal (0). Similarly, `y_pred` contains the predicted labels produced by an anomaly detection model.

```

1 from tsadmetrics.metrics.tem.tstm.VusPr import VusPr
2
3 y_true = [0,0,0,0,0,1,1,1,1,1,1,1,1,0,0,0,0,0,0,0,1,1,1,1,1,1,1]
4 y_pred = [0,0,0,0,0,0.4,0.5,0.6,0.7,0.8,0.9,0.95,0.99,0,0,0,0,0,0,0,0,0,0,0,0,0]
5
6 metric = VusPr(window=4)
7 result = metric.compute(y_true, y_pred)
8 print("VusPr:", result)

```

5.2.5.7.8 Automated evaluation via Runner Class

In this case, the same data sequences (`y_true` and `y_pred`) are organized into a dataset structure that allows batch evaluation across multiple datasets and metrics. The `Runner` class automatically handles the execution of the metric computations and aggregation of results.

```
1 from tsadmetrics.evaluation.Runner import Runner
2
3 dataset_evaluations = [
4     ("dataset1", y_true, y_pred)
5 ]
6
7 metrics = [
8     ("vus_pr", {"window":4})
9 ]
10
11 runner = Runner(dataset_evaluations, metrics)
12 results = runner.run()
13 print(results)
```


Carry out an experimental evaluation of Anomaly Detection Metrics

In this chapter, we present the evaluation framework provided by the **TSADmetrics** library, designed to facilitate the assessment of time series anomaly detection models through a unified and automated process. This framework defines the mechanisms required to compute, organize, and analyze metric results efficiently across multiple datasets and configurations.

The chapter focuses primarily on the **Runner** class, which serves as the central component for executing large-scale evaluations. This class automates the computation of metrics, manages input datasets and configuration files, and generates structured reports summarizing the performance of different models and approaches.

Throughout this chapter, we describe how to perform automated evaluations using **Runner**, including its configuration options, supported input formats, and output structure. Practical examples are provided to illustrate how this component can streamline the benchmarking process and ensure consistency in metric evaluation across experiments.

6.1 Input artifacts and naming conventions

Before introducing the **Runner** parameters, we define the files and labels referenced throughout this chapter so that examples are self-contained.

6.1.1 Result files

CSV files store ground truth labels and model predictions. Two formats are supported:

6.1.1.1 Binary predictions.

```

1 y_true;y_pred
2 0;0
3 0;0
4 ...
5 1;1

```

6.1.1.2 Binary and continuous predictions.

```

1 y_true;y_pred_binary;y_pred_continuous
2 0;0;0.1
3 0;0;0.4
4 ...
5 1;1;0.9

```

Here, `y_pred_binary` contains thresholded predictions, while `y_pred_continuous` contains raw anomaly scores. The example files used later in this chapter (e.g., `results1.csv` and `results2.csv`) follow one of the formats defined in Section 6.1.1.

6.1.2 Metric configuration file

A YAML file enumerating the metrics to compute and, optionally, their parameters, as defined in Section 6.1.2:

```

- adc
- dair
- pakf:
  k: 0.2
- pakf:
  k: 0.4

```

In this example, `adc` and `dair` use default settings, while `pakf` is evaluated twice with different k values.

6.1.3 Global evaluation configuration

A YAML file specifying (i) the datasets to evaluate and (ii) the metric configuration file, as defined in Section 6.1.3:

```

- dataset1: example_input/results1.csv
- dataset2: example_input/results2.csv
- metrics_config: example_input/example_metrics_config.yaml

```

Each dataset identifier (e.g., `dataset1`, `dataset2`) points to a results CSV as defined in Section 6.1.1, and `metrics_config` references the metric configuration file as defined in Section 6.1.2.

6.1.4 Dataset identifiers and paths

Labels such as `dataset1`, `dataset2`, etc. are user-defined identifiers used to reference datasets and results. Paths like `example_input/results1.csv` are relative to the working directory unless otherwise noted.

6.2 Runner Class

The **Runner** class is the central component responsible for automating the evaluation process within the **TSADmetrics** framework. It provides a unified interface for executing multiple metrics over one or more datasets, simplifying large-scale experiments and benchmarking tasks. Through this class, users can configure, execute, and summarize evaluations efficiently—whether by directly providing data and metrics in memory or by defining them in structured configuration files. Additionally, **Runner** can automatically generate summary reports, consolidating all metric results into a single, easily interpretable output.

6.2.1 Parameters

dataset_evaluations (list or str) Defines the datasets to be evaluated and supports multiple input formats:

- **Global configuration file (str)**: If a string is provided and the parameter `metrics` is set to `None`, the value is interpreted as the path to a YAML configuration file that specifies both the datasets and the metrics configuration file to be used during the evaluation. The structure follows the example defined in Section 6.1.3.
- **Direct data (list of tuples)**: Define datasets directly in memory as:

```

1  [
2      ("dataset1", y_true1, (y_pred_binary1, y_pred_continuous1)),
3      ("dataset2", y_true2, (y_pred_binary2, y_pred_continuous2)),
4      ("dataset3", y_true3, y_pred3)
5  ]

```

Here, "dataset1", "dataset2", ... are dataset identifiers chosen by the user; `y_true` holds labels, and the last element supplies predictions (binary and/or continuous).

- **File references (list of tuples)**: Reference external CSVs (following the formats defined in Section 6.1.1) as:

```

1  [
2      ("dataset1", "example_input/results1.csv"),
3      ("dataset2", "example_input/results2.csv")
4  ]

```

Each file must include the column `y_true`, and either:

- both `y_pred_binary` and `y_pred_continuous`, or
- a single `y_pred` column.

metrics (list or str, optional) Defines the set of metrics to be computed during evaluation. Two modes are supported:

- **Direct definition (list of tuples)**:

```

1 [
2     ("pwf", {"beta": 1.0}),
3     ("rpate", {"alpha": 0.5}),
4     ("adc", {})
5 ]

```

- **Configuration file (str):** A YAML file listing metrics and parameters, as defined in Section 6.1.2. This enables flexible batch experiments with multiple parameter settings.

6.2.2 Running evaluation

Once the **Runner** instance has been properly configured with the desired datasets and metrics, the evaluation process is initiated by calling the `run()` method. This method automatically iterates over all datasets and metrics, computes the corresponding results, and organizes them in a structured tabular format.

The method accepts the following parameters:

generate_report (bool, optional) If set to `True`, a report file summarizing the evaluation results will be automatically generated. The default value is `False`.

report_file (str, optional) Specifies the path and filename where the report will be saved when `generate_report=True`. By default, the file is saved as `"evaluation_report.csv"` in the current working directory.

The `run()` method returns a `pandas.DataFrame` object containing the results of the evaluation. Each column corresponds to a metric, while each row represents a dataset and its associated metric values. If `generate_report` is enabled, the same information is also written to the specified CSV file.

Detailed usage examples of the **Runner** class and its `run()` method are provided in Section 6.3.

6.2.3 Reports

When the `run()` method of the **Runner** class is executed with the parameter `generate_report=True`, the evaluation results are automatically passed to the **Report** class. This class is responsible for generating a CSV file that summarizes the computed metrics across all datasets.

The output file contains a tabular representation of the evaluation, as illustrated below:

```

1 dataset;adc;dair;pakf;pakf
2 params;;{'k': 0.2};{'k': 0.4}
3 dataset1;0.5;1.0;0.6666666666666666;0.6666666666666666
4 dataset2;0.5;1.0;0.6666666666666666;0.6666666666666666

```

In this structure:

- The **first row** lists the names of the metrics that were evaluated.
- The **second row (params)** contains the configuration parameters used for each metric instance.

- The **subsequent rows** correspond to the individual datasets evaluated (identified by the user-defined labels introduced in Section 6.1), with each cell containing the computed value for the corresponding metric.

This CSV report provides a clear and standardized summary of all evaluations, facilitating comparison across metrics and datasets, and serving as a direct input for later benchmarking or visualization tasks.

6.3 Evaluation Examples

This section presents several practical examples illustrating how to perform evaluations using the `Runner` class. All examples follow the conventions and file definitions introduced in Section 6.1. Each example demonstrates a different configuration mode, ranging from direct data input in memory to fully automated evaluation through YAML configuration files.

6.3.0.1 Example 1: Direct data evaluation

In this mode, datasets and predictions are defined directly in Python memory. This approach is convenient for quick experiments or small-scale tests.

```

1 from tsadmetrics.evaluation.Runner import Runner
2
3 y_true1 = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,0,1,1,1,1,1,1,1]
4 y_true2 = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,0,1,1,1,1,1,1,1]
5 y_pred1 = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,0,0,0,0,0,0,0,0]
6 y_pred1_cont = [0,0,0,0,0,0.4,0.5,0.6,0.7,0.8,0.9,0.95,0.99,0,0,0,0,0,0,0,0,0,0,0,0,0,0]
7 y_pred2 = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,0,0,0,0,0,0,0,0]
8 y_pred2_cont = [0,0,0,0,0,0.4,0.5,0.6,0.7,0.8,0.9,0.95,0.99,0,0,0,0,0,0,0,0,0,0,0,0,0,0]
9
10 dataset_evaluations = [
11     ("dataset1", y_true1, (y_pred1, y_pred1_cont)),
12     ("dataset2", y_true2, (y_pred2, y_pred2_cont))
13 ]
14
15 metrics = [
16     ("adc", {}),
17     ("dair", {}),
18     ("pakf", {"k":0.2}),
19     ("pakf", {"k":0.4}),
20     ("pakf", {"k":0.5}),
21 ]
22
23 runner = Runner(dataset_evaluations, metrics)
24 results = runner.run(generate_report=True,
25     report_file="./example_output/example_direct_data_report.csv")
26 print(results)

```

6.3.1 Example 2: Direct binary data evaluation

This version simplifies the previous example by using only binary predictions, suitable for metrics that do not require continuous input values.

```

1  from tsadmetrics.evaluation.Runner import Runner
2
3  y_true1 = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
4  y_true2 = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
5  y_pred1 = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,0,0,0,0,0,0,0]
6  y_pred2 = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,0,0,0,0,0,0,0]
7
8  dataset_evaluations = [
9      ("dataset1", y_true1, (y_pred1, y_pred1)),
10     ("dataset2", y_true2, (y_pred2, y_pred2))
11 ]
12
13 metrics = [
14     ("adc", {}),
15     ("dair", {}),
16     ("pakf", {"k": 0.2}),
17     ("pakf", {"k": 0.4})
18 ]
19
20 runner = Runner(dataset_evaluations, metrics)
21 results = runner.run(generate_report=True,
22                      report_file="./example_output/example_direct_single_data_report.csv")
23 print(results)

```

6.3.2 Example 3: Using external result files

Instead of defining datasets in memory, this mode allows referencing existing CSV files containing the prediction results (with formats defined in Section 6.1.1).

```

1  from tsadmetrics.evaluation.Runner import Runner
2
3  dataset_evaluations = [
4      ("dataset1", "example_input/results1.csv"),
5      ("dataset2", "example_input/results2.csv")
6  ]
7
8  metrics = [
9      ("adc", {}),
10     ("dair", {}),
11     ("pakf", {"k": 0.2}),
12     ("pakf", {"k": 0.4})
13 ]
14
15 runner = Runner(dataset_evaluations, metrics)
16 results = runner.run(generate_report=True,
17                      report_file="./example_output/example_file_reference_report.csv")
18 print(results)

```

6.3.3 Example 4: Using a global configuration file

When working with multiple datasets and metrics, it is often convenient to define the full experiment setup in a single YAML configuration file (format defined in Section 6.1.3).

```

1 from tsadmetrics.evaluation.Runner import Runner
2
3 global_config_file = "example_input/example_evaluation_config.yaml"
4
5 runner = Runner(global_config_file)
6 results = runner.run(generate_report=True,
7     report_file="./example_output/example_global_config_file_report.csv")
8 print(results)

```

6.3.4 Example 5: Using a metric configuration file

In this case, the datasets are defined directly, but the metric list is loaded from an external configuration file, as defined in Section 6.1.2.

```

1 from tsadmetrics.evaluation.Runner import Runner
2
3 dataset_evaluations = [
4     ("dataset1", "example_input/results1.csv"),
5     ("dataset2", "example_input/results2.csv")
6 ]
7
8 metrics_file = "example_input/example_metrics_config.yaml"
9
10 runner = Runner(dataset_evaluations, metrics_file)
11 results = runner.run(generate_report=True,
12     report_file="./example_output/example_metric_config_file_report.csv")
13 print(results)

```

6.3.5 Example 6: Computing a single metric manually

For comparison, the following example illustrates how a single metric can be computed directly without using the Runner class.

```

1 from tsadmetrics.metrics.tem.mdpt import PointadjustedFScore
2
3 y_true = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,1,1,1,1,1,1,1]
4 y_pred = [0,0,0,0,0,1,1,1,1,1,1,1,0,0,0,0,0,0,0,0,0,0,0,0,0]
5
6 metric = PointadjustedFScore()
7 result = metric.compute(y_true, y_pred)
8 print(f"PointadjustedFScore: {result}")

```

Each of these examples demonstrates a different way to interact with the TSADmetrics evaluation framework, ranging from lightweight manual computation to fully automated large-scale evaluation pipelines.

Add a New Metric

In this chapter, we describe the process required to extend the **TSADmetrics** library by adding new evaluation metrics. The goal is to enable developers and researchers to contribute additional metrics that can be seamlessly integrated into the existing evaluation framework.

Before detailing the implementation steps, it is important to understand the role of the **Registry** class, which manages the registration and instantiation of all metrics within the library. This class allows each metric to be identified by a unique string identifier — for example, the metric **PointadjustedFScore** is registered under the name "**paf**". The registry automatically detects and registers all metric classes that inherit from the base class **Metric**, so developers only need to implement their metric correctly for it to become available.

7.1 Metric Implementation Process

To add a new metric to the **TSADmetrics** library, follow these steps:

1. **Clone the repository** containing the source code of the library from its official version control system.
2. **Create a new file** within one of the subdirectories of the **metrics/** folder, according to the category of the new metric. The file should be named after the metric to be implemented.
3. **Implement the metric class**, ensuring it inherits from **Metric** and fulfills the following requirements:
 - Define a class attribute **name** (string) containing the identifier of the metric.
 - Define a class attribute **binary_prediction** (boolean) indicating whether the metric operates on binary predictions.
 - Define a dictionary **param_schema** describing the metric's configurable parameters, their types, and default values.
 - Redefine the protected method **_compute**, which performs the actual metric calculation. This method must receive the following arguments:
 - **y_true**: ground truth labels.
 - **y_pred**: predicted labels or scores.

4. **Document the metric** by providing detailed docstrings in the class and its methods, including parameter descriptions, usage examples, and references if applicable.
5. **Create a unit test file** inside the `tests/` directory to verify that the metric produces correct results under various conditions. Tests should validate both expected values and error handling.
6. **Commit and push the changes** to the repository.
7. **Submit a pull request** so that the new metric can be reviewed and integrated into the main branch of the library.

7.2 Example Implementation

The following code shows a generic example of a metric implementation that meets all required specifications:

```

1 from ....base.Metric import Metric
2 import numpy as np
3
4 class NewMetric(Metric):
5     """
6     Example implementation of a new metric for TSADmetrics.
7
8     This metric demonstrates the required structure for extending the
9     library. Each metric must define its name, input type (binary or
10    continuous), parameter schema, and computation logic.
11    """
12
13    name = "new_metric_name"
14    binary_prediction = True
15    param_schema = {
16        "param1": {
17            "default": 1.0,
18            "type": float
19        },
20        "param2": {
21            "default": 5,
22            "type": int
23        }
24    }
25
26    def __init__(self, **kwargs):
27        super().__init__(name="new_metric_name", **kwargs)
28
29    def _compute(self, y_true, y_pred):
30        # Example metric computation logic
31        return 1

```

This example illustrates the minimal structure needed for a new metric. When the library is reloaded, the `Registry` will automatically detect and register the new metric, making it available for use in evaluations, including through the `Runner` class and YAML configurations.

Once these steps are completed, the new metric becomes a fully functional component of the **TSADmetrics** ecosystem, capable of being invoked using its identifier and integrated seamlessly into both manual and automated evaluation pipelines.

Reporting bugs

Feel free to open an issue at Github if anything is not working as expected <https://github.com/kdislab/tsadmetrics/issues>. Merge request are also encouraged, it will be carefully reviewed and merged if everything is all right.

CHAPTER

9

Api Reference

TSADmetrics API Reference

Time Series Anomaly Detection Evaluation Metrics

Pedro Rafael Velasco Priego

Amelia Zafra Gómez

Licensed under the MIT License

CONTENTS

1	Base	1
2	Evaluation	5
3	Metrics	9
3.1	Registry	9
3.2	Metric Types	10
3.2.1	Single-Point Based Metrics (SPM)	10
3.2.2	Temporal Evaluation Metrics (TEM)	13
3.2.2.1	Tolerant Partial Detection Metrics (TPDM)	13
3.2.2.2	Precise Temporal Detection Metrics (PTDM)	17
3.2.2.3	Temporal Matching Evaluation Metrics (TMEM)	23
3.2.2.4	Delay-Penalized Metrics (DPM)	24
3.2.2.5	Temporal Shift-Tolerant Metrics (TSTM)	28
	Python Module Index	33
	Index	35


```
class tsadmetrics.base.Metric.Metric(name=None, config_file=None, **params)
```

Bases: `object`

Base class for time series anomaly detection metrics.

This class provides common functionality for metric configuration, including parameter validation from a YAML configuration file and support for a parameter schema defined in each subclass.

Attributes:

name (str):

Name of the metric instance.

params (dict):

Dictionary of parameters used by the metric.

binary_prediction (bool):

Whether the metric expects binary predictions (True) or continuous scores (False).

Parameters:

name (str, optional):

The name of the metric. If not provided, it defaults to the lowercase name of the subclass.

config_file (str, optional):

Path to a YAML configuration file. Parameters defined in the file under the metrics name will be loaded automatically.

****params:**

Additional parameters passed directly to the metric. These override those loaded from the configuration file.

Raises:

ValueError:

If a required parameter is missing or if the configuration file is not found.

TypeError:

If a parameter does not match its expected type as defined in the schema.

_compute(*y_true, y_pred*)

Compute the value of the metric (core implementation).

This method contains the actual logic of the metric and must be implemented by subclasses. It is automatically called by *compute()* after input validation.

Parameters:

y_true (array-like):
Ground truth binary labels.

y_pred (array-like):
Predicted binary labels.

Returns:

float: The value of the metric.

Raises:

NotImplementedError: If the method is not overridden by a subclass.

`_validate_inputs(y_true, y_pred)`

Validate that y_true and y_pred are valid sequences of the same length.

If binary_prediction = True:

Both y_true and y_pred must be binary (0 or 1).

If binary_prediction = False:

y_true must be binary (0 or 1), y_pred can be continuous values.

Raises:

ValueError: If lengths differ or values are not valid. **TypeError:** If inputs are not array-like.

`compute(y_true, y_pred)`

Compute the value of the metric (wrapper method).

This method performs input validation and then calls the internal *_compute()* method, which contains the actual metric logic.

Important: Subclasses **should not override** this method. Instead, implement *_compute()* to define the behavior of the metric.

Parameters:

y_true (array-like):
Ground truth binary labels.

y_pred (array-like):
Predicted binary labels.

Returns:

float: The value of the metric..

Raises:**NotImplementedError**

If *_compute()* is not implemented by the subclass.

`configure(config_file=None, **params)`

Load and validate metric parameters from a YAML configuration file and/or from explicit keyword arguments.

Parameters:

config_file (str, optional):

Path to the configuration file. If provided, it will load parameters under the section with the metrics name.

****params:**

Parameters passed directly to the metric instance.

Raises:

ValueError:

If a required parameter is not specified or the configuration file is missing.

TypeError:

If a parameter value does not match the expected type.

EVALUATION

```
class tsadmetrics.evaluation.Report.Report
```

Bases: object

```
generate_report(results, output_file)
```

Generate a report from the evaluation results.

Parameters:

results (dict):

Dictionary containing evaluation results.

output_file (str):

Path to the output file where the report will be saved.

```
class tsadmetrics.evaluation.Runner.Runner(dataset_evaluations, metrics=None)
```

Bases: object

Orchestrates the evaluation of datasets using a set of metrics.

The *Runner* class provides functionality to:

- Load datasets from direct data, file references, or a global YAML configuration file.
- Load metrics either directly from a list or from a configuration file.
- Evaluate all datasets against all metrics.
- Optionally generate a report summarizing the evaluation results.

Parameters:

dataset_evaluations (list or str):

Accepted formats:

1. **Global config file (str)** If a string is provided and *metrics* is None, it is assumed to be the path to a configuration file that defines both datasets and metrics.
2. **Direct data (list of tuples)** Example:

```
[
    ("dataset1", y_true1, (y_pred_binary1, y_pred_
    ↪continuous1)),
    ("dataset2", y_true2, (y_pred_binary2, y_pred_
    ↪continuous2)),
```

(continues on next page)

(continued from previous page)

```
("dataset3", y_true3, y_pred3)
]
```

where *y_pred* may be binary or continuous.

3. File references (list of tuples) Example:

```
[
    ("dataset1", "result1.csv"),
    ("dataset2", "result2.csv")
]
```

Each file must contain:

- *y_true*
- Either: * (*y_pred_binary* and *y_pred_continuous*) * or (*y_pred*)

metrics (list or str, optional):

- **List of metrics:** Each element is a tuple of the form [(metric_name, {param_name: value, }),]

Example:

```
[
    ("pwf", {"beta": 1.0}),
    ("rpate", {"alpha": 0.5}),
    ("adc", {})
]
```

- **Config file (str):** Path to a YAML file containing metric definitions.

Attributes:

dataset_evaluations (list):

Loaded datasets in normalized format: (name, y_true, y_pred_binary, y_pred_continuous, y_pred)

metrics (list):

List of metrics with their configurations.

Raises:

ValueError:

If a configuration file is invalid or required fields are missing.

run(*generate_report=False*, *report_file='evaluation_report.csv'*)

Run the evaluation for all datasets and metrics.

Parameters:

generate_report (bool, optional):

If True, generates a report of the evaluation results. Defaults to False.

report_file (str, optional):

Path where the report will be saved if *generate_report* is True. Defaults to *evaluation_report.csv*.

Returns:

pd.DataFrame:

DataFrame structured as follows:

- The **first row** contains the parameters of each metric.
- The **subsequent rows** contain the metric values for each dataset.
- The **index** column represents the dataset names, with the first row labeled as *params*.

Example:

dataset	metric1	metric2
params	{'param1':0.2}	{}
dataset1	0.5	1.0
dataset2	0.125	1.0

3.1 Registry

`class tsadmetrics.metrics.Registry.Registry`

Bases: object

Central registry for anomaly detection metrics.

This class provides a centralized interface to register, retrieve, and load metric classes for anomaly detection tasks.

`classmethod available_metrics()`

List all registered metric names.

Returns:

list[str]: A list of registered metric names.

`classmethod get_metric(name: str, **params) → Metric`

Retrieve and instantiate a registered metric by name.

Args:

name (str): Name of the metric to retrieve. **params: Parameters to initialize the metric instance.

Returns:

Metric: An instance of the requested metric.

Raises:

ValueError: If the metric name is not registered.

`classmethod load_metrics_from_file(filepath: str)`

Load and instantiate metrics from a YAML configuration file.

Args:

filepath (str): Path to the YAML configuration file.

Returns:

list[tuple[str, dict]]: A list of tuples containing the metric name and the parameters used to instantiate it.

`classmethod load_metrics_info_from_file(filepath: str)`

Load metric definitions (names and parameters) from a YAML configuration file.

Args:

filepath (str): Path to the YAML file.

Returns:

list[tuple[str, dict]]: A list of tuples containing the metric name and its parameters, e.g. [("metric_name", {"param1": value, ...}), ...].

Raises:

ValueError: If the YAML file contains invalid entries or unsupported format.

classmethod register(*metric_cls*: Type[Metric])

Register a metric class using its *name* attribute.

Args:

metric_cls (Type[Metric]): The metric class to register.

The class must define a **name** attribute.

Raises:

ValueError: If the metric class does not define a name
attribute or if a metric with the same name is already registered.

tsadmetrics.metrics.Registry.auto_register()

Automatically register all subclasses of **Metric** found in the project.

This function inspects the current inheritance tree of **Metric** and registers each subclass in the central registry.

3.2 Metric Types

3.2.1 Single-Point Based Metrics (SPM)

These metrics evaluate predictions by considering each point independently, without taking into account the temporal context in which anomalies occur. In other words, they treat each instant in isolation, ignoring the continuity or structure of anomalies over time.

class tsadmetrics.metrics.spm.DiceCoefficient(**kwargs)

Bases: *Metric*

Calculate the Dice Coefficient for anomaly detection in time series.

The Dice Coefficient is a similarity measure between the predicted and ground-truth binary anomaly segments. It is mathematically equivalent to the F1-score but derived from a set-theoretic perspective. The metric quantifies the overlap between the predicted anomalies and the actual anomalies, taking values between 0 and 1, where 1 indicates perfect agreement.

The Dice Coefficient is defined as:

$$\text{DiceCoefficient} = \frac{2 \cdot TP}{2 \cdot TP + FP + FN}$$

where:

- *TP* is the number of true positives (correctly detected anomaly points),
- *FP* is the number of false positives (incorrectly predicted anomalies),
- *FN* is the number of false negatives (missed true anomalies).

Notes:

- The Dice Coefficient is symmetric: swapping prediction and ground truth yields the same result.
- If both *y_true* and *y_pred* are all zeros (no anomalies), the metric returns 1.0 to represent perfect agreement in the absence of anomalies.

Reference:

For more information, see the original paper:

<https://www.sciencedirect.com/science/article/pii/S0094576522003162>

Attributes:**name (str):**

Fixed name identifier for this metric: *dicec*.

binary_prediction (bool):

Indicates whether this metric expects binary predictions. Always *True* since it requires binary anomaly scores.

Parameters:

(none)

```
class tsadmetrics.metrics.spm.PointwiseAucPr(**kwargs)
```

Bases: *Metric*

Point-wise Area Under the Precision-Recall Curve (AUC-PR) for anomaly detection.

This metric computes the standard Area Under the Precision-Recall Curve (AUC-PR) in a **point-wise manner**. Each time-series data point is treated independently when calculating precision and recall, making this suitable for anomaly detection tasks where anomalies are labeled at the individual point level.

Reference:

Implementation based on:

<https://link.springer.com/article/10.1007/s10618-023-00988-8>

Attributes:**name (str):**

Fixed name identifier for this metric: *pw_auc_pr*.

binary_prediction (bool):

Indicates whether this metric expects binary predictions. Always *False* since it requires continuous anomaly scores.

```
class tsadmetrics.metrics.spm.PointwiseAucRoc(**kwargs)
```

Bases: *Metric*

Point-wise Area Under the Receiver Operating Characteristic Curve (AUC-ROC) for anomaly detection.

This metric computes the standard Area Under the ROC Curve (AUC-ROC) in a **point-wise manner**. Each time-series data point is treated independently when calculating true positives, false positives, and false negatives. It is widely used to evaluate the ability of anomaly scoring functions to distinguish between normal and anomalous points.

Reference:

Implementation based on:

<https://link.springer.com/article/10.1007/s10618-023-00988-8>

Attributes:**name (str):**

Fixed name identifier for this metric: *pw_auc_roc*.

binary_prediction (bool):

Indicates whether this metric expects binary predictions. Always *False* since it requires continuous anomaly scores.

```
class tsadmetrics.metrics.spm.PointwiseFScore(**kwargs)
```

Bases: *Metric*

Point-wise F-score for anomaly detection in time series.

This metric computes the classical F-score without considering temporal context, treating each time-series point independently. It balances precision and recall according to the configurable parameter *beta*.

Reference:**Implementation based on:**

<https://link.springer.com/article/10.1007/s10618-023-00988-8>

Attributes:**name (str):**

Fixed name identifier for this metric: *pwf*.

binary_prediction (bool):

Indicates whether this metric expects binary predictions. Always *True* since it requires binary anomaly scores.

Parameters:**beta (float):**

The beta value, which determines the weight of precision in the combined score.

```
class tsadmetrics.metrics.spm.PrecisionAtK(**kwargs)
```

Bases: *Metric*

Precision at K (P@K) for anomaly detection in time series.

This metric evaluates how many of the top-*k* points with the highest anomaly scores correspond to true anomalies. It is particularly useful when focusing on identifying the most anomalous points rather than setting a global threshold.

By definition, *k* is automatically set to the number of true anomalies present in *y_true*.

$$k = \sum(y_true)$$

Reference:**Implementation based on:**

<https://link.springer.com/article/10.1007/s10618-023-00988-8>

Attributes:**name (str):**

Fixed name identifier for this metric: *pak*.

binary_prediction (bool):

Indicates whether this metric expects binary predictions. Always *False* since it requires continuous anomaly scores.

3.2.2 Temporal Evaluation Metrics (TEM)

This category includes metrics that incorporate temporal context in the evaluation process. They consider not only whether an anomaly was detected, but also when and how it occurred relative to the original sequence. These metrics are specifically designed for time series anomaly detection and are particularly useful for analyzing one or more properties of the model related to the temporal structure of anomalies, such as their duration, anticipation, coverage, or overlap.

3.2.2.1 Tolerant Partial Detection Metrics (TPDM)

These metrics consider a predicted anomaly valid if it occurs at any point within the interval of a real anomaly. They assume that partial detection is sufficient to signal a potential anomaly, allowing further verification.

```
class tsadmetrics.metrics.tem.tpdm.BalancedPointadjustedFScore(**kwargs)
```

Bases: *Metric*

Balanced point-adjusted F-score for anomaly detection in time series. This metric modifies the standard F-score by applying a temporal adjustment: for each ground-truth anomalous segment, if at least one point is predicted as anomalous, the entire segment is considered correctly detected. Additionally, for each false positive point at time t , all points in the range $[t - \text{floor}(w/2), t + \text{floor}(w/2)]$ are set to 1, which can generate additional false positives. The adjusted predictions are then compared to the ground-truth labels using the standard point-wise F-score formula.

Reference:

For more information, see the original paper:

<https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=10890568>

Attributes:**name (str):**

Fixed name identifier for this metric: *bpaF*.

binary_prediction (bool):

Indicates whether this metric expects binary predictions. Always *True* since it requires binary anomaly scores.

Parameters:**beta (float):**

Weight factor for recall in the F-score (default = 1.0).

w (int):

Temporal window size for expanding each true positive point.

```
class tsadmetrics.metrics.tem.tpdm.CompositeFScore(**kwargs)
```

Bases: *Metric*

Composite F-score for anomaly detection in time series.

This metric combines aspects of the point-wise F-score and the segment-wise F-score. It is defined as the harmonic mean of point-wise precision and segment-wise recall. Using

point-wise precision ensures that false positives are properly penalized, a limitation often found in purely segment-wise metrics.

Reference:

For more information, see the original paper:

<https://doi.org/10.1109/TNNLS.2021.3105827>

Attributes:**name (str):**

Fixed name identifier for this metric: *cf*.

binary_prediction (bool):

Indicates whether this metric expects binary predictions. Always *True* since it requires binary anomaly scores.

```
class tsadmetrics.metrics.tem.tpdm.PointadjustedAucPr(**kwargs)
```

Bases: *Metric*

Point-adjusted Area Under the Precision-Recall Curve (AUC-PR) for anomaly detection.

Unlike the standard point-wise AUC-PR, this variant uses a point-adjusted evaluation:

- Each anomalous segment in *y_true* is considered correctly detected if **at least one point** within that segment is predicted as anomalous.
- Once a segment is detected, all its points are marked as detected in the adjusted prediction.

This adjustment accounts for the fact that detecting any part of an anomalous segment is often sufficient in practice.

Reference:

Implementation based on:

<https://link.springer.com/article/10.1007/s10618-023-00988-8>

Attributes:**name (str):**

Fixed name identifier for this metric: *pa_auc_pr*.

binary_prediction (bool):

Indicates whether this metric expects binary predictions. Always *False* since it requires continuous anomaly scores.

```
class tsadmetrics.metrics.tem.tpdm.PointadjustedAucRoc(**kwargs)
```

Bases: *Metric*

Point-adjusted Area Under the ROC Curve (AUC-ROC) for anomaly detection in time series.

Unlike standard point-wise AUC-ROC, this metric applies **point-adjusted evaluation**:

- Each anomalous segment in *y_true* is considered correctly detected if **at least one point** within that segment is predicted as anomalous.
- Once a segment is detected, all its points are marked as detected in the adjusted predictions.

- Adjusted predictions are then used to compute true positive rate (TPR) and false positive rate (FPR) at multiple thresholds to construct the ROC curve.

Reference:**Implementation based on:**

<https://link.springer.com/article/10.1007/s10618-023-00988-8>

Attributes:**name (str):**

Fixed name identifier for this metric: *pa_auc_pr*.

binary_prediction (bool):

Indicates whether this metric expects binary predictions. Always *False* since it requires continuous anomaly scores.

Raises:**ValueError:**

If *y_true* and *y_anomaly_scores* have mismatched lengths.

TypeError:

If inputs are not array-like.

```
class tsadmetrics.metrics.tem.tpdm.PointadjustedFScore(**kwargs)
```

Bases: *Metric*

Point-adjusted F-score for anomaly detection in time series.

This metric modifies the standard F-score by applying a temporal adjustment to the predictions:

- For each ground-truth anomalous segment, if at least one point is predicted as anomalous, all points in that segment are considered correctly detected.
- The adjusted predictions are then compared to the ground-truth labels using the standard point-wise F-score formula.

Reference:**Implementation based on:**

<https://link.springer.com/article/10.1007/s10618-023-00988-8>

For more information, see the original paper:

<https://doi.org/10.1145/3178876.3185996>

Attributes:**name (str):**

Fixed name identifier for this metric: *paf*.

binary_prediction (bool):

Indicates whether this metric expects binary predictions. Always *True* since it requires binary anomaly scores.

Parameters:**beta (float):**

Weight factor for recall in the F-score (default = 1.0).

```
class tsadmetrics.metrics.tem.tpdm.RangebasedFScore(**kwargs)
```

Bases: *Metric*

Range-based F-score for anomaly detection in time series.

This metric evaluates anomaly detection performance over temporal ranges, combining range-based precision and recall into a harmonic mean. It accounts for positional bias, existence and overlap rewards, and cardinality penalties, allowing fine-grained control over missed detections and false alarms.

Reference:

Implementation based on:

<https://link.springer.com/article/10.1007/s10618-023-00988-8>

For more information, see the original paper:

https://proceedings.neurips.cc/paper_files/paper/2018/file/8f468c873a32bb0619eae2050ba45d1-Paper.pdf

Attributes:

name (str):

Fixed name identifier for this metric: *rbf*.

binary_prediction (bool):

Indicates whether this metric expects binary predictions. Always *True* since it requires binary anomaly scores.

Parameters:

p_alpha (float):

Relative importance of existence reward for precision ($0 \leq \alpha_p \leq 1$).

r_alpha (float):

Relative importance of existence reward for recall ($0 \leq \alpha_r \leq 1$).

p_bias (str):

Positional bias for precision (flat, front, middle, back).

r_bias (str):

Positional bias for recall (flat, front, middle, back).

cardinality_mode (str, optional):

Cardinality factor type (one, reciprocal, gamma).

beta (float):

Weight of precision in the F-score. Default = 1.

```
class tsadmetrics.metrics.tem.tpdm.SegmentwiseFScore(**kwargs)
```

Bases: *Metric*

Segment-wise F-score for anomaly detection in time series.

This metric computes the F-score at the segment level rather than point-wise. Each contiguous segment of anomalies in the ground truth is treated as a unit. - True positive (TP): at least one predicted anomaly within a ground-truth segment. - False negative (FN): no predicted anomaly in a ground-truth segment. - False positive (FP): predicted segment with no overlap with any ground-truth segment.

Reference:

Implementation based on:

<https://link.springer.com/article/10.1007/s10618-023-00988-8>

For more information, see the original paper:

<https://doi.org/10.1145/3219819.3219845>

Attributes:**name (str):**

Fixed name identifier for this metric: *swf*.

binary_prediction (bool):

Indicates whether this metric expects binary predictions. Always *True* since it requires binary anomaly scores.

Parameters:**beta (float): Weight of precision in the harmonic mean.**

Default is 1.0 (balanced F1-score).

3.2.2.2 Precise Temporal Detection Metrics (PTDM)

These metrics require the predicted anomaly to cover a significant portion of the real anomaly duration. They are stricter about temporal accuracy than other types and value precise detection over partial alignment.

```
class tsadmetrics.metrics.tem.ptdm.AverageDetectionCount(**kwargs)
```

Bases: *Metric*

Calculate average detection count for anomaly detection in time series.

This metric computes, for each ground-truth anomalous segment, the percentage of points within that segment that are predicted as anomalous. It then averages these percentages across all true anomaly events, providing an estimate of detection coverage per event.

Reference:**Implementation based on:**

<https://ceur-ws.org/Vol-1226/paper31.pdf>

Attributes:**name (str):**

Fixed name identifier for this metric: *adc*.

binary_prediction (bool):

Indicates whether this metric expects binary predictions. Always *True* since it requires binary anomaly scores.

```
class tsadmetrics.metrics.tem.ptdm.DetectionAccuracyInRange(**kwargs)
```

Bases: *Metric*

Calculate detection accuracy in range for anomaly detection in time series.

This metric measures the proportion of predicted anomaly events that correspond to true anomalies. It is defined as:

$$\text{DAIR} = \frac{EM + DA}{EM + DA + FA}$$

Where:

- **EM (Exact Match):**
Number of predicted anomaly segments that exactly match a true anomaly segment.
- **DA (Detected Anomaly):**
Number of true anomaly points not exactly matched where at least one prediction falls within a window $[i-k, i+k]$ around the true point index i or within the true segment range.
- **FA (False Anomaly):**
Number of predicted anomaly segments that do not overlap any true anomaly segment even within a k -step tolerance window around true points.

Reference:

For more information, see the original paper:

<https://acta.sapientia.ro/content/docs/evaluation-metrics-for-anomaly-detection.pdf>

Attributes:**name (str):**

Fixed name identifier for this metric: *dair*.

binary_prediction (bool):

Indicates whether this metric expects binary predictions. Always *True* since it requires binary anomaly scores.

Parameters:**k (int):**

Half-window size for tolerance around each true anomaly point. A prediction within k time steps of a true point counts toward detection.

```
class tsadmetrics.metrics.tem.ptdm.PointadjustedAtKFScore(**kwargs)
```

Bases: *Metric*

Calculate the Point-adjusted at $K\%$ F-score for anomaly detection in time series.

This metric extends the standard Point-adjusted F-Score by introducing a minimum coverage threshold K for each anomalous segment. For every ground-truth anomalous segment, if at least $K\%$ of its points are predicted as anomalous, the entire segment is marked as detected (all points are marked positive in the adjusted prediction). Otherwise, the segment remains unchanged, preserving only the correctly detected points for point-level evaluation.

In this way, partial detections below the threshold contribute proportionally at the point level, but not at the segment level.

Reference:

Implementation based on:

<https://link.springer.com/article/10.1007/s10618-023-00988-8>

For more information, see the original paper:

<https://ojs.aaai.org/index.php/AAAI/article/view/20680>

Attributes:

name (str):

Fixed name identifier for this metric: *pakf*.

binary_prediction (bool):

Indicates whether this metric expects binary predictions. Always *True* since it requires binary anomaly scores.

Parameters:**k (float):**

The minimum percentage of the anomaly that must be detected to consider the anomaly as detected.

beta (float):

The beta value, which determines the weight of precision in the combined score. Default is 1, which gives equal weight to precision and recall.

```
class tsadmetrics.metrics.tem.ptdm.PointadjustedAtKLFScore(**kwargs)
```

Bases: *Metric*

Calculates the point-adjusted at K% F-score with a tolerance window *l* for anomaly detection in time series. It extends the standard F-score by applying a temporal adjustment: if at least K% of the points within an anomalous segment are predicted as anomalous, the entire segment is considered correctly detected. Additionally, a tolerance window of size *l* is applied around each predicted positive point, so that points within $\pm l$ positions of a true anomaly are also counted as detected, making the metric more robust to small temporal misalignments.

Reference:

For more information, see the original paper:

<https://www.mdpi.com/1424-8220/24/16/5310>

Attributes:**name (str):**

Fixed name identifier for this metric: *paklf*.

binary_prediction (bool):

Indicates whether this metric expects binary predictions. Always *True* since it requires binary anomaly scores.

Parameters:**k (float):**

The minimum percentage of the anomaly that must be detected to consider the anomaly as detected.

l (int):

The tolerance window (in time steps) applied around each predicted positive point. Points within $\pm l$ distance of a true anomaly are treated as correctly detected. Default is 0, meaning no tolerance.

beta (float):

The beta value, which determines the weight of precision in the combined score. Default is 1, which gives equal weight to precision and recall.

```
class tsadmetrics.metrics.tem.ptdm.TimeseriesAwareFScore(**kwargs)
```

Bases: *Metric*

Calculate time series aware F-score for anomaly detection in time series.

This metric is based on the `range_based_f_score`, but introduces two key modifications. First, a predicted anomalous segment is only counted as a true positive if it covers at least a fraction θ of the groundtruth anomaly range. Second, each labeled anomaly is extended by a tolerance window of length δ at its end, within which any overlap contribution decays linearly from full weight down to zero. Unlike the original range-based formulation, this variant omits cardinality and positional bias terms, focusing solely on overlap fraction and endtolerance decay.

Reference:

Implementation based on:

<https://link.springer.com/article/10.1007/s10618-023-00988-8>

For more information, see the original paper:

<https://doi.org/10.1145/3357384.3358118>

Attributes:

name (str):

Fixed name identifier for this metric: *taf*.

binary_prediction (bool):

Indicates whether this metric expects binary predictions. Always *True* since it requires binary anomaly scores.

Parameters:

y_true (np.array):

The ground truth binary labels for the time series data.

y_pred (np.array):

The predicted binary labels for the time series data.

beta (float):

The beta value, which determines the weight of precision in the combined score. Default is 1, which gives equal weight to precision and recall.

alpha (float):

Relative importance of the existence reward versus overlap reward ($0 \leq \alpha \leq 1$).

delta (float):

Tolerance window length at the end of each true anomaly segment.

- If **past_range** is **True**, δ must be a float in $(0, 1]$, representing the **fraction of the segments** length to extend. E.g., $\delta = 0.5$ extends a segment of length 10 by 5 time steps.
- If **past_range** is **False**, δ must be a non-negative integer, representing an **absolute number of** time steps to extend each segment.

theta (float):

Minimum fraction ($0 \leq \theta \leq 1$) of the true anomaly range that must be overlapped by predictions for the segment to count as detected.

past_range (bool):

Determines how δ is interpreted.

- True: δ is treated as a fractional extension of each segments length.
- False: δ is treated as an absolute number of time steps.

class tsadmetrics.metrics.tem.ptdm.TotalDetectedInRange(kwargs)**

Bases: *Metric*

Calculate total detected in range for anomaly detection in time series.

This metric measures the proportion of true anomaly events that are correctly detected. It is defined as:

$$\text{TDIR} = \frac{EM + DA}{EM + DA + MA}$$

Where:

- **EM (Exact Match):**
Number of predicted anomaly segments that exactly match a true anomaly segment.
- **DA (Detected Anomaly):**
Number of true anomaly points not exactly matched where at least one prediction falls within a window $[i-k, i+k]$ around the true point index i or within the true segment range.
- **MA (Missed Anomaly):**
Number of true anomaly segments that do not overlap any predicted anomaly segment even within a k -step tolerance window around true points.

Reference:

For more information, see the original paper:

<https://acta.sapientia.ro/content/docs/evaluation-metrics-for-anomaly-detection.pdf>

Attributes:

name (str):

Fixed name identifier for this metric: *tdir*.

binary_prediction (bool):

Indicates whether this metric expects binary predictions. Always *True* since it requires binary anomaly scores.

Parameters:

k (int):

Half-window size for tolerance around each true anomaly point. A prediction within k time steps of a true point counts toward detection.

class tsadmetrics.metrics.tem.ptdm.WeightedDetectionDifference(kwargs)**

Bases: *Metric*

Calculate weighted detection difference for anomaly detection in time series.

For each true anomaly segment, each point in the segment is assigned a Gaussian weight centered at the segment midpoint, so detections close to the center receive higher reward than detections near the edges. Predicted anomaly points that do not belong to any true anomaly segment are penalized according to their temporal distance to the nearest true anomaly: the farther they are, the larger their false-alarm penalty, capped by the tolerance parameter k .

WS (Weighted Sum) is defined as the sum of Gaussian weights for all predicted anomaly points that fall inside true anomaly segments. WF (False Alarm Weight) is defined as the average penalty weight assigned to false positive predicted points, and FA is the number of those false positive points.

The final score is:

$$WDD = WS - WF \cdot FA$$

Where:

- **WS:**
Sum of Gaussian weights for all predicted anomaly points that fall within any true anomaly segment (extended by delta time steps at the ends).
- **WF:**
Average penalty weight of predicted anomaly points that do not overlap any true anomaly segment.
- **FA (False Anomaly):**
Number of predicted anomaly points that do not overlap any true anomaly segment.

Reference:

For more information, see the original paper:

<https://acta.sapientia.ro/content/docs/evaluation-metrics-for-anomaly-detection.pdf>

Attributes:

name (str):

Fixed name identifier for this metric: *wdd*.

binary_prediction (bool):

Indicates whether this metric expects binary predictions. Always *True* since it requires binary anomaly scores.

Parameters:

k (int):

The maximum number of time steps within which an anomaly must be predicted to be considered detected.

3.2.2.3 Temporal Matching Evaluation Metrics (TMEM)

These metrics measure how well real and predicted anomalies are aligned, penalizing temporal deviations in start, duration, or end of the events.

```
class tsadmetrics.metrics.tem.tmem.AbsoluteDetectionDistance(**kwargs)
```

Bases: *Metric*

Calculate absolute detection distance for anomaly detection in time series.

This metric computes, for each predicted anomaly point that overlaps a ground-truth anomaly segment, the distance from that point to the temporal center of the corresponding segment, normalized by the segment length. It then averages those normalized distances over the predicted anomaly points that correctly overlap a real anomaly event.

If there are no predicted anomaly points, the metric returns 0.0 because there are no detections for which a distance to the center of a ground-truth anomaly segment can be computed. In this edge case, 0.0 should be interpreted as the absence of computable detections, not as a perfectly centered detection.

Reference:

For more information, see the original paper:

<https://ceur-ws.org/Vol-1226/paper31.pdf>

Attributes:

name (str):

Fixed name identifier for this metric: *add*.

binary_prediction (bool):

Indicates whether this metric expects binary predictions. Always *True* since it requires binary anomaly scores.

```
class tsadmetrics.metrics.tem.tmem.EnhancedTimeseriesAwareFScore(**kwargs)
```

Bases: *Metric*

Calculate enhanced time series aware F-score for anomaly detection in time series.

This metric is similar to the range-based F-score in that it accounts for both detection existence and overlap proportion. Additionally, it requires that a significant fraction θ_r of each true anomaly segment be detected, and that a significant fraction θ_p of each predicted segment overlaps with the ground truth. Finally, F-score contributions from each event are weighted by the square root of the true segments length, providing a compromise between point-wise and segment-wise approaches.

Reference:

Implementation based on:

<https://link.springer.com/article/10.1007/s10618-023-00988-8>

For more information, see the original paper:

<https://doi.org/10.1145/3477314.3507024>

Attributes:

name (str):

Fixed name identifier for this metric: *etaf*.

binary_prediction (bool):

Indicates whether this metric expects binary predictions. Always *True* since it requires binary anomaly scores.

Parameters:**theta_p (float):**

Minimum fraction ($0 \leq \theta_p \leq 1$) of a predicted segment that must be overlapped by ground truth to count as detected.

theta_r (float):

Minimum fraction ($0 \leq \theta_r \leq 1$) of a true segment that must be overlapped by predictions to count as detected.

```
class tsadmetrics.metrics.tem.tmem.TemporalDistance(**kwargs)
```

Bases: *Metric*

Calculate temporal distance for anomaly detection in time series.

This metric computes the sum of the distances from each labelled anomaly point to the closest predicted anomaly point, and from each predicted anomaly point to the closest labelled anomaly point.

Reference:**Implementation based on:**

<https://link.springer.com/article/10.1007/s10618-023-00988-8>

For more information, see the original paper:

<https://sciendocom/article/10.2478/ausi-2019-0008>

Attributes:**name (str):**

Fixed name identifier for this metric: *td*.

binary_prediction (bool):

Indicates whether this metric expects binary predictions. Always *True* since it requires binary anomaly scores.

Parameters:**distance (int):**

The distance type parameter for the temporal distance calculation. - 0: Euclidean distance - 1: Squared Euclidean distance

3.2.2.4 Delay-Penalized Metrics (DPM)

These metrics penalize predictions that occur significantly after the real anomaly starts, incentivizing early detection.

```
class tsadmetrics.metrics.tem.dpm.DelayThresholdedPointadjustedFScore(**kwargs)
```

Bases: *Metric*

Calculate delay thresholded point-adjusted F-score for anomaly detection in time series.

This metric is based on the standard F-score, but applies a temporal adjustment to the predictions before computing it. Specifically, for each ground-truth anomalous segment, if at least one point within the first *k* time steps of the segment is predicted as anomalous, all

points in the segment are marked as correctly detected. The adjusted predictions are then compared to the ground-truth labels using the standard point-wise F-score formulation.

Reference:

Implementation based on:

<https://link.springer.com/article/10.1007/s10618-023-00988-8>

For more information, see the original paper:

<https://doi.org/10.1145/3292500.3330680>

Attributes:

name (str):

Fixed name identifier for this metric: *dtmaf*.

binary_prediction (bool):

Indicates whether this metric expects binary predictions. Always *True* since it requires binary anomaly scores.

Parameters:

k (int):

Maximum number of time steps from the start of an anomaly segment within which a prediction must occur for the segment to be considered detected.

beta (float):

The beta value, which determines the weight of precision in the combined score. Default is 1, which gives equal weight to precision and recall.

`class tsadmetrics.metrics.tem.dpm.EarlyDetectionScore(**kwargs)`

Bases: *Metric*

Calculate the Early Detection (ED) score for anomaly detection in time series.

This metric quantifies **how early** an anomaly is detected relative to its true occurrence by evaluating the timing of the first prediction within a defined anomaly window around each ground-truth anomaly.

For every anomaly window $aw_d(a) = [t_b, t_e]$, the ED score is computed as $ed_a = 1 - \frac{\text{pos}_d(T(a)) - b}{e - b}$ if a detection occurs inside the window, and 0 otherwise, where $\text{pos}_d(T(a))$ denotes the position of the first predicted anomaly within the window and b and e represent the windows start and end indices, respectively. Consequently, detections closer to the start of the window yield scores approaching 1 (indicating early detection), while detections near the end yield values closer to 0.

Following the specification in the ADE paper, the anomaly window length is defined as $\|aw_d\| = 0.1 \cdot \|d\| / \|A_d\|$, where $\|d\|$ is the length of the time series and $\|A_d\|$ is the total number of anomalies. Each window is symmetrically centered on the anomaly and bounded within the series limits.

The final ED score is obtained by averaging the individual ed_a values for all ground-truth anomalies, thus providing a single measure of how promptly the model detects anomalies across the entire series.

Reference:

For more information, see the original paper:

<https://ieeexplore.ieee.org/document/7987370>

Attributes:**name (str):**

Fixed name identifier for this metric: *eds*.

binary_prediction (bool):

Indicates whether this metric expects binary predictions. Always *True* since it requires binary anomaly labels.

```
class tsadmetrics.metrics.tem.dpm.LatencySparsityawareFScore(**kwargs)
```

Bases: *Metric*

Calculate latency and sparsity aware F-score for anomaly detection in time series.

This metric is based on the standard F-score, but applies a temporal adjustment to the predictions before computing it. Specifically, for each ground-truth anomalous segment, all points in the segment are marked as correctly detected only after the first true positive is predicted within that segment. This encourages early detection by delaying credit for correct predictions until the anomaly is initially detected. Additionally, to reduce the impact of scattered false positives, predictions are subsampled using a sparsity factor n , so that only one prediction is considered every n time steps. The adjusted predictions are then used to compute the standard point-wise F-score.

Reference:**Implementation based on:**

<https://dl.acm.org/doi/10.1145/3447548.3467174>

For more information, see the original paper:

<https://doi.org/10.1145/3447548.3467174>

Attributes:**name (str):**

Fixed name identifier for this metric: *lsaf*.

binary_prediction (bool):

Indicates whether this metric expects binary predictions. Always *True* since it requires binary anomaly scores.

Parameters:**ni (int):**

The batch size used in the implementation to handle latency and sparsity.

beta (float):

The beta value, which determines the weight of precision in the combined score. Default is 1, which gives equal weight to precision and recall.

```
class tsadmetrics.metrics.tem.dpm.MeanTimeToDetect(**kwargs)
```

Bases: *Metric*

Calculate mean time to detect for anomaly detection in time series.

This metric quantifies the average delay between the start of each ground-truth anomaly segment and the first predicted anomaly point that appears at or after that start index.

For each ground-truth anomaly segment, let i be the index where the segment starts. The implementation searches for the first index j such that $j \geq i$ and the model predicts an

anomaly at j . The predicted anomaly point j does not need to fall inside the corresponding ground-truth anomaly segment.

The detection delay for that event is defined as:

$$\Delta t = j - i$$

The returned MTTD is computed as the sum of the detection delays found for matched ground-truth anomaly starts, divided by the total number of ground-truth anomaly segments.

Ground-truth anomaly segments with no predicted anomaly point at or after their start do not add any delay to the numerator, but they are still included in the denominator. Therefore, missed anomaly segments are not penalized with an increased delay.

If no predicted anomaly points are found at or after any ground-truth anomaly start, the metric emits a warning and returns 0.0 as a fallback value for an undefined/non-informative delay. In this edge case, 0.0 should be interpreted as the absence of computable detections, not as immediate detection.

Reference:

For more information, see the original paper:

<https://dl.acm.org/doi/10.1145/3691338>

Attributes:

name (str):

Fixed name identifier for this metric: *mttd*.

binary_prediction (bool):

Indicates whether this metric expects binary predictions. Always *True* since it requires binary anomaly scores.

```
class tsadmetrics.metrics.tem.dpm.NabScore(**kwargs)
```

Bases: *Metric*

Calculate NAB score for anomaly detection in time series.

This metric rewards early and accurate detections of anomalies while penalizing false positives. For each ground truth anomaly segment, only the first correctly predicted anomaly point contributes positively to the score, with earlier detections receiving higher rewards. In contrast, every false positive prediction contributes negatively.

Reference:

Implementation based on:

<https://link.springer.com/article/10.1007/s10618-023-00988-8>

For more information, see the original paper:

<https://doi.org/10.1109/ICMLA.2015.141>

Attributes:

name (str):

Fixed name identifier for this metric: *nab_score*.

binary_prediction (bool):

Indicates whether this metric expects binary predictions. Always *True* since it requires binary anomaly scores.

3.2.2.5 Temporal Shift-Tolerant Metrics (TSTM)

These metrics allow a temporal tolerance in detecting an anomaly, considering predictions correct if they occur near the real event, even if they do not exactly match its start or end. This flexibility is useful when exact timing is less critical, but detection within a reasonable window is important.

```
class tsadmetrics.metrics.tem.tstm.AffiliationbasedFScore(**kwargs)
```

Bases: *Metric*

Calculate affiliation based F-score for anomaly detection in time series.

This metric combines the affiliation-based precision and recall into a single score using the harmonic mean, adjusted by a weight β to control the relative importance of recall versus precision. Since both precision and recall are distance-based, the F-score reflects a balance between how well predicted anomalies align with true anomalies and vice versa.

Reference:

Implementation based on:

<https://link.springer.com/article/10.1007/s10618-023-00988-8>

For more information, see the original paper:

<https://dl.acm.org/doi/10.1145/3534678.3539339>

Attributes:

name (str):

Fixed name identifier for this metric: *aff_f*.

binary_prediction (bool):

Indicates whether this metric expects binary predictions. Always *True* since it requires binary anomaly scores.

Parameters:

beta (float):

The beta value, which determines the weight of precision in the combined score. Default is 1, which gives equal weight to precision and recall.

```
class tsadmetrics.metrics.tem.tstm.NormalizedAffiliationbasedFScore(**kwargs)
```

Bases: *Metric*

Calculate normalized affiliation-based F-score for anomaly detection in time series.

This metric combines the affiliation-based precision and recall into a single score, weighted by β , but first applies an affine normalization to the affiliation-based precision using a threshold parameter α :

$$P_u^{aff} = \frac{\text{precision}_{aff} - \alpha}{1 - \alpha}$$

Then the (signed) F-score is computed as:

$$F_\beta = \frac{(1 + \beta^2) |P_u^{aff}| \text{recall}_{aff}}{\beta^2 P_u^{aff} + \text{recall}_{aff}} \times \text{sign}(P_u^{aff})$$

Notes:

- If there are no predicted anomalies, the score is 0.

- If $1 - \alpha = 0$, the score is 0 to avoid division by zero.
- If the denominator $\beta^2 P_u^{aff} + \text{recall}_{aff}$ is 0, the score is 0 to avoid division by zero.

Reference:

For more information, see the original paper:

<https://ieeexplore.ieee.org/abstract/document/11099055>

Attributes:**name (str):**

Fixed name identifier for this metric: *naff_f*.

binary_prediction (bool):

Indicates whether this metric expects binary predictions. Always *True* since it requires binary anomaly scores.

Parameters:**beta (float):**

The beta value, which determines the weight of precision in the combined score. Default is 1, which gives equal weight to precision and recall.

alpha (float):

Normalization threshold applied to affiliation-based precision before computing the F-score. Must satisfy $\alpha < 1$. Default is 0.

```
class tsadmetrics.metrics.tem.tstm.Pate(**kwargs)
```

Bases: *Metric*

Calculate PATE score for anomaly detection in time series using real-valued anomaly scores.

This version of PATE operates on continuous anomaly scores rather than binary predictions. It assigns weights to each score according to its temporal proximity to the true anomaly intervals. An early buffer of length *early* and a delay buffer of length *delay* define the tolerance regions before and after each anomaly. High scores within the true interval receive full weight, while scores in the buffer zones are linearly decayed based on their distance from the interval edges. Scores outside all tolerance zones contribute as false positives, and intervals with insufficiently high scores are penalized as false negatives.

The final PATE score aggregates these weighted contributions to produce a smooth, continuous performance measure sensitive to both timing and confidence.

Reference:**Implementation based on:**

<https://arxiv.org/abs/2405.12096>

For more information, see the original paper:

<https://arxiv.org/abs/2405.12096>

Attributes:**name (str):**

Fixed name identifier for this metric: *pate*.

binary_prediction (bool):

Indicates whether this metric expects binary predictions. Always *False* since it requires continuous anomaly scores.

Parameters:**early (int):**

Length of the early buffer zone before each anomaly interval.

delay (int):

Length of the delay buffer zone after each anomaly interval.

```
class tsadmetrics.metrics.tem.tstm.PateFScore(**kwargs)
```

Bases: *Metric*

Calculate PATE F-Score for anomaly detection in time series using binary predictions.

This metric evaluates how well the predicted binary anomalies align with the ground truth, considering temporal proximity around each real anomaly interval. It defines two tolerance zones: an early buffer of length *early* preceding the true interval and a delay buffer of length *delay* following it. Detections within the true interval receive full credit, while those in the buffer zones receive linearly decaying weights depending on their temporal distance from the true anomaly. Predictions outside these regions are treated as false positives, and missed intervals as false negatives.

The weighted contributions of precision and recall are combined into a final F-Score, measuring both timing accuracy and detection completeness.

Reference:**Implementation based on:**

<https://arxiv.org/abs/2405.12096>

For more information, see the original paper:

<https://arxiv.org/abs/2405.12096>

Attributes:**name (str):**

Fixed name identifier for this metric: *pate_f1*.

binary_prediction (bool):

Indicates whether this metric expects binary predictions. Always *True* since it requires binary anomaly scores.

Parameters:**early (int):**

The maximum number of time steps before an anomaly must be predicted to be considered early.

delay (int):

The maximum number of time steps after an anomaly must be predicted to be considered delayed.

```
class tsadmetrics.metrics.tem.tstm.TimeTolerantFScore(**kwargs)
```

Bases: *Metric*

Calculate time tolerant F-score for anomaly detection in time series. This metric is based on the standard F-score, but applies a temporal adjustment to the predictions before

computing it. Specifically, a predicted anomalous point is considered a true positive if it lies within a temporal window of size τ around any ground-truth anomalous point. This allows for small temporal deviations in the predictions to be tolerated. The adjusted predictions are then used to `_compute` the standard point-wise F-Score.

Reference:**Implementation based on:**

<https://link.springer.com/article/10.1007/s10618-023-00988-8>

For more information, see the original paper:

<https://arxiv.org/abs/2008.05788>

Attributes:**name (str):**

Fixed name identifier for this metric: *ttf*.

binary_prediction (bool):

Indicates whether this metric expects binary predictions. Always *True* since it requires binary anomaly scores.

Parameters:**t (int):**

The time tolerance parameter.

beta (float):

The beta value, which determines the weight of precision in the combined score. Default is 1, which gives equal weight to precision and recall.

```
class tsadmetrics.metrics.tem.tstm.VusPr(**kwargs)
```

Bases: *Metric*

Calculate the VUS-PR (Volume Under the PR Surface) score for anomaly detection in time series.

This metric is an extension of the classical AUC-PR, incorporating a temporal tolerance parameter *window* that smooths the binary ground-truth labels. It allows for some flexibility in the detection of anomalies that are temporally close to the true events. The final metric integrates the PR-AUC over several levels of temporal tolerance (from 0 to *window*), yielding a volume under the PR surface.

Reference:**Implementation based on:**

<https://link.springer.com/article/10.1007/s10618-023-00988-8>

For more information, see the original paper:

<https://dl.acm.org/doi/10.14778/3551793.3551830>

Attributes:**name (str):**

Fixed name identifier for this metric: *vus_pr*.

binary_prediction (bool):

Indicates whether this metric expects binary predictions. Always *False* since it requires continuous anomaly scores.

Parameters:

window (int):

Maximum temporal tolerance used to smooth the evaluation. Default is 4.

```
class tsadmetrics.metrics.tem.tstm.VusRoc(**kwargs)
```

Bases: *Metric*

Calculate the VUS-ROC (Volume Under the ROC Surface) score for anomaly detection in time series.

This metric extends the classical AUC-ROC by introducing a temporal tolerance parameter l , which smooths the binary ground-truth labels. The idea is to allow a flexible evaluation that tolerates small misalignments in the detection of anomalies. The final score is computed by integrating the ROC-AUC over different values of the tolerance parameter, from 0 to *window*, thus producing a volume under the ROC surface.

Reference:**Implementation based on:**

<https://link.springer.com/article/10.1007/s10618-023-00988-8>

For more information, see the original paper:

<https://dl.acm.org/doi/10.14778/3551793.3551830>

Attributes:**name (str):**

Fixed name identifier for this metric: *vus_roc*.

binary_prediction (bool):

Indicates whether this metric expects binary predictions. Always *False* since it requires continuous anomaly scores.

Parameters:**window (int):**

Maximum temporal tolerance l used to smooth the evaluation. Default is 4.

PYTHON MODULE INDEX

t

- `tsadmetrics.base.Metric`, 1
- `tsadmetrics.evaluation.Report`, 5
- `tsadmetrics.evaluation.Runner`, 5
- `tsadmetrics.metrics.Registry`, 9
- `tsadmetrics.metrics.spm`, 10
- `tsadmetrics.metrics.tem.dpm`, 24
- `tsadmetrics.metrics.tem.ptdm`, 17
- `tsadmetrics.metrics.tem.tmem`, 23
- `tsadmetrics.metrics.tem.tpdm`, 13
- `tsadmetrics.metrics.tem.tstm`, 28

Symbols

`_compute()` (*tsadmetrics.base.Metric.Metric* method), 1
`_validate_inputs()` (*tsadmetrics.base.Metric.Metric* method), 2

A

`AbsoluteDetectionDistance` (class in *tsadmetrics.metrics.tem.tmem*), 23
`AffiliationbasedFScore` (class in *tsadmetrics.metrics.tem.tstm*), 28
`auto_register()` (in module *tsadmetrics.metrics.Registry*), 10
`available_metrics()` (*tsadmetrics.metrics.Registry.Registry* class method), 9
`AverageDetectionCount` (class in *tsadmetrics.metrics.tem.ptdm*), 17

B

`BalancedPointadjustedFScore` (class in *tsadmetrics.metrics.tem.ptdm*), 13

C

`CompositeFScore` (class in *tsadmetrics.metrics.tem.ptdm*), 13
`compute()` (*tsadmetrics.base.Metric.Metric* method), 2
`configure()` (*tsadmetrics.base.Metric.Metric* method), 2

D

`DelayThresholdedPointadjustedFScore` (class in *tsadmetrics.metrics.tem.dpm*), 24
`DetectionAccuracyInRange` (class in *tsadmetrics.metrics.tem.ptdm*), 17
`DiceCoefficient` (class in *tsadmetrics.metrics.spm*), 10

E

`EarlyDetectionScore` (class in *tsadmetrics.metrics.tem.dpm*), 25
`EnhancedTimeseriesAwareFScore` (class in *tsadmetrics.metrics.tem.tmem*), 23

G

`generate_report()` (*tsadmetrics.evaluation.Report.Report* method), 5
`get_metric()` (*tsadmetrics.metrics.Registry.Registry* class method), 9

L

`LatencySparsityawareFScore` (class in *tsadmetrics.metrics.tem.dpm*), 26
`load_metrics_from_file()` (*tsadmetrics.metrics.Registry.Registry* class method), 9
`load_metrics_info_from_file()` (*tsadmetrics.metrics.Registry.Registry* class method), 9

M

`MeanTimeToDetect` (class in *tsadmetrics.metrics.tem.dpm*), 26
`Metric` (class in *tsadmetrics.base.Metric*), 1
module
 tsadmetrics.base.Metric, 1
 tsadmetrics.evaluation.Report, 5
 tsadmetrics.evaluation.Runner, 5
 tsadmetrics.metrics.Registry, 9
 tsadmetrics.metrics.spm, 10
 tsadmetrics.metrics.tem.dpm, 24
 tsadmetrics.metrics.tem.ptdm, 17
 tsadmetrics.metrics.tem.tmem, 23
 tsadmetrics.metrics.tem.ptdm, 13
 tsadmetrics.metrics.tem.tstm, 28

N

NabScore (class in *tsadmetrics.metrics.tem.dpm*), 27

NormalizedAffiliationbasedFScore (class in *tsadmetrics.metrics.tem.tstm*), 28

P

Pate (class in *tsadmetrics.metrics.tem.tstm*), 29

PateFScore (class in *tsadmetrics.metrics.tem.tstm*), 30

PointadjustedAtKFScore (class in *tsadmetrics.metrics.tem.ptdm*), 18

PointadjustedAtKLFScore (class in *tsadmetrics.metrics.tem.ptdm*), 19

PointadjustedAucPr (class in *tsadmetrics.metrics.tem.tpdm*), 14

PointadjustedAucRoc (class in *tsadmetrics.metrics.tem.tpdm*), 14

PointadjustedFScore (class in *tsadmetrics.metrics.tem.tpdm*), 15

PointwiseAucPr (class in *tsadmetrics.metrics.spm*), 11

PointwiseAucRoc (class in *tsadmetrics.metrics.spm*), 11

PointwiseFScore (class in *tsadmetrics.metrics.spm*), 12

PrecisionAtK (class in *tsadmetrics.metrics.spm*), 12

R

RangebasedFScore (class in *tsadmetrics.metrics.tem.tpdm*), 16

register() (*tsadmetrics.metrics.Registry.Registry* class method), 10

Registry (class in *tsadmetrics.metrics.Registry*), 9

Report (class in *tsadmetrics.evaluation.Report*), 5

run() (*tsadmetrics.evaluation.Runner.Runner* method), 6

Runner (class in *tsadmetrics.evaluation.Runner*), 5

S

SegmentwiseFScore (class in *tsadmetrics.metrics.tem.tpdm*), 16

T

TemporalDistance (class in *tsadmetrics.metrics.tem.tmem*), 24

TimeseriesAwareFScore (class in *tsadmetrics.metrics.tem.ptdm*), 19

TimeTolerantFScore (class in *tsadmetrics.metrics.tem.tstm*), 30

TotalDetectedInRange (class in *tsadmetrics.metrics.tem.ptdm*), 21

tsadmetrics.base.Metric

module, 1

tsadmetrics.evaluation.Report

module, 5

tsadmetrics.evaluation.Runner

module, 5

tsadmetrics.metrics.Registry

module, 9

tsadmetrics.metrics.spm

module, 10

tsadmetrics.metrics.tem.dpm

module, 24

tsadmetrics.metrics.tem.ptdm

module, 17

tsadmetrics.metrics.tem.tmem

module, 23

tsadmetrics.metrics.tem.tpdm

module, 13

tsadmetrics.metrics.tem.tstm

module, 28

V

VusPr (class in *tsadmetrics.metrics.tem.tstm*), 31

VusRoc (class in *tsadmetrics.metrics.tem.tstm*), 32

W

WeightedDetectionDifference (class in *tsadmetrics.metrics.tem.ptdm*), 21

Bibliography

- [1] Yingxu Lai, Zenghui Liu, Zhanwei Song, Yusheng Wang, and Yiwei Gao. Anomaly detection in industrial autonomous decentralized system based on time series. *Simulation Modelling Practice and Theory*, 65:57–71, 2016.
- [2] Bogdan Dumitrescu, Andra Băltoiu, and Stefania Budulan. Anomaly detection in graphs of bank transactions for anti money laundering applications. *IEEE Access*, 10:47699–47714, 2022.
- [3] Xavier Clotet, José Moyano, and Gladys León. A real-time anomaly-based ids for cyber-attack detection at the industrial process level of critical infrastructures. *International Journal of Critical Infrastructure Protection*, 23:11–20, 2018.
- [4] Won-Seok Hwang, Jeong-Han Yun, Jonguk Kim, and Hyoung Chun Kim. Time-series aware precision and recall for anomaly detection: Considering variety of detection result and addressing ambiguous labeling. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management*, CIKM '19, page 2241–2244, New York, NY, USA, 2019. Association for Computing Machinery.
- [5] Alexis Huet, Jose Manuel Navarro, and Dario Rossi. Local evaluation of time series anomaly detection algorithms. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, KDD '22, page 635–645, New York, NY, USA, 2022. Association for Computing Machinery.
- [6] Ahmed Abdulaal, Zhuanghua Liu, and Tomer Lancewicki. Practical approach to asynchronous multivariate time series anomaly detection and localization. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, KDD '21, page 2485–2494, New York, NY, USA, 2021. Association for Computing Machinery.
- [7] Hansheng Ren, Bixiong Xu, Yujing Wang, Chao Yi, Congrui Huang, Xiaoyu Kou, Tony Xing, Mao Yang, Jie Tong, and Qi Zhang. Time-series anomaly detection service at microsoft. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, pages 3009–3017, 2019.
- [8] Jiehui Xu, Haixu Wu, Jianmin Wang, and Mingsheng Long. Anomaly transformer: Time series anomaly detection with association discrepancy. *arXiv preprint arXiv:2110.02642*, 2021.
- [9] Alexander Geiger, Dongyu Liu, Sarah Alnegheimish, Alfredo Cuesta-Infante, and Kalyan Veeramachaneni. Tadgan: Time series anomaly detection using generative adversarial networks. In *2020 IEEE International Conference on Big Data (Big Data)*, pages 33–43. IEEE, 2020.
- [10] Nesime Tatbul, Tae Jun Lee, Stan Zdonik, Mejbah Alam, and Justin Gottschlich. Precision and recall for time series. In Samy Bengio, Hanna M. Wallach, Hugo Larochelle, Kristen Grauman, Nicolò Cesa-Bianchi, and Roman Garnett, editors, *Advances in Neural Information Processing Systems 31: Annual Conference on Neural Information Processing Systems 2018, NeurIPS 2018, December 3-8, 2018, Montréal, Canada*, pages 1924–1934, 2018.

- [11] Sondre Sørbo and Massimiliano Ruocco. Navigating the metric maze: a taxonomy of evaluation metrics for anomaly detection in time series. *Data Mining and Knowledge Discovery*, 38:1–42, 11 2023.
- [12] Jakub Nalepa, Michal Myller, Jacek Andrzejewski, Pawel Benecki, Szymon Piechaczek, and Daniel Kostrzewa. Evaluating algorithms for anomaly detection in satellite telemetry data. *Acta Astronautica*, 198:689–701, 2022.
- [13] Tom Fawcett. An introduction to roc analysis. *Pattern Recognition Letters*, 27(8):861–874, 2006. ROC Analysis in Pattern Recognition.
- [14] John Paparrizos, Yuhao Kang, Paul Boniol, Ruey S. Tsay, Themis Palpanas, and Michael J. Franklin. Tsb-uad: an end-to-end benchmark suite for univariate time-series anomaly detection. *Proc. VLDB Endow.*, 15(8):1697–1711, April 2022.
- [15] Haowen Xu, Wenxiao Chen, Nengwen Zhao, Zeyan Li, Jiahao Bu, Zhihan Li, Ying Liu, Youjian Zhao, Dan Pei, Yang Feng, Jie Chen, Zhaogang Wang, and Honglin Qiao. Unsupervised anomaly detection via variational auto-encoder for seasonal kpis in web applications. In *Proceedings of the 2018 World Wide Web Conference, WWW '18*, page 187–196, Republic and Canton of Geneva, CHE, 2018. International World Wide Web Conferences Steering Committee.
- [16] Debarpan Bhattacharya, Sumanta Mukherjee, Chandramouli Kamanchi, Vijay Ekambaram, A. Jati, and Pankaj Dayama. Towards unbiased evaluation of time-series anomaly detector. 09 2024.
- [17] Kyle Hundman, Valentino Constantinou, Christopher Laporte, Ian Colwell, and Tom Söderström. Detecting spacecraft anomalies using lstms and nonparametric dynamic thresholding. In Yike Guo and Faisal Farooq, editors, *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD 2018, London, UK, August 19-23, 2018*, pages 387–395. ACM, 2018.
- [18] Astha Garg, Wenyu Zhang, Jules Samaran, Ramasamy Savitha, and Chuan-Sheng Foo. An evaluation of anomaly detection and diagnosis in multivariate time series. *IEEE Trans. Neural Networks Learn. Syst.*, 33(6):2508–2517, 2022.
- [19] Liang Dai, Tao Lin, Chang Liu, Bo Jiang, Yanwei Liu, Zhen Xu, and Zhi-Li Zhang. Sdfvae: Static and dynamic factorized vae for anomaly detection of multivariate cdn kpis. In *Proceedings of the Web Conference 2021, WWW '21*, page 3076–3086, New York, NY, USA, 2021. Association for Computing Machinery.
- [20] Hansheng Ren, Bixiong Xu, Yujing Wang, Chao Yi, Congrui Huang, Xiaoyu Kou, Tony Xing, Mao Yang, Jie Tong, and Qi Zhang. Time-series anomaly detection service at microsoft. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD '19*, page 3009–3017, New York, NY, USA, 2019. Association for Computing Machinery.
- [21] Teodora Sandra Buda, Haytham Assem, and Lei Xu. Ade: An ensemble approach for early anomaly detection. In *2017 IFIP/IEEE Symposium on Integrated Network and Service Management (IM)*, pages 442–448, 2017.
- [22] Ahmed Abdulaal, Zhuanghua Liu, and Tomer Lancewicki. Practical approach to asynchronous multivariate time series anomaly detection and localization. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD '21*, page 2485–2494, New York, NY, USA, 2021. Association for Computing Machinery.
- [23] Zahra Zamanzadeh Darban, Geoffrey I. Webb, Shirui Pan, Charu Aggarwal, and Mahsa Salehi. Deep learning for time series anomaly detection: A survey. *ACM Comput. Surv.*, 57(1), October 2024.

- [24] Alexander Lavin and Subutai Ahmad. Evaluating real-time anomaly detection algorithms - the numenta anomaly benchmark. In Tao Li, Lukasz A. Kurgan, Vasile Palade, Randy Goebel, Andreas Holzinger, Karin Verspoor, and M. Arif Wani, editors, *14th IEEE International Conference on Machine Learning and Applications, ICMLA 2015, Miami, FL, USA, December 9-11, 2015*, pages 38–44. IEEE, 2015.
- [25] André Gensler and Bernhard Sick. Novel criteria to measure performance of time series segmentation techniques. In *LWA*, pages 193–204, 2014.
- [26] György Kovács, Gheorghe Sebestyen, and Anca Hangan. Evaluation metrics for anomaly detection algorithms in time-series. *Acta Universitatis Sapientiae, Informatica*, 11:113 – 130, 2019.
- [27] Spyros Rigas, Paraskevi Tzouveli, and Stefanos Kollias. An end-to-end deep learning framework for fault detection in marine machinery. *Sensors*, 24(16), 2024.
- [28] Siwon Kim, Kukjin Choi, Hyun-Soo Choi, Byunghan Lee, and Sungroh Yoon. Towards a rigorous evaluation of time-series anomaly detection. In *Thirty-Sixth AAAI Conference on Artificial Intelligence, AAAI 2022, Thirty-Fourth Conference on Innovative Applications of Artificial Intelligence, IAAI 2022, The Twelveth Symposium on Educational Advances in Artificial Intelligence, EAAI 2022 Virtual Event, February 22 - March 1, 2022*, pages 7194–7201. AAAI Press, 2022.
- [29] Won-Seok Hwang, Jeong-Han Yun, Jonguk Kim, and HyoungChun Kim. Time-series aware precision and recall for anomaly detection: Considering variety of detection result and addressing ambiguous labeling. In Wenwu Zhu, Dacheng Tao, Xueqi Cheng, Peng Cui, Elke A. Rundensteiner, David Carmel, Qi He, and Jeffrey Xu Yu, editors, *Proceedings of the 28th ACM International Conference on Information and Knowledge Management, CIKM 2019, Beijing, China, November 3-7, 2019*, pages 2241–2244. ACM, 2019.
- [30] Won-Seok Hwang, Jeong-Han Yun, Jonguk Kim, and Byung Gil Min. "do you know existing accuracy metrics overrate time-series anomaly detections?". In *Proceedings of the 37th ACM/SIGAPP Symposium on Applied Computing, SAC '22*, page 403–412, New York, NY, USA, 2022. Association for Computing Machinery.
- [31] Alexis Huet, Jose Manuel Navarro, and Dario Rossi. Local evaluation of time series anomaly detection algorithms. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD '22*, page 635–645, New York, NY, USA, 2022. Association for Computing Machinery.
- [32] Zhijie Zhong, Zhiwen Yu, Xing Xi, Yue Xu, Wenming Cao, Yiyuan Yang, Kaixiang Yang, and Jane You. Simad: A simple dissimilarity-based approach for time-series anomaly detection. *IEEE Transactions on Neural Networks and Learning Systems*, pages 1–12, 2025.
- [33] Ramin Ghorbani, Marcel Reinders, and David Tax. Pate: Proximity-aware time series anomaly evaluation. 05 2024.
- [34] Erik Scharwächter and Emmanuel Müller. Statistical Evaluation of Anomaly Detectors for Sequences. In *6th ACM SIGKDD Workshop on Mining and Learning from Time Series (KDD MiLeTS 2020)*, 2020.
- [35] John Paparrizos, Paul Boniol, Themis Palpanas, Ruey Tsay, Aaron Elmore, and Michael Franklin. Volume under the surface: A new accuracy evaluation measure for time-series anomaly detection. *Proceedings of the VLDB Endowment*, 15:2774–2787, 09 2022.